Explore more resources
Altera® Design Hub

Quartus® Prime Pro Edition User Guide

Design Recommendations

Updated for Quartus® Prime Design Suite: **25.1**

This document is part of a collection - Quartus® Prime Pro Edition User Guides - Combined PDF link

Answers to Top FAQs:

Q ***Do you have code templates?***
A Using Provided HDL Templates on page 5

Q ***How do I instantiate IP in HDL?***
A Instantiating IP Cores in HDL on page 6

Q ***How can I infer DSP functions?***
A Inferring Multipliers and DSPs on page 6

Q ***What are coding guidelines?***
A General Coding Guidelines on page 99

Q ***How do optimize clocks?***
A Optimizing Clocking Schemes on page 74

Q ***Can the tool check my HDL?***
A Design Assistant Design Rule Checking on page 74

Contents

1. Answers to Top FAQs 4

2. Recommended HDL Coding Styles 5

2.1. Using Provided HDL Templates 5
2.1.1. Inserting HDL Code from a Provided Template 5
2.2. Instantiating IP Cores in HDL 6
2.3. Inferring Multipliers and DSP Functions 6
2.3.1. Inferring Multipliers 7
2.3.2. Inferring Multiply-Accumulator and Multiply-Adder Functions 8
2.4. Inferring Memory Functions from HDL Code 9
2.4.1. Inferring RAM functions from HDL Code 10
2.4.2. Inferring ROM Functions from HDL Code 28
2.4.3. Inferring Shift Registers in HDL Code 30
2.4.4. Inferring FIFOs in HDL Code 34
2.5. Register and Latch Coding Guidelines 40
2.5.1. Register Power-Up Values 40
2.5.2. Secondary Register Control Signals Such as Clear and Clock Enable 42
2.5.3. Latches 44
2.6. General Coding Guidelines 47
2.6.1. Tri-State Signals 47
2.6.2. Clock Multiplexing 47
2.6.3. Adder Trees 50
2.6.4. State Machine HDL Guidelines 51
2.6.5. Multiplexer HDL Guidelines 58
2.6.6. Cyclic Redundancy Check Functions 61
2.6.7. Comparator HDL Guidelines 63
2.6.8. Counter HDL Guidelines 64
2.7. Designing with Low-Level Primitives 64
2.8. Cross-Module Referencing (XMR) in HDL Code 65
2.9. Using `force` Statements in HDL Code 67
2.10. Recommended HDL Coding Styles Revision History 69

3. Recommended Design Practices 72

3.1. Following Synchronous FPGA Design Practices 72
3.1.1. Implementing Synchronous Designs 72
3.1.2. Asynchronous Design Hazards 73
3.2. HDL Design Guidelines 74
3.2.1. Considerations for the Hyperflex FPGA Architecture 74
3.2.2. Optimizing Combinational Logic 74
3.2.3. Optimizing Clocking Schemes 77
3.2.4. Optimizing Physical Implementation and Timing Closure 82
3.2.5. Optimizing Power Consumption 85
3.2.6. Managing Design Metastability 85
3.3. Use Clock and Register-Control Architectural Features 85
3.3.1. Use Global Reset Resources 85
3.3.2. Use Global Clock Network Resources 94
3.3.3. Use Clock Region Assignments to Optimize Clock Constraints 95

3.3.4. Avoid Asynchronous Register Control Signals..... 97
3.4. Implementing Embedded RAM..... 97
3.5. Design Assistant Design Rule Checking..... 99
3.5.1. Setting Up Design Assistant..... 100
3.5.2. Running Design Assistant During Compilation..... 101
3.5.3. Running Design Assistant in Analysis Mode..... 104
3.5.4. Cross-Probing from Design Assistant..... 107
3.5.5. Managing Design Assistant Rules..... 110
3.5.6. Design Assistant Rule Categories..... 120
3.6. Recommended Design Practices Revision History..... 121

4. Managing Metastability with the Quartus Prime Software..... 125
4.1. Metastability Analysis in the Quartus Prime Software..... 125
4.1.1. Data Synchronization Register Chains..... 126
4.1.2. Identify Synchronizers for Metastability Analysis..... 127
4.1.3. How Timing Constraints Affect Synchronizer Identification and Metastability Analysis..... 127
4.2. Metastability and MTBF Reporting..... 128
4.2.1. Metastability Reports..... 128
4.2.2. Synchronizer Data Toggle Rate in MTBF Calculation..... 130
4.3. MTBF Optimization..... 131
4.3.1. Synchronization Register Chain Length..... 132
4.4. Reducing Metastability Effects..... 132
4.4.1. Apply Complete System-Centric Timing Constraints for the Timing Analyzer... 133
4.4.2. Force the Identification of Synchronization Registers..... 133
4.4.3. Set the Synchronizer Data Toggle Rate..... 133
4.4.4. Optimize Metastability During Fitting..... 134
4.4.5. Increase the Length of Synchronizers to Protect and Optimize..... 134
4.4.6. Increase the Number of Stages Used in Synchronizers..... 134
4.4.7. Select a Faster Speed Grade Device..... 134
4.5. Scripting Support..... 135
4.5.1. Identifying Synchronizers for Metastability Analysis..... 135
4.5.2. Synchronizer Data Toggle Rate in MTBF Calculation 135
4.5.3. report_metastability and Tcl Command..... 135
4.5.4. MTBF Optimization..... 136
4.5.5. Synchronization Register Chain Length..... 136
4.6. Managing Metastability..... 136
4.7. Managing Metastability with the Quartus Prime Software Revision History..... 137

5. Quartus Prime Pro Edition User Guide: Design Recommendations Archive..... 138

A. Quartus Prime Pro Edition User Guides..... 139

1. Answers to Top FAQs

Q Do you have code templates?	A Using Provided HDL Templates on page 5
Q How do I instantiate IP in HDL?	A Instantiating IP Cores in HDL on page 6
Q How can I infer DSP functions?	A Inferring Multipliers and DSPs on page 6
Q What are coding guidelines?	A General Coding Guidelines on page 99
Q How do optimize clocks?	A Optimizing Clocking Schemes on page 74
Q Can the tool check my HDL?	A Design Assistant Design Rule Checking on page 74

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

2. Recommended HDL Coding Styles

This chapter provides Hardware Description Language (HDL) coding style recommendations to ensure optimal synthesis results when targeting Altera FPGA devices.

HDL coding styles have a significant effect on the quality of results for programmable logic designs. Synthesis tools optimize HDL code for both logic utilization and performance; however, synthesis tools cannot interpret the intent of your design. Therefore, the most effective optimizations require conformance to recommended coding styles.

Note: For style recommendations, options, or HDL attributes specific to your synthesis tool, refer to the synthesis tool vendor's documentation.

2.1. Using Provided HDL Templates

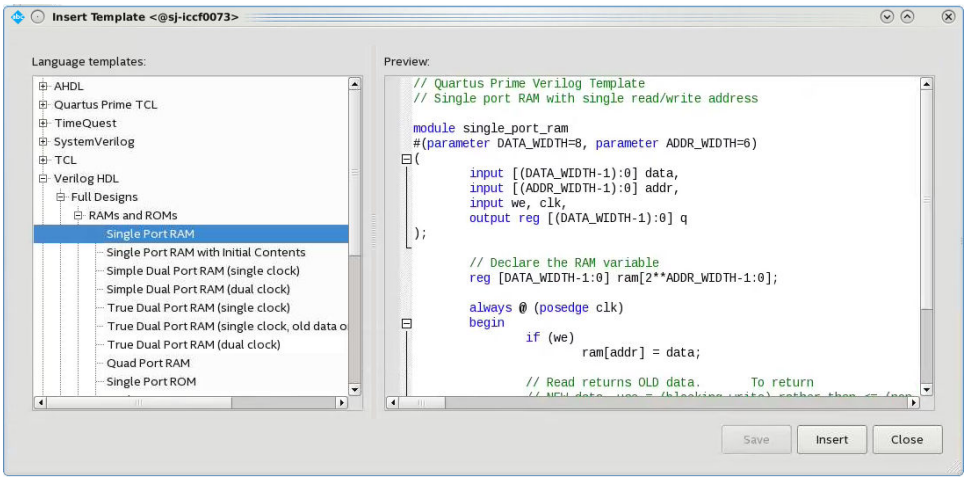
The Quartus Prime software provides templates for Verilog HDL, SystemVerilog, and VHDL templates to start your HDL designs. Many of the HDL examples in this document correspond with the **Full Designs** examples in the **Quartus Prime Templates**. You can insert HDL code into your own design using the templates or examples.

2.1.1. Inserting HDL Code from a Provided Template

1. Click **File ➤ New**.
2. In the **New** dialog box, select the HDL language for the design files: **SystemVerilog HDL File**, **VHDL File**, or **Verilog HDL File**; and click **OK**. A text editor tab with a blank file opens.
3. Right-click the blank file and click **Insert Template**.
4. In the **Insert Template** dialog box, expand the section corresponding to the appropriate HDL, then expand the **Full Designs** section.
5. Select a template. The template now appears in the **Preview** pane.
6. To paste the HDL design into the blank Verilog or VHDL file you created, click **Insert**.
7. Click **Close** to close the **Insert Template** dialog box.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Figure 1. Inserting a RAM Template



Note: Use the Quartus Prime Text Editor to modify the HDL design or save the template as an HDL file to edit in your preferred text editor.

2.2. Instantiating IP Cores in HDL

Altera provides parameterizable IP cores that are optimized for Altera FPGA device architectures. Using IP cores instead of coding your own logic saves valuable design time.

Additionally, the Altera-provided IP cores offer more efficient logic synthesis and device implementation. Scale the IP core's size and specify various options by setting parameters. To instantiate the IP core directly in your HDL file code, invoke the IP core name and define its parameters as you would do for any other module, component, or sub design. Alternatively, you can use the IP Catalog (**Tools ➤ IP Catalog**) and parameter editor GUI to simplify customization of your IP core variation. You can infer or instantiate IP cores that optimize device architecture features, for example:

- Transceivers
- LVDS drivers
- Memory and DSP blocks
- Phase-locked loops (PLLs)
- Double-data rate input/output (DDIO) circuitry

For some types of logic functions, such as memories and DSP functions, you can infer device-specific dedicated architecture blocks instead of instantiating an IP core. Quartus Prime synthesis recognizes certain HDL code structures and automatically infers the appropriate IP core or map directly to device atoms.

2.3. Inferring Multipliers and DSP Functions

The following sections describe how to infer multiplier and DSP functions from generic HDL code, and, if applicable, how to target the dedicated DSP block architecture in Altera FPGA devices.

Related Information

Digital Signal Processing

2.3.1. Inferring Multipliers

To infer multiplier functions, synthesis tools detect multiplier logic and implement this in Altera FPGA IP cores, or map the logic directly to device atoms.

For devices with DSP blocks, Quartus Prime synthesis can implement the function in a DSP block instead of logic, depending on device utilization. The Quartus Prime fitter can also place input and output registers in DSP blocks (that is, perform register packing) to improve performance and area utilization.

The following Verilog HDL and VHDL code examples show that synthesis tools can infer signed and unsigned multipliers as IP cores or DSP block atoms. Each example fits into one DSP block element. In addition, when register packing occurs, no extra logic cells for registers are required.

Example 1. Verilog HDL Unsigned Multiplier

```
module unsigned_mult (out, a, b);
    output [15:0] out;
    input [7:0] a;
    input [7:0] b;
    assign out = a * b;
endmodule
```

Note: The `signed` declaration in Verilog HDL is a feature of the Verilog 2001 Standard.

Example 2. Verilog HDL Signed Multiplier with Input and Output Registers (Pipelining = 2)

```
module signed_mult (out, clk, a, b);
   output [15:0] out;
   input clk;
   input signed [7:0] a;
   input signed [7:0] b;

   reg signed [7:0] a_reg;
   reg signed [7:0] b_reg;
   reg signed [15:0] out;
   wire signed [15:0] mult_out;

   assign mult_out = a_reg * b_reg;

   always @ (posedge clk)
   begin
      a_reg <= a;
      b_reg <= b;
      out <= mult_out;
   end
endmodule
```

Example 3. VHDL Unsigned Multiplier with Input and Output Registers (Pipelining = 2)

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY unsigned_mult IS
   PORT (
```

```
      a: IN UNSIGNED (7 DOWNTO 0);
      b: IN UNSIGNED (7 DOWNTO 0);
      clk: IN STD_LOGIC;
      aclr: IN STD_LOGIC;
      result: OUT UNSIGNED (15 DOWNTO 0)
   );
END unsigned_mult;

ARCHITECTURE rtl OF unsigned_mult IS
   SIGNAL a_reg, b_reg: UNSIGNED (7 DOWNTO 0);
BEGIN
   PROCESS (clk, aclr)
   BEGIN
      IF (aclr ='1') THEN
         a_reg <= (OTHERS => '0');
         b_reg <= (OTHERS => '0');
         result <= (OTHERS => '0');
      ELSIF (rising_edge(clk)) THEN
         a_reg <= a;
         b_reg <= b;
         result <= a_reg * b_reg;
      END IF;
   END PROCESS;
END rtl;
```

Example 4. VHDL Signed Multiplier

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY signed_mult IS
   PORT (
      a: IN SIGNED (7 DOWNTO 0);
      b: IN SIGNED (7 DOWNTO 0);
      result: OUT SIGNED (15 DOWNTO 0)
   );
END signed_mult;

ARCHITECTURE rtl OF signed_mult IS
BEGIN
   result <= a * b;
END rtl;
```

2.3.2. Inferring Multiply-Accumulator and Multiply-Adder Functions

Synthesis tools detect multiply-accumulator or multiply-adder functions, and either implement them as Altera FPGA IP cores or map them directly to device atoms. During placement and routing, the Quartus Prime software places multiply-accumulator and multiply-adder functions in DSP blocks.

Note: Synthesis tools infer multiply-accumulator and multiply-adder functions only if the Altera device family has dedicated DSP blocks that support these functions.

A simple multiply-accumulator consists of a multiplier feeding an addition operator. The addition operator feeds a set of registers that then feeds the second input to the addition operator. A simple multiply-adder consists of two to four multipliers feeding one or two levels of addition, subtraction, or addition/subtraction operators. Addition is always the second-level operator, if it is used. In addition to the multiply-accumulator and multiply-adder, the Quartus Prime Fitter also places input and output registers into the DSP blocks to pack registers and improve performance and area utilization.

Some device families offer additional advanced multiply-adder and accumulator functions, such as complex multiplication, input shift register, or larger multiplications.

The Verilog HDL and VHDL code samples infer multiply-accumulator and multiply-adder functions with input, output, and pipeline registers, as well as an optional asynchronous `clear` signal. Using the three sets of registers provides the best performance through the function, with a latency of three. To reduce latency, remove the registers in your design.

Note: To obtain high performance in DSP designs, use register pipelining and avoid unregistered DSP functions.

Example 5. Verilog HDL Multiply-Accumulator

```
module sum_of_four_multiply_accumulate
   #(parameter INPUT_WIDTH=18, parameter OUTPUT_WIDTH=44)
   (
      input clk, ena,
      input [INPUT_WIDTH-1:0] dataa, datab, datac, datad,
      input [INPUT_WIDTH-1:0] datae, dataf, datag, datah,
      output reg [OUTPUT_WIDTH-1:0] dataout
   );
   // Each product can be up to 2*INPUT_WIDTH bits wide.
   // The sum of four of these products can be up to 2 bits wider.
   wire [2*INPUT_WIDTH+1:0] mult_sum;

   // Store the results of the operations on the current inputs
   assign mult_sum = (dataa * datab + datac * datad) +
                     (datae * dataf + datag * datah);

   // Store the value of the accumulation
   always @ (posedge clk)
   begin
      if (ena == 1)
         begin
            dataout <= dataout + mult_sum;
         end
   end
endmodule
```

2.4. Inferring Memory Functions from HDL Code

The following coding recommendations provide portable examples of generic HDL code targeting dedicated Altera FPGA memory IP cores. However, if you want to use some of the advanced memory features in Altera FPGA devices, consider using the IP core directly so that you can customize the ports and parameters easily.

You can also use the Quartus Prime templates provided in the Quartus Prime software as a starting point.

Table 1. Altera Memory HDL Language Templates

Language	Full Design Name
VHDL	Single-Port RAM Single-Port RAM with Initial Contents Simple Dual-Port RAM (single clock) Simple Dual-Port RAM (dual clock) True Dual-Port RAM (single clock)

continued...

Language	Full Design Name
	True Dual-Port RAM (dual clock) Mixed-Width RAM Mixed-Width True Dual-Port RAM Byte-Enabled Simple Dual-Port RAM Byte-Enabled True Dual-Port RAM Single-Port ROM Dual-Port ROM
Verilog HDL	Single-Port RAM Single-Port RAM with Initial Contents Simple Dual-Port RAM (single clock) Simple Dual-Port RAM (dual clock) True Dual-Port RAM (single clock) True Dual-Port RAM (dual clock) Single-Port ROM Dual-Port ROM
SystemVerilog	Mixed-Width Port RAM Mixed-Width True Dual-Port RAM Mixed-Width True Dual-Port RAM (new data on same port read during write) Byte-Enabled Simple Dual Port RAM Byte-Enabled True Dual-Port RAM

Related Information

- Introduction to Altera® FPGA IP Cores
- Hyperflex™ Architecture High-Performance Design Handbook
- Arria® 10 Core Fabric and General Purpose I/Os Handbook

2.4.1. Inferring RAM functions from HDL Code

To infer RAM functions, synthesis tools recognize certain types of HDL code and map the detected code to technology-specific implementations. For device families that have dedicated RAM blocks, the Quartus Prime software uses an Altera FPGA IP core to target the device memory architecture.

Synthesis tools typically consider all signals and variables that have a multi-dimensional array type and then create a RAM block, if applicable. This is based on the way the signals or variables are assigned or referenced in the HDL source description.

Standard synthesis tools recognize single-port and simple dual-port (one read port and one write port) RAM blocks. Some synthesis tools (such as the Quartus Prime software) also recognize true dual-port (two read ports and two write ports) RAM blocks that map to the memory blocks in certain Altera FPGA devices.

Some tools (such as the Quartus Prime software) also infer memory blocks for array variables and signals that are referenced (read/written) by two indexes, to recognize mixed-width and byte-enabled RAMs for certain coding styles.

Note: If your design contains a RAM block that your synthesis tool does not recognize and infer, the design might require a large amount of system memory that can potentially cause compilation problems.

2.4.1.1. Use Synchronous Memory Blocks

Memory blocks in Altera FPGA are synchronous. Therefore, RAM designs must be synchronous to map directly into dedicated memory blocks. For these devices, Quartus Prime synthesis implements asynchronous memory logic in regular logic cells.

Synchronous memory offers several advantages over asynchronous memory, including higher frequencies and thus higher memory bandwidth, increased reliability, and less standby power. To convert asynchronous memory, move registers from the datapath into the memory block.

A memory block is synchronous if it has one of the following read behaviors:

- Memory read occurs in a Verilog HDL `always` block with a `clock` signal or a VHDL clocked process. The recommended coding style for synchronous memories is to create your design with a registered read output.
- Memory read occurs outside a clocked block, but there is a synchronous read address (that is, the address used in the read statement is registered). Synthesis does not always infer this logic as a memory block, or may require external bypass logic, depending on the target device architecture. Avoid this coding style for synchronous memories.

Note: The synchronous memory structures in Altera FPGA devices can differ from the structures in other vendors' devices. For best results, match your design to the target device architecture.

This chapter provides coding recommendations for various memory types. All the examples in this document are synchronous to ensure that they can be directly mapped into the dedicated memory architecture available in Altera FPGAs.

2.4.1.2. Avoid Unsupported Reset and Control Conditions

To ensure correct implementation of HDL code in the target device architecture, avoid unsupported reset conditions or other control logic that does not exist in the device architecture.

The RAM contents of Altera FPGA memory blocks cannot be cleared with a `reset` signal during device operation. If your HDL code describes a RAM with a `reset` signal for the RAM contents, the logic is implemented in regular logic cells instead of a memory block. Do not place RAM read or write operations in an `always` block or `process` block with a `reset` signal. To specify memory contents, initialize the memory or write the data to the RAM during device operation.

In addition to reset signals, other control logic can prevent synthesis from inferring memory logic as a memory block. For example, if you use a clock enable on the read address registers, you can alter the output latch of the RAM, resulting in the synthesized RAM result not matching the HDL description. Use the address stall feature as a read address clock enable to avoid this limitation. Check the documentation for your FPGA device to ensure that your code matches the hardware available in the device.

Example 6. Verilog RAM with Reset Signal that Clears RAM Contents: Not Supported in Device Architecture

```
module clear_ram
(
    input clock, reset, we,
    input [7:0] data_in,
    input [4:0] address,
    output reg [7:0] data_out
);

    reg [7:0] mem [0:31];
    integer i;

    always @ (posedge clock or posedge reset)
    begin
        if (reset == 1'b1)
            mem[address] <= 0;
        else if (we == 1'b1)
            mem[address] <= data_in;

        data_out <= mem[address];
    end
endmodule
```

Related Information

Specifying Initial Memory Contents at Power-Up on page 25

2.4.1.3. Check Read-During-Write Behavior

Ensure the read-during-write behavior of the memory block described in your HDL design is consistent with your target device architecture.

Your HDL source code specifies the memory behavior when you read and write from the same memory address in the same clock cycle. The read returns either the old data at the address, or the new data written to the address. This is referred to as the read-during-write behavior of the memory block. Altera FPGA memory blocks have different read-during-write behavior depending on the target device family, memory mode, and block type.

Synthesis tools preserve the functionality described in your source code. Therefore, if your source code specifies unsupported read-during-write behavior for the RAM blocks, the Quartus Prime software implements the logic in regular logic cells as opposed to the dedicated RAM hardware.

Example 7. Continuous read in HDL code

One common problem occurs when there is a continuous read in the HDL code, as in the following examples. Avoid using these coding styles:

```
//Verilog HDL concurrent signal assignment
assign q = ram[raddr_reg];
```

```
-- VHDL concurrent signal assignment
q <= ram(raddr_reg);
```

This type of HDL implies that when a write operation takes place, the read immediately reflects the new data at the address independent of the read clock, which is the behavior of asynchronous memory blocks. Synthesis cannot directly map this behavior to a synchronous memory block. If the write clock and read clock are the

same, synthesis can infer memory blocks and add extra bypass logic so that the device behavior matches the HDL behavior. If the write and read clocks are different, synthesis cannot reliably add bypass logic, so it implements the logic in regular logic cells instead of dedicated RAM blocks. The examples in the following sections discuss some of these differences for read-during-write conditions.

In addition, the MLAB memories in certain device logic array blocks (LABs) does not easily support old data or new data behavior for a read-during-write in the dedicated device architecture. Implementing the extra logic to support this behavior significantly reduces timing performance through the memory.

Note: For best performance in MLAB memories, ensure that your design does not depend on the read data during a write operation.

In many synthesis tools, you can declare that the read-during-write behavior is not important to your design (for example, if you never read from the same address to which you write in the same clock cycle). In Quartus Prime Pro Edition synthesis, set the synthesis attribute `ramstyle` to `no_rw_check` to allow Quartus Prime software to define the read-during-write behavior of a RAM, rather than use the behavior specified by your HDL code. This attribute can prevent the synthesis tool from using extra logic to implement the memory block, or can allow memory inference when it would otherwise be impossible.

2.4.1.4. Controlling RAM Inference and Implementation

Quartus Prime synthesis provides options to control RAM inference and implementation for Altera FPGA devices with synchronous memory blocks. Synthesis tools usually do not infer small RAM blocks because implementing small RAM blocks is more efficient if using the registers in regular logic.

To direct the Quartus Prime software to infer RAM blocks globally for all sizes, enable the **Allow Any RAM Size for Recognition** option in the **Advanced Analysis & Synthesis Settings** dialog box (**Assignments ➤ Settings ➤ Compiler Settings ➤ Synthesis Settings (Advanced)**).

Alternatively, use the `ramstyle` RTL attribute to specify how an inferred RAM is implemented, including the type of memory block or the use of regular logic instead of a dedicated memory block. Quartus Prime synthesis does not map inferred memory into MLABs unless the HDL code specifies the appropriate `ramstyle` attribute, although the Fitter may map some memories to MLABs.

Set the `ramstyle` attribute in the RTL or in the `.qsf` file.

```
(* ramstyle = "mlab" *) my_shift_reg
```

```
set_instance_assignment -name RAMSTYLE_ATTRIBUTE LOGIC -to ram
```

. This attribute controls the implementation of an inferred memory. Apply the attribute to a variable declaration that infers a RAM, ROM, or shift-register. Legal values are: `"M9K"`, `"M10K"`, `"M20K"`, `"M144K"`, `"MLAB"`, `"no_rw_check"`, `"logic"`

You can also specify the maximum depth of memory blocks for RAM or ROM inference in RTL. Specify the `max_depth` synthesis attribute to the declaration of a variable that represents a RAM or ROM in your design file. For example:

```
// Limit the depth of the memory blocks implement "ram" to 512
// This forces the Quartus Prime software to use two M512 blocks instead of one
M4K block to implement this RAM
(* max_depth = 512 *) reg [7:0] ram[0:1023];
```

In addition, you can specify the `no_ram` synthesis attribute to prevent RAM inference on a specific array. For example:

```
 (* no_ram *) logic [11:0] my_array [0:12];
```

2.4.1.5. Single-Clock Synchronous RAM with Old Data Read-During-Write Behavior

The code examples in this section show Verilog HDL and VHDL code that infers simple dual-port, single-clock synchronous RAM. Single-port RAM blocks use a similar coding style.

The read-during-write behavior in these examples is to read the old data at the memory address. For best performance in MLAB memories, use the appropriate attribute so that your design does not depend on the read data during a write operation. The simple dual-port RAM code samples map directly into Altera synchronous memory.

Single-port versions of memory blocks (that is, using the same read address and write address signals) allow better RAM utilization than dual-port memory blocks, depending on the device family. Refer to the appropriate device handbook for recommendations on your target device.

Example 8. Verilog HDL Single-Clock, Simple Dual-Port Synchronous RAM with Old Data Read-During-Write Behavior

```
module single_clk_ram(
    output reg [7:0] q,
    input [7:0] d,
    input [4:0] write_address, read_address,
    input we, clk
);
    reg [7:0] mem [31:0];

    always @ (posedge clk) begin
        if (we)
            mem[write_address] <= d;
        q <= mem[read_address]; // q doesn't get d in this clock cycle
    end
endmodule
```

Example 9. VHDL Single-Clock, Simple Dual-Port Synchronous RAM with Old Data Read-During-Write Behavior

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY single_clock_ram IS
    PORT (
        clock: IN STD_LOGIC;
        data: IN STD_LOGIC_VECTOR (7 DOWNTO 0);
        write_address: IN INTEGER RANGE 0 to 31;
```

```
        read_address: IN INTEGER RANGE 0 to 31;
        we: IN STD_LOGIC;
        q: OUT STD_LOGIC_VECTOR (7 DOWNTO 0)
    );
END single_clock_ram;

ARCHITECTURE rtl OF single_clock_ram IS
    TYPE MEM IS ARRAY(0 TO 31) OF STD_LOGIC_VECTOR(7 DOWNTO 0);
    SIGNAL ram_block: MEM;
BEGIN
    PROCESS (clock)
    BEGIN
        IF (rising_edge(clock)) THEN
            IF (we = '1') THEN
                ram_block(write_address) <= data;
            END IF;
            q <= ram_block(read_address);
            -- VHDL semantics imply that q doesn't get data
            -- in this clock cycle
        END IF;
    END PROCESS;
END rtl;
```

Note: The small size of this `single_clock_ram` causes the Compiler to infer the memory as MLAB memory blocks, rather than M20K memory blocks. If `single_clock_ram` specifies a larger width, the Compiler infers the memory as M20K memory blocks.

2.4.1.6. Single-Clock Synchronous RAM with New Data Read-During-Write Behavior

The examples in this section describe RAM blocks in which the read-during-write behavior returns the new value being written at the memory address.

To implement this behavior in the target device, synthesis tools add bypass logic around the RAM block. This bypass logic increases the area utilization of the design, and decreases the performance if the RAM block is part of the design's critical path. If the device memory supports new data read-during-write behavior when in single-port mode (same clock, same read address, and same write address), the Verilog memory block doesn't require any bypass logic. Refer to the appropriate device handbook for specifications on your target device.

The following examples use a blocking assignment for the write so that the data is assigned intermediately.

Example 10. Verilog HDL Single-Clock, Simple Dual-Port Synchronous RAM with New Data Read-During-Write Behavior

```
module single_clock_wr_ram(
    output reg [7:0] q,
    input [7:0] d,
    input [6:0] write_address, read_address,
    input we, clk
);
    reg [7:0] mem [127:0];

    always @ (posedge clk) begin
        if (we)
            mem[write_address] = d;
        q = mem[read_address]; // q does get d in this clock
                               // cycle if we is high
    end
endmodule
```

Example 11. VHDL Single-Clock, Simple Dual-Port Synchronous RAM with New Data Read-During-Write Behavior:

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY single_clock_ram IS
    PORT (
        clock: IN STD_LOGIC;
        data: IN STD_LOGIC_VECTOR (2 DOWNTO 0);
        write_address: IN INTEGER RANGE 0 to 31;
        read_address: IN INTEGER RANGE 0 to 31;
        we: IN STD_LOGIC;
        q: OUT STD_LOGIC_VECTOR (2 DOWNTO 0)
    );
END single_clock_ram;

ARCHITECTURE rtl OF single_clock_ram IS
    TYPE MEM IS ARRAY(0 TO 31) OF STD_LOGIC_VECTOR(2 DOWNTO 0);

BEGIN
    PROCESS (clock)
    VARIABLE ram_block: MEM;
    BEGIN
        IF (rising_edge(clock)) THEN
            IF (we = '1') THEN
                ram_block(write_address) := data;
            END IF;
            q <= ram_block(read_address);
            -- VHDL semantics imply that q doesn't get data
            -- in this clock cycle
        END IF;
    END PROCESS;
END rtl;
```

It is possible to create a single-clock RAM by using an `assign` statement to read the address of `mem` and create the output `q`. By itself, the RTL describes new data read-during-write behavior. However, if the RAM output feeds a register in another hierarchy, a read-during-write results in the old data. Synthesis tools may not infer a RAM block if the tool cannot determine which behavior is described, such as when the memory feeds a hard hierarchical partition boundary. Avoid this type of RTL.

Example 12. Avoid Verilog Coding Style with Vague read-during-write Behavior

```
reg [7:0] mem [127:0];
reg [6:0] read_address_reg;

always @ (posedge clk) begin
    if (we)
        mem[write_address] <= d;
    read_address_reg <= read_address;
end
assign q = mem[read_address_reg];
```

Example 13. Avoid VHDL Coding Style with Vague read-during-write Behavior

The following example uses a concurrent signal assignment to read from the RAM, and presents a similar behavior.

```
ARCHITECTURE rtl OF single_clock_rw_ram IS
    TYPE MEM IS ARRAY(0 TO 31) OF STD_LOGIC_VECTOR(2 DOWNTO 0);
    SIGNAL ram_block: MEM;
    SIGNAL read_address_reg: INTEGER RANGE 0 to 31;
BEGIN
    PROCESS (clock)
```

```
    BEGIN
        IF (rising_edge(clock)) THEN
            IF (we = '1') THEN
                ram_block(write_address) <= data;
            END IF;
            read_address_reg <= read_address;
        END IF;
    END PROCESS;
    q <= ram_block(read_address_reg);
END rtl;
```

2.4.1.7. Simple Dual-Port, Dual-Clock Synchronous RAM

With dual-clock designs, synthesis tools cannot accurately infer the read-during-write behavior because it depends on the timing of the two clocks within the target device. Therefore, the read-during-write behavior of the synthesized design is undefined and may differ from your original HDL code.

Example 14. Verilog HDL Simple Dual-Port, Dual-Clock Synchronous RAM

```
module simple_dual_port_ram_dual_clock
#(parameter DATA_WIDTH=8, parameter ADDR_WIDTH=6)
(
    input [(DATA_WIDTH-1):0] data,
    input [(ADDR_WIDTH-1):0] read_addr, write_addr,
    input we, read_clock, write_clock,
    output reg [(DATA_WIDTH-1):0] q
);

    // Declare the RAM variable
    reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];

    always @ (posedge write_clock)
    begin
        // Write
        if (we)
            ram[write_addr] <= data;
    end

    always @ (posedge read_clock)
    begin
        // Read
        q <= ram[read_addr];
    end

endmodule
```

Example 15. VHDL Simple Dual-Port, Dual-Clock Synchronous RAM

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
ENTITY dual_clock_ram IS
    PORT (
        clock1, clock2: IN STD_LOGIC;
        data: IN STD_LOGIC_VECTOR (3 DOWNTO 0);
        write_address: IN INTEGER RANGE 0 to 31;
        read_address: IN INTEGER RANGE 0 to 31;
        we: IN STD_LOGIC;
        q: OUT STD_LOGIC_VECTOR (3 DOWNTO 0)
    );
END dual_clock_ram;
ARCHITECTURE rtl OF dual_clock_ram IS
    TYPE MEM IS ARRAY(0 TO 31) OF STD_LOGIC_VECTOR(3 DOWNTO 0);
    SIGNAL ram_block: MEM;
```

```
    SIGNAL read_address_reg : INTEGER RANGE 0 to 31;
BEGIN
    PROCESS (clock1)
    BEGIN
        IF (rising_edge(clock1)) THEN
            IF (we = '1') THEN
                ram_block(write_address) <= data;
            END IF;
        END IF;
    END PROCESS;
    PROCESS (clock2)
    BEGIN
        IF (rising_edge(clock2)) THEN
            q <= ram_block(read_address_reg);
            read_address_reg <= read_address;
        END IF;
    END PROCESS;
END rtl;
```

Related Information

Check Read-During-Write Behavior on page 12

2.4.1.8. True Dual-Port Synchronous RAM

The code examples in this section show Verilog HDL and VHDL code that infers true dual-port synchronous RAM. Different synthesis tools may differ in their support for these types of memories.

Altera FPGA synchronous memory blocks have two independent address ports, allowing for operations on two unique addresses simultaneously. A read operation and a write operation can share the same port if they share the same address.

The Quartus Prime software infers true dual-port RAMs in Verilog HDL and VHDL, with the following characteristics:

- Any combination of independent read or write operations in the same clock cycle.
- At most two unique port addresses.
- In one clock cycle, with one or two unique addresses, they can perform:
  - ☐ Two reads and one write
  - ☐ Two writes and one read
  - ☐ Two writes and two reads

In the synchronous RAM block architecture, there is no priority between the two ports. Therefore, if you write to the same location on both ports at the same time, the result is indeterminate in the device architecture. You must ensure your HDL code does not imply priority for writes to the memory block, if you want the design to be implemented in a dedicated hardware memory block. For example, if both ports are defined in the same process block, the code is synthesized and simulated sequentially so that there is a priority between the two ports. If your code does imply a priority, the logic cannot be implemented in the device RAM blocks and is implemented in regular logic cells. You must also consider the read-during-write behavior of the RAM block to ensure that it can be mapped directly to the device RAM architecture.

When a read and write operation occurs on the same port for the same address, the read operation may behave as follows:

- **Read new data**—Arria® 10 and Stratix® 10 devices support this behavior.
- **Read old data**—Not supported.

When a read and write operation occurs on different ports for the same address (also known as mixed port), the read operation may behave as follows:

- **Read new data**—Quartus Prime Pro Edition synthesis supports this mode by creating bypass logic around the synchronous memory block.
- **Read old data**—Arria 10 and Cyclone® 10 devices support this behavior.
- **Read don't care**—Synchronous memory blocks support this behavior in simple dual-port mode.

The Verilog HDL single-clock code sample maps directly into synchronous Arria 10 memory blocks. When a read and write operation occurs on the same port for the same address, the new data being written to the memory is read. When a read and write operation occurs on different ports for the same address, the old data in the memory is read. Simultaneous writes to the same location on both ports results in indeterminate behavior.

If you generate a dual-clock version of this design describing the same behavior, the inferred memory in the target device presents undefined mixed port read-during-write behavior, because it depends on the relationship between the clocks.

Example 16. Verilog HDL True Dual-Port RAM with Single Clock

```
/ Quartus Prime Verilog Template
// True Dual Port RAM with single clock
//
// Read-during-write behavior is undefined for mixed ports
// and "new data" on the same port

module true_dual_port_ram_single_clock
#(parameter DATA_WIDTH=8, parameter ADDR_WIDTH=6)
(
    input [(DATA_WIDTH-1):0] data_a, data_b,
    input [(ADDR_WIDTH-1):0] addr_a, addr_b,
    input we_a, we_b, clk,
    output reg [(DATA_WIDTH-1):0] q_a, q_b
);

    // Declare the RAM variable
    reg [DATA_WIDTH-1:0] ram[2**ADDR_WIDTH-1:0];

    // Port A
    always @ (posedge clk)
    begin
        if (we_a)
        begin
            ram[addr_a] = data_a;
        end
        q_a <= ram[addr_a];
    end

    // Port B
    always @ (posedge clk)
    begin
        if (we_b)
        begin
            ram[addr_b] = data_b;
        end
```

```
        q_b <= ram[addr_b];
    end

endmodule
```

Example 17. VHDL Read Statement Example

```
-- Port A
process(clk)
    begin
    if(rising_edge(clk)) then
        if(we_a = '1') then
            ram(addr_a) := data_a;
        end if;
        q_a <= ram(addr_a);
    end if;
end process;

-- Port B
process(clk)
    begin
    if(rising_edge(clk)) then
        if(we_b = '1') then
            ram(addr_b) := data_b;
        end if;
        q_b <= ram(addr_b);
    end if;
end process;
```

The VHDL single-clock code sample maps directly into Altera FPGA synchronous memory. When a read and write operation occurs on the same port for the same address, the new data writing to the memory is read. When a read and write operation occurs on different ports for the same address, the behavior results in old data for Arria 10 and Cyclone 10 devices, and is undefined for Stratix 10 devices. Simultaneous write operations to the same location on both ports results in indeterminate behavior.

If you generate a dual-clock version of this design describing the same behavior, the memory in the target device presents undefined mixed port read-during-write behavior because it depends on the relationship between the clocks.

Example 18. VHDL True Dual-Port RAM with Single Clock

```
-- Quartus Prime VHDL Template
-- True Dual-Port RAM with single clock
--
-- Read-during-write behavior is undefined for mixed ports
-- and "new data" on the same port

library ieee;
use ieee.std_logic_1164.all;

entity true_dual_port_ram_single_clock is

    generic
    (
        DATA_WIDTH : natural := 8;
        ADDR_WIDTH : natural := 6
    );

    port
    (
        clk        : in std_logic;
        addr_a     : in natural range 0 to 2**ADDR_WIDTH - 1;
        addr_b     : in natural range 0 to 2**ADDR_WIDTH - 1;
```

```
        data_a    : in std_logic_vector((DATA_WIDTH-1) downto 0);
        data_b    : in std_logic_vector((DATA_WIDTH-1) downto 0);
        we_a    : in std_logic := '1';
        we_b    : in std_logic := '1';
        q_a        : out std_logic_vector((DATA_WIDTH -1) downto 0);
        q_b        : out std_logic_vector((DATA_WIDTH -1) downto 0)
    );

end true_dual_port_ram_single_clock;

architecture rtl of true_dual_port_ram_single_clock is

    -- Build a 2-D array type for the RAM
    subtype word_t is std_logic_vector((DATA_WIDTH-1) downto 0);
    type memory_t is array(2**ADDR_WIDTH-1 downto 0) of word_t;

    -- Declare the RAM
    shared variable ram : memory_t;

begin

    -- Port A
    process(clk)
    begin
    if(rising_edge(clk)) then
        if(we_a = '1') then
            ram(addr_a) := data_a;
        end if;
        q_a <= ram(addr_a);
    end if;
    end process;

    -- Port B
    process(clk)
    begin
    if(rising_edge(clk)) then
        if(we_b = '1') then
            ram(addr_b) := data_b;
        end if;
          q_b <= ram(addr_b);
    end if;
    end process;

end rtl;
```

Related Information

Arria® 10 Core Fabric and General Purpose I/Os Handbook

2.4.1.9. Mixed-Width Dual-Port RAM

The RAM code examples in this section show SystemVerilog and VHDL code that infers RAM with data ports with different widths.

Verilog-1995 doesn't support mixed-width RAMs because the standard lacks a multi-dimensional array to model the different read width, write width, or both. Verilog-2001 doesn't support mixed-width RAMs because this type of logic requires multiple packed dimensions. Different synthesis tools may differ in their support for these memories. This section describes the inference rules for Quartus Prime Pro Edition synthesis.

The first dimension of the multi-dimensional packed array represents the ratio of the wider port to the narrower port. The second dimension represents the narrower port width. The read and write port widths must specify a read or write ratio supported by the memory blocks in the target device. Otherwise, the synthesis tool does not infer a RAM.

Refer to the Quartus Prime HDL templates for parameterized examples with supported combinations of read and write widths. You can also find examples of true dual port RAMs with two mixed-width read ports and two mixed-width write ports.

Example 19. SystemVerilog Mixed-Width RAM with Read Width Smaller than Write Width

```
module mixed_width_ram    // 256x32 write and 1024x8 read
(
        input [7:0] waddr,
        input [31:0] wdata,
        input we, clk,
        input [9:0] raddr,
        output logic [7:0] q
);
    logic [3:0][7:0] ram[0:255];
    always_ff@(posedge clk)
        begin
            if(we) ram[waddr] <= wdata;
            q <= ram[raddr / 4][raddr % 4];
        end
endmodule : mixed_width_ram
```

Example 20. SystemVerilog Mixed-Width RAM with Read Width Larger than Write Width

```
module mixed_width_ram     // 1024x8 write and 256x32 read
(
        input [9:0] waddr,
        input [31:0] wdata,
        input we, clk,
        input [7:0] raddr,
        output logic [9:0] q
);
    logic [3:0][7:0] ram[0:255];
    always_ff@(posedge clk)
         begin
            if(we) ram[waddr / 4][waddr % 4] <= wdata;
            q <= ram[raddr];
         end
endmodule : mixed_width_ram
```

Example 21. VHDL Mixed-Width RAM with Read Width Smaller than Write Width

```
library ieee;
use ieee.std_logic_1164.all;

package ram_types is
    type word_t is array (0 to 3) of std_logic_vector(7 downto 0);
    type ram_t is array (0 to 255) of word_t;
end ram_types;

library ieee;
use ieee.std_logic_1164.all;
library work;
use work.ram_types.all;

entity mixed_width_ram is
    port (
        we, clk : in  std_logic;
        waddr   : in  integer range 0 to 255;
        wdata   : in  word_t;
        raddr   : in  integer range 0 to 1023;
        q       : out std_logic_vector(7 downto 0));
end mixed_width_ram;

architecture rtl of mixed_width_ram is
```

```
    signal ram : ram_t;
begin  -- rtl
    process(clk, we)
    begin
        if(rising_edge(clk)) then
            if(we = '1') then
                ram(waddr) <= wdata;
            end if;
            q <= ram(raddr / 4 )(raddr mod 4);
        end if;
    end process;
end rtl;
```

Example 22. VHDL Mixed-Width RAM with Read Width Larger than Write Width

```
library ieee;
use ieee.std_logic_1164.all;

package ram_types is
    type word_t is array (0 to 3) of std_logic_vector(7 downto 0);
    type ram_t is array (0 to 255) of word_t;
end ram_types;

library ieee;
use ieee.std_logic_1164.all;
library work;
use work.ram_types.all;

entity mixed_width_ram is
    port (
        we, clk : in  std_logic;
        waddr   : in  integer range 0 to 1023;
        wdata   : in  std_logic_vector(7 downto 0);
        raddr   : in  integer range 0 to 255;
        q       : out word_t);
end mixed_width_ram;

architecture rtl of mixed_width_ram is
    signal ram : ram_t;
begin  -- rtl
    process(clk, we)
    begin
        if(rising_edge(clk)) then
            if(we = '1') then
                ram(waddr / 4)(waddr mod 4) <= wdata;
            end if;
            q <= ram(raddr);
        end if;
    end process;
end rtl;
```

2.4.1.10. Inferring RAM with Byte-Enable Signals

The RAM code examples in this section show SystemVerilog and VHDL code that infers RAM with controls for writing single bytes into the memory word, or byte-enable signals.

Note: Starting in version 25.1, the Quartus Prime Pro Edition software now supports byte enable inference for all legal byte widths (5, 8, 9, and 10). Prior to version 25.1, the Quartus Prime synthesis only supports byte enable inference for byte widths of 8.

Synthesis models byte-enable signals by creating write expressions with two indexes, and writing part of a RAM "word." With these implementations, you can also write more than one byte at once by enabling the appropriate byte enables.

Verilog-1995 doesn't support mixed-width RAMs because the standard lacks a multi-dimensional array to model the different read width, write width, or both. Verilog-2001 doesn't support mixed-width RAMs because this type of logic requires multiple packed dimensions. Different synthesis tools may differ in their support for these memories. This section describes the inference rules for Quartus Prime Pro Edition synthesis.

Refer to the Quartus Prime HDL templates for parameterized examples that you can use for different address widths, and true dual port RAM examples with two read ports and two write ports.

Example 23. SystemVerilog Simple Dual-Port Synchronous RAM with Byte Enable

```
module byte_enabled_simple_dual_port_ram
(
    input we, clk,
    input [ADDRESS_WIDTH-1:0] waddr, raddr,// address width = 6
    input [NUM_BYTES-1:0] be, // 4 bytes per word
    input [(BYTE_WIDTH * NUM_BYTES -1):0] wdata, // byte width = 8, 4 bytes per
word
    output reg [(BYTE_WIDTH * NUM_BYTES -1):0] q // byte width = 8, 4 bytes per
word
);

   parameter ADDRESS_WIDTH = 6;
   parameter DEPTH = 2**ADDRESS_WIDTH;
   parameter BYTE_WIDTH = 8;
   parameter NUM_BYTES = 4;

   // use a multi-dimensional packed array
   //to model individual bytes within the word
   logic [NUM_BYTES-1:0][BYTE_WIDTH-1:0] ram[0:DEPTH-1];
     // # words = 1 << address width

    // port A
   always@(posedge clk)
   begin
       if(we) begin
          for (int i = 0; i < NUM_BYTES; i = i + 1) begin
            if(be[i]) ram[waddr][i] <= wdata[i*BYTE_WIDTH +: BYTE_WIDTH];
          end
      end
      q <= ram[raddr];
   end
endmodule
```

Example 24. VHDL Simple Dual-Port Synchronous RAM with Byte Enable

```
library ieee;
use ieee.std_logic_1164.all;
library work;

entity byte_enabled_simple_dual_port_ram is
generic (DEPTH      : integer := 64;
         NUM_BYTES  : integer :=  4;
         BYTE_WIDTH : integer :=  8
);
port (
    we, clk : in  std_logic;
    waddr, raddr : in  integer range 0 to DEPTH -1 ;     -- address width = 6
    be    : in  std_logic_vector (NUM_BYTES-1 downto 0);   -- 4 bytes per word
    wdata: in  std_logic_vector((NUM_BYTES * BYTE_WIDTH -1) downto 0);   --
width = 32
    q     : out std_logic_vector((NUM_BYTES * BYTE_WIDTH -1) downto 0) ); --
width = 32
end byte_enabled_simple_dual_port_ram;
```

```
architecture rtl of byte_enabled_simple_dual_port_ram is

    --  build up 2D array to hold the memory
    type word_t is array (0 to NUM_BYTES-1) of std_logic_vector(BYTE_WIDTH-1
downto 0);
    type ram_t is array (0 to DEPTH-1) of word_t;

    signal ram : ram_t;
    signal q_local : word_t;

    begin  -- Re-organize the read data from the RAM to match the output
        unpack: for i in 0 to NUM_BYTES-1 generate
            q(BYTE_WIDTH*(i+1) - 1 downto BYTE_WIDTH*i) <= q_local(i);
    end generate unpack;

    -- port A
    process(clk)
    begin
        if(rising_edge(clk)) then
            if(we = '1') then
                for I in (NUM_BYTES-1) downto 0 loop
                    if(be(I) = '1') then
                        ram(waddr)(I) <= wdata(((I+1)*BYTE_WIDTH-1) downto
I*BYTE_WIDTH);
                    end if;
                 end loop;
            end if;
            q_local <= ram(raddr);
        end if;
    end process;
end rtl;
```

2.4.1.11. Inferring Simple Quad-Port RAM

A simple quad-port RAM has a total of four ports, two read ports and two write ports.

Starting in version 25.1, Quartus Prime Pro Edition synthesis can now infer simple quad-port RAM.

2.4.1.12. Specifying Initial Memory Contents at Power-Up

Your synthesis tool may offer various ways to specify the initial contents of an inferred memory. There are slight power-up and initialization differences between dedicated RAM blocks and the MLAB memory, due to the continuous read of the MLAB.

Altera FPGA dedicated RAM block outputs always power-up to zero, and are set to the initial value on the first read. For example, if address `0` is pre-initialized to FF, the RAM block powers up with the output at `0`. A subsequent read after power-up from address `0` outputs the pre-initialized value of FF. Therefore, if a RAM powers up and an enable (read enable or clock enable) is held low, the power-up output of `0` maintains until the first valid read cycle. The synthesis tool implements MLAB using registers that power-up to `0`, but initialize to their initial value immediately at power-up or reset. Therefore, the initial value is seen, regardless of the enable status. The Quartus Prime software maps inferred memory to MLABs when the HDL code specifies an appropriate `ramstyle` attribute.

In Verilog HDL, you can use an initial block to initialize the contents of an inferred memory. Quartus Prime Pro Edition synthesis automatically converts the initial block into a Memory Initialization File (`.mif`) for the inferred RAM.

Example 25. Verilog HDL RAM with Initialized Contents

```
module ram_with_init(
   output reg [7:0] q,
   input [7:0] d,
   input [4:0] write_address, read_address,
   input we, clk
);
   reg [7:0] mem [0:31];
   integer i;

   initial begin
      for (i = 0; i < 32; i = i + 1)
         mem[i] = i[7:0];
   end

   always @ (posedge clk) begin
      if (we)
         mem[write_address] <= d;
      q <= mem[read_address];
   end
endmodule
```

Quartus Prime Pro Edition synthesis and other synthesis tools also support the `$readmemb` and `$readmemh` attributes. These attributes allow RAM initialization and ROM initialization work identically in synthesis and simulation.

Example 26. Verilog HDL RAM Initialized with the readmemb Command

```
reg [7:0] ram[0:15];
initial
begin
    $readmemb("ram.txt", ram);
end
```

In VHDL, you can initialize the contents of an inferred memory by specifying a default value for the corresponding signal. Quartus Prime Pro Edition synthesis automatically converts the default value into a `.mif` file for the inferred RAM.

Example 27. VHDL RAM with Initialized Contents

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY ram_with_init IS
    PORT(
            clock: IN STD_LOGIC;
            data: IN UNSIGNED (7 DOWNTO 0);
            write_address: IN integer RANGE 0 to 31;
            read_address: IN integer RANGE 0 to 31;
            we: IN std_logic;
            q: OUT UNSIGNED (7 DOWNTO 0));
END;

ARCHITECTURE rtl OF ram_with_init IS

    TYPE MEM IS ARRAY(31 DOWNTO 0) OF unsigned(7 DOWNTO 0);
    FUNCTION initialize_ram
        return MEM is
        variable result : MEM;
    BEGIN
        FOR i IN 31 DOWNTO 0 LOOP
            result(i) := to_unsigned(natural(i), natural'(8));
        END LOOP;
        RETURN result;
```

```
    END initialize_ram;

    SIGNAL ram_block : MEM := initialize_ram;
BEGIN
    PROCESS (clock)
    BEGIN
        IF (rising_edge(clock)) THEN
            IF (we = '1') THEN
            ram_block(write_address) <= data;
            END IF;
            q <= ram_block(read_address);
        END IF;
    END PROCESS;
END rtl;
```

Example 28. Verilog HDL Single-Clock, Simple Dual-Port RAM with Synchronous Reset

```
// Simple Dual-Port Block with Single Clock

module simple_dual_port_ram_single_clock
#(
   parameter DATA_WIDTH = 8,
   parameter ADDR_WIDTH = 6
)
(
    input clk, clr, we,
    input [ADDR_WIDTH-1:0] waddr,
    input [ADDR_WIDTH-1:0] raddr,
    input [DATA_WIDTH-1:0] data_in,
    output reg [DATA_WIDTH-1:0] data_out
);

(*ramstyle = "M20K"*) reg [DATA_WIDTH-1:0] memory[2**ADDR_WIDTH-1:0];

always @(posedge clk) begin
    if (clr) begin
       data_out <= 0;
    end
    else begin
          data_out <= memory[raddr];
    end
end

always @(posedge clk) begin
    if (we) begin
       memory[waddr] <= data_in;
    end
end


endmodule
```

RAM inference with synchronous reset is supported for single-port, simple-dual port, and true dual port RAM.

Example 29. Verilog HDL Single-Clock, Simple Dual-Port RAM with Asynchronous Reset

```
// Simple Dual-Port Block with Single Clock

module simple_dual_port_ram_single_clock_aclr
#(
   parameter DATA_WIDTH = 8,
   parameter ADDR_WIDTH = 6
)
(
    input clk, clr, we,
    input [ADDR_WIDTH-1:0] waddr,
    input [ADDR_WIDTH-1:0] raddr,
```

```
    input [DATA_WIDTH-1:0] data_in,
    output reg [DATA_WIDTH-1:0] data_out
);

(*ramstyle = "M20K"*) reg [DATA_WIDTH-1:0] memory[2**ADDR_WIDTH-1:0];


always @(posedge clk or posedge clr) begin
    if (clr) begin
       data_out <= 0;
    end
    else begin
           data_out <= memory[raddr];
    end
end

always @(posedge clk) begin
    if (we) begin
        memory[waddr] <= data_in;
    end
end


endmodule
```

RAM inference with asynchronous reset is supported for single-port and simple-dual port RAM.

2.4.2. Inferring ROM Functions from HDL Code

Synthesis tools infer ROMs when a `CASE` statement exists in which a value is set to a constant for every choice in the `CASE` statement.

Because small ROMs typically achieve the best performance when they are implemented using the registers in regular logic, each ROM function must meet a minimum size requirement for inference and placement in memory.

For device architectures with synchronous RAM blocks, to infer a ROM block, synthesis must use registers for either the address or the output. When your design uses output registers, synthesis implements registers from the input registers of the RAM block without affecting the functionality of the ROM. If you register the address, the power-up state of the inferred ROM can be different from the HDL design. In this scenario, Quartus Prime synthesis issues a warning.

The following ROM examples map directly to the Altera FPGA memory architecture.

Example 30. Verilog HDL Synchronous ROM

```
module sync_rom (clock, address, data_out);
    input clock;
    input [7:0] address;
    output reg [5:0] data_out;
    reg [5:0] data_out;

    always @ (posedge clock)
    begin
        case (address)
            8'b00000000: data_out = 6'b101111;
            8'b00000001: data_out = 6'b110110;
            ...
            8'b11111110: data_out = 6'b000001;
            8'b11111111: data_out = 6'b101010;
```

```
        endcase
    end
endmodule
```

Example 31. VHDL Synchronous ROM

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY sync_rom IS
    PORT (
        clock: IN STD_LOGIC;
        address: IN STD_LOGIC_VECTOR(7 downto 0);
        data_out: OUT STD_LOGIC_VECTOR(5 downto 0)
    );
END sync_rom;

ARCHITECTURE rtl OF sync_rom IS
BEGIN
PROCESS (clock)
    BEGIN
    IF rising_edge (clock) THEN
        CASE address IS
            WHEN "00000000" => data_out <= "101111";
            WHEN "00000001" => data_out <= "110110";
            ...
            WHEN "11111110" => data_out <= "000001";
            WHEN "11111111" => data_out <= "101010";
            WHEN OTHERS     => data_out <= "101111";
        END CASE;
    END IF;
    END PROCESS;
END rtl;
```

Example 32. Verilog HDL Dual-Port Synchronous ROM Using readmemb

```
module dual_port_rom
#(parameter data_width=8, parameter addr_width=8)
(
    input [(addr_width-1):0] addr_a, addr_b,
    input clk,
    output reg [(data_width-1):0] q_a, q_b
);
    reg [data_width-1:0] rom[2**addr_width-1:0];

    initial // Read the memory contents in the file
             //dual_port_rom_init.txt.
    begin
        $readmemb("dual_port_rom_init.txt", rom);
    end

    always @ (posedge clk)
    begin
        q_a <= rom[addr_a];
        q_b <= rom[addr_b];
    end
endmodule
```

Example 33. VHDL Dual-Port Synchronous ROM Using Initialization Function

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dual_port_rom is
    generic (
```

```
        DATA_WIDTH : natural := 8;
        ADDR_WIDTH : natural := 8
    );
    port (
        clk     : in std_logic;
        addr_a  : in natural range 0 to 2**ADDR_WIDTH - 1;
        addr_b  : in natural range 0 to 2**ADDR_WIDTH - 1;
        q_a     : out std_logic_vector((DATA_WIDTH -1) downto 0);
        q_b     : out std_logic_vector((DATA_WIDTH -1) downto 0)
    );
end entity;

architecture rtl of dual_port_rom is
    -- Build a 2-D array type for the ROM
    subtype word_t is std_logic_vector((DATA_WIDTH-1) downto 0);
    type memory_t is array(2**ADDR_WIDTH - 1 downto 0) of word_t;

    function init_rom
        return memory_t is
        variable tmp : memory_t := (others => (others => '0'));
    begin
        for addr_pos in 0 to 2**ADDR_WIDTH - 1 loop
            -- Initialize each address with the address itself
            tmp(addr_pos) := std_logic_vector(to_unsigned(addr_pos, DATA_WIDTH));
        end loop;
        return tmp;
    end init_rom;

    -- Declare the ROM signal and specify a default initialization value.
    signal rom : memory_t := init_rom;
begin
    process(clk)
    begin
    if (rising_edge(clk)) then
        q_a <= rom(addr_a);
        q_b <= rom(addr_b);
    end if;
    end process;
end rtl;
```

2.4.3. Inferring Shift Registers in HDL Code

The Quartus Prime software Analysis & Synthesis stage of the Compiler automatically detects and infers shift registers in your HDL code according to the following guidelines:

- Shift Register Inference for Stratix 10 and Agilex™ FPGA Portfolio Devices on page 30
- Shift Register Inference for Arria 10 and Cyclone 10 GX Devices on page 31

Shift Register Inference for Stratix 10 and Agilex™ FPGA Portfolio Devices

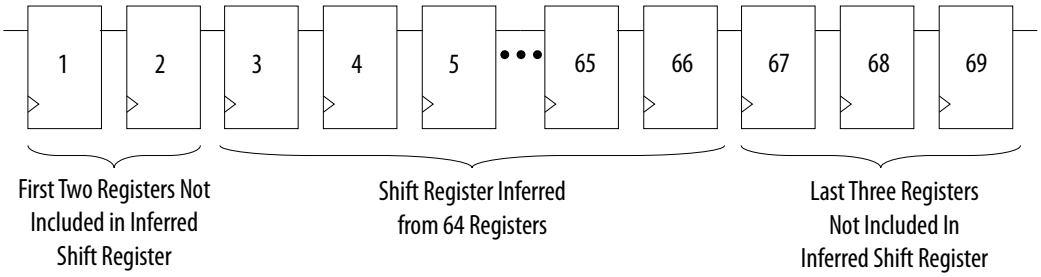
Because of the high prevalence of registers in routing segments of the Hyperflex® architecture, the Compiler's threshold for shift register inference is increased for Stratix 10 and Agilex™ FPGA Portfolio devices. This increase in the threshold means that some logic that the Compiler infers as a shift register in a previous generation FPGA, may not be inferred as a shift register when targeting Stratix 10 or Agilex FPGA portfolio devices. This threshold increase allows more register retiming, thus improving overall design performance.

The following criteria apply to shift register detection and inference for Stratix 10 and Agilex FPGA portfolio devices.

Default shift register inference requirements for Stratix 10 and Agilex FPGA Portfolio Devices:

1. The minimum number of registers inferred in the shift register is 64. When the width of a chain of registers is 1, the chain must contain at least 69 registers for synthesis to infer a shift register. From these 69 registers, synthesis does not include the first and second registers in the chain in the inferred shift register. Synthesis places these first and second shift registers in ALMs. Synthesis infers a 64 bit long shift register with the third through sixty sixth registers. Synthesis does not include the last three registers in the chain in the inferred shift register. Rather, synthesis places the last three registers in ALMs.
2. The minimum depth of registers inferred in the shift register is 32. When the width of a chain of registers is two or more, the chain must contain at least 37 register levels for synthesis to infer a shift register. As in the first requirement, synthesis does not include the first and second registers levels in each chain in the inferred shift register, nor are the last three register levels.

Figure 2. Shift Register Inference for Stratix 10 and Agilex FPGA Portfolio Devices



- With the following assignment, the total number of required registers (depth * width) drops to 37:

```
set_global_assignment -name ALLOW_ANY_SHIFT_REGISTER_SIZE_FOR_RECOGNITION ON
```

  Note: An additional stage of inference takes place during the early retiming stage as physical synthesis optimization recovers area for registers that have not been retimed.

- With both of the following assignments, the total number of required registers (`depth * width`) drops to 13:

```
set_global_assignment -name ALLOW_ANY_SHIFT_REGISTER_SIZE_FOR_RECOGNITION ON
set_global_assignment -name PHYSICAL_SHIFT_REGISTER_INFERENCE=OFF
```

Note: Reducing the shift register inference threshold can negatively impact design performance, as the technique reduces the number of registers available for retiming.

Shift Register Inference for Arria 10 and Cyclone 10 GX Devices

For Arria 10 devices, Analysis & Synthesis detects a group of shift registers of the same length, and implements the registers using the Shift Register Altera* IP.

For automatic detection, all of the shift registers must have the following characteristics:

- Use the same clock and clock enable
- Have no other secondary signals
- Have equally spaced taps that are at least three registers apart

Synthesis recognizes shift registers only for device families with dedicated RAM blocks. Quartus Prime Pro Edition synthesis uses the following guidelines:

- The Quartus Prime software determines whether to infer the Shift Register Altera IP based on the width of the registered bus (*W*), the length between each tap (*L*), or the number of taps (*N*).
- If the **Auto Shift Register Recognition** option is set to **Auto**, Quartus Prime Pro Edition synthesis determines which shift registers are implemented in RAM blocks for logic by using the following methods:
  - ☐ The **Optimization Technique** setting
  - ☐ Logic and RAM utilization information about the design
  - ☐ Timing information from **Timing-Driven Synthesis**
- If the registered bus width is one (*W = 1*), Quartus Prime synthesis infers the shift register IP if the number of taps, times the length between each tap, is greater than or equal to 64 (*N x L > 64*).
- If the registered bus width is greater than one (*W > 1*), and the registered bus width times the number of taps times the length between each tap is greater than or equal to 32 (*W × N × L > 32*), then Quartus Prime synthesis the Shift Register Altera IP.
- If the length between each tap (*L*) is not a power of two, Quartus Prime synthesis needs external logic (LEs or ALMs) to decode the read and write counters, because of different sizes of shift registers. This extra decode logic eliminates the performance and utilization advantages of implementing shift registers in memory.

The registers that Quartus Prime synthesis maps to the Shift Register Altera IP, and places in RAM are not available in a Verilog HDL or VHDL output file for simulation tools, because their node names do not exist after synthesis.

Note: The Compiler cannot implement a shift register that uses a `shift enable` signal into MLAB memory; instead, the Compiler uses dedicated RAM blocks. To control the type of memory structure that implements the shift register, use the `ramstyle` attribute. For example:

```
(* ramstyle = "mlab" *) my_shift_reg
```

2.4.3.1. Simple Shift Register

The examples in this section show a simple, single-bit wide, 69-bit long shift register.

Quartus Prime synthesis implements the register (`W = 1` and `M = 69`) by using the Shift Register Altera IP, and maps it to RAM in the device, which may be placed in dedicated RAM blocks or MLAB memory. If the length of the register is less than 69 bits, Quartus Prime synthesis implements the shift register in logic.

Example 34. Verilog HDL Single-Bit Wide, 69-Bit Long Shift Register

```
module shift_1x69 (clk, shift, sr_in, sr_out);
    input clk, shift;
    input sr_in;
    output sr_out;

    reg [68:0] sr;

    always @ (posedge clk)
    begin
        if (shift == 1'b1)
        begin
            sr[68:1] <= sr[67:0];
            sr[0] <= sr_in;
        end
    end
    assign sr_out = sr[68];
endmodule
```

Example 35. VHDL Single-Bit Wide, 69-Bit Long Shift Register

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.all;
ENTITY shift_1x69 IS
    PORT (
        clk: IN STD_LOGIC;
        shift: IN STD_LOGIC;
        sr_in: IN STD_LOGIC;
        sr_out: OUT STD_LOGIC
    );
END shift_1x69;

ARCHITECTURE arch OF shift_1x69 IS
    TYPE sr_length IS ARRAY (68 DOWNTO 0) OF STD_LOGIC;
    SIGNAL sr: sr_length;
BEGIN
    PROCESS (clk)
        BEGIN
        IF (rising_edge(clk)) THEN
            IF (shift = '1') THEN
            sr(68 DOWNTO 1) <= sr(67 DOWNTO 0);
            sr(0) <= sr_in;
            END IF;
        END IF;
    END PROCESS;
    sr_out <= sr(68);
END arch;
```

2.4.3.2. Shift Register with Evenly Spaced Taps

The following examples show a Verilog HDL and VHDL 8-bit wide, 255-bit long shift register (`W > 1` and `M = 255`) with evenly spaced taps at 64, 128, 192, and 254.

The synthesis software implements this function in a single ALTSHIFT_TAPS IP core and maps it to RAM in supported devices, which is allowed placement in dedicated RAM blocks or MLAB memory.

Example 36. Verilog HDL 8-Bit Wide, 255-Bit Long Shift Register with Evenly Spaced Taps

```
module top (clk, shift, sr_in, sr_out, sr_tap_one, sr_tap_two,
                sr_tap_three );
    input clk, shift;
    input [7:0] sr_in;
    output [7:0] sr_tap_one, sr_tap_two, sr_tap_three, sr_out;
    reg [7:0] sr [254:0];
```

```
    integer n;
    always @ (posedge clk)
        begin
        if (shift == 1'b1)
            begin
            for (n = 254; n>0; n = n-1)
                begin
                sr[n] <= sr[n-1];
                end
            sr[0] <= sr_in;
        end
    end
    assign sr_tap_one = sr[64];
    assign sr_tap_two = sr[128];
    assign sr_tap_three = sr[192];
    assign sr_out = sr[254];
endmodule
```

2.4.4. Inferring FIFOs in HDL Code

There are various methods of implementing dual clock FIFOs, depending on the features needed in your design. The following dual clock FIFO example shows the basic FIFO functionality, with a design goal of high speed (f_{MAX}) and small area.

The FIFO supports parameterization up to 32 words deep, and targets memory LABs (MLABs) for its memory block. Synthesis infers the MLABs from behavioral RTL in the `generic_mlab_dc` module.

Note: If you don't want to code your own FIFO, you can parameterize the dual clock FIFO IP with the IP parameter editor in the Quartus Prime software. Refer to the *FIFO IP User Guide*.

Related Information

FIFO IP User Guide

2.4.4.1. Dual Clock FIFO Example in Verilog HDL

```
// Copyright 2021 Altera Corporation.
//
// This reference design file is subject licensed to you by the terms and
// conditions of the applicable License Terms and Conditions for Hardware
// Reference Designs and/or Design Examples (either as signed by you or
// found at https://www.altera.com/common/legal/leg-license_agreement.html ).
//
// As stated in the license, you agree to only use this reference design
// solely in conjunction with Altera FPGAs or  CPLDs.
//
// THE REFERENCE DESIGN IS PROVIDED "AS IS" WITHOUT ANY EXPRESS OR IMPLIED
// WARRANTY OF ANY KIND INCLUDING WARRANTIES OF MERCHANTABILITY,
// NONINFRINGEMENT, OR FITNESS FOR A PARTICULAR PURPOSE. Altera does not
// warrant or assume responsibility for the accuracy or completeness of any
// information, links or other items within the Reference Design and any
// accompanying materials.
//
// In the event that you do not agree with such terms and conditions, do not
// use the reference design file.
/////////////////////////////////////////////////////////////////////////////

module dcfifo_example
#(
    parameter LOG_DEPTH      = 5,
    parameter WIDTH          = 20,
    parameter ALMOST_FULL_VALUE = 30,
```

```
    parameter ALMOST_EMPTY_VALUE = 2,
    parameter NUM_WORDS = 2**LOG_DEPTH - 1,
    parameter OVERFLOW_CHECKING = 0, // Overflow checking circuitry is \
       using extra area. Use only if you need it
    parameter UNDERFLOW_CHECKING = 0 // Underflow checking circuitry is \
       using extra area. Use only if you need it
)
(
    input aclr,

    input wrclk,
    input wrreq,
    input [WIDTH-1:0] data,
    output reg wrempty,
    output reg wrfull,
    output reg wr_almost_empty,
    output reg wr_almost_full,
    output [LOG_DEPTH-1:0] wrusedw,

    input rdclk,
    input rdreq,
    output [WIDTH-1:0] q,
    output reg rdempty,
    output reg rdfull,
    output reg rd_almost_empty,
    output reg rd_almost_full,
    output [LOG_DEPTH-1:0] rdusedw
);

initial begin
    if ((LOG_DEPTH > 5) || (LOG_DEPTH < 3))
        $error("Invalid parameter value: LOG_DEPTH = %0d; valid range is 2 \
        < LOG_DEPTH < 6", LOG_DEPTH);

    if ((ALMOST_FULL_VALUE > 2 ** LOG_DEPTH - 1) || (ALMOST_FULL_VALUE < 1))
        $error("Incorrect parameter value: ALMOST_FULL_VALUE = %0d; valid \
        range is 0 < ALMOST_FULL_VALUE < %0d",
            ALMOST_FULL_VALUE, 2 ** LOG_DEPTH);

    if ((ALMOST_EMPTY_VALUE > 2 ** LOG_DEPTH - 1) || (ALMOST_EMPTY_VALUE < 1))
        $error("Incorrect parameter value: ALMOST_EMPTY_VALUE = %0d; valid \
        range is 0 < ALMOST_EMPTY_VALUE < %0d",
            ALMOST_EMPTY_VALUE, 2 ** LOG_DEPTH);

    if ((NUM_WORDS > 2 ** LOG_DEPTH - 1) || (NUM_WORDS < 1))
        $error("Incorrect parameter value: NUM_WORDS = %0d; \
        valid range is 0 < NUM_WORDS < %0d",
            NUM_WORDS, 2 ** LOG_DEPTH);
end

(* altera_attribute = "-name AUTO_CLOCK_ENABLE_RECOGNITION OFF" *) reg \
     [LOG_DEPTH-1:0] write_addr = 0;
(* altera_attribute = "-name AUTO_CLOCK_ENABLE_RECOGNITION OFF" *) reg \
     [LOG_DEPTH-1:0] read_addr = 0;
reg [LOG_DEPTH-1:0] wrcapacity = 0;
reg [LOG_DEPTH-1:0] rdcapacity = 0;

wire [LOG_DEPTH-1:0] wrcapacity_w;
wire [LOG_DEPTH-1:0] rdcapacity_w;

wire [LOG_DEPTH-1:0] rd_write_addr;
wire [LOG_DEPTH-1:0] wr_read_addr;

wire wrreq_safe;
wire rdreq_safe;
assign wrreq_safe = OVERFLOW_CHECKING ? wrreq & ~wrfull : wrreq;
assign rdreq_safe = UNDERFLOW_CHECKING ? rdreq & ~rdempty : rdreq;

initial begin
    write_addr = 0;
```

```
    read_addr = 0;
    wrempty = 1;
    wrfull = 0;
    rdempty = 1;
    rdfull = 0;
    wrcapacity = 0;
    rdcapacity = 0;
    rd_almost_empty = 1;
    rd_almost_full = 0;
    wr_almost_empty = 1;
    wr_almost_full = 0;
end


// ------------------ Write ------------------------

add_a_b_s0_s1 #(LOG_DEPTH) wr_adder(
    .a(write_addr),
    .b(~wr_read_addr),
    .s0(wrreq_safe),
    .s1(1'b1),
    .out(wrcapacity_w)
);

always @(posedge wrclk or posedge aclr) begin

    if (aclr) begin
        write_addr <= 0;
        wrcapacity <= 0;
        wrempty <= 1;
        wrfull <= 0;
        wr_almost_full <= 0;
        wr_almost_empty <= 1;
    end else begin
        write_addr <= write_addr + wrreq_safe;
        wrcapacity <= wrcapacity_w;
        wrempty <= (wrcapacity == 0) && (wrreq == 0);
        wrfull <= (wrcapacity == NUM_WORDS) || (wrcapacity == NUM_WORDS - 1) \
        && (wrreq == 1);

        wr_almost_empty <=
            (wrcapacity < (ALMOST_EMPTY_VALUE-1)) ||
            (wrcapacity == (ALMOST_EMPTY_VALUE-1)) && (wrreq == 0);

        wr_almost_full <=
            (wrcapacity >= ALMOST_FULL_VALUE) ||
            (wrcapacity == ALMOST_FULL_VALUE - 1) && (wrreq == 1);
    end
end

assign wrusedw = wrcapacity;

// ------------------ Read ------------------------

add_a_b_s0_s1 #(LOG_DEPTH) rd_adder(
    .a(rd_write_addr),
    .b(~read_addr),
    .s0(1'b0),
    .s1(~rdreq_safe),
    .out(rdcapacity_w)
);

always @(posedge rdclk or posedge aclr) begin
    if (aclr) begin
        read_addr <= 0;
        rdcapacity <= 0;
        rdempty <= 1;
        rdfull <= 0;
        rd_almost_empty <= 1;
        rd_almost_full <= 0;
```

```
    end else begin
        read_addr <= read_addr + rdreq_safe;
        rdcapacity <= rdcapacity_w;
        rdempty <= (rdcapacity == 0) || (rdcapacity == 1) && (rdreq == 1);
        rdfull <= (rdcapacity == NUM_WORDS) && (rdreq == 0);
        rd_almost_empty <=
            (rdcapacity < ALMOST_EMPTY_VALUE) ||
            (rdcapacity == ALMOST_EMPTY_VALUE) && (rdreq == 1);

        rd_almost_full <=
            (rdcapacity > ALMOST_FULL_VALUE) ||
            (rdcapacity == ALMOST_FULL_VALUE) && (rdreq == 0);
    end
end

assign rdusedw = rdcapacity;

// ---------------- Synchronizers --------------------

wire [LOG_DEPTH-1:0] gray_read_addr;
wire [LOG_DEPTH-1:0] wr_gray_read_addr;
wire [LOG_DEPTH-1:0] gray_write_addr;
wire [LOG_DEPTH-1:0] rd_gray_write_addr;

binary_to_gray #(.WIDTH(LOG_DEPTH)) rd_b2g (.clock(rdclk), .aclr(aclr), \
     .din(read_addr), .dout(gray_read_addr));
synchronizer_ff_r2 #(.WIDTH(LOG_DEPTH)) rd2wr
(.din_clk(rdclk), .din(gray_read_addr), \
     .dout_clk(wrclk), .dout(wr_gray_read_addr));
gray_to_binary #(.WIDTH(LOG_DEPTH)) rd_g2b (.clock(wrclk), .aclr(aclr), \
     .din(wr_gray_read_addr), .dout(wr_read_addr));


binary_to_gray #(.WIDTH(LOG_DEPTH)) wr_b2g
(.clock(wrclk), .aclr(aclr), .din(write_addr), \
     .dout(gray_write_addr));
synchronizer_ff_r2 #(.WIDTH(LOG_DEPTH)) wr2rd
(.din_clk(wrclk), .din(gray_write_addr), \
     .dout_clk(rdclk), .dout(rd_gray_write_addr));
gray_to_binary #(.WIDTH(LOG_DEPTH)) wr_g2b (.clock(rdclk), .aclr(aclr), \
     .din(rd_gray_write_addr), .dout(rd_write_addr));

// ------------------ MLAB ----------------------------

generic_mlab_dc #(.WIDTH(WIDTH), .ADDR_WIDTH(LOG_DEPTH)) mlab_inst (
    .rclk(rdclk),
    .wclk(wrclk),
    .din(data),
    .waddr(write_addr),
    .we(1'b1),
    .re(1'b1),
    .raddr(read_addr),
    .dout(q)
);

endmodule

module add_a_b_s0_s1 #(
    parameter SIZE = 5
)(
    input [SIZE-1:0] a,
    input [SIZE-1:0] b,
    input s0,
    input s1,
    output [SIZE-1:0] out
);
    wire [SIZE:0] left;
    wire [SIZE:0] right;
    wire temp;
```

```
    assign left = {a ^ b, s0};
    assign right = {a[SIZE-2:0] & b[SIZE-2:0], s1, s0};
    assign {out, temp} = left + right;

endmodule

module binary_to_gray #(
    parameter WIDTH = 5
) (
    input clock,
    input aclr,
    input [WIDTH-1:0] din,
    output reg [WIDTH-1:0] dout
);

    always @(posedge clock or posedge aclr) begin
         if (aclr)
              dout <= 0;
         else
              dout <= din ^ (din >> 1);
    end

endmodule

module gray_to_binary #(
    parameter WIDTH = 5
) (
    input clock,
    input aclr,
    input [WIDTH-1:0] din,
    output reg [WIDTH-1:0] dout
);

    wire [WIDTH-1:0] dout_w;

    genvar i;
    generate
         for (i = 0; i < WIDTH; i=i+1) begin : loop
              assign dout_w[i] = ^(din[WIDTH-1:i]);
         end
    endgenerate

    always @(posedge clock or posedge aclr) begin
         if (aclr)
              dout <= 0;
         else
              dout <= dout_w;
    end

endmodule

(* altera_attribute = "-name SYNCHRONIZER_IDENTIFICATION OFF" *)
module generic_mlab_dc #(
    parameter WIDTH = 8,
    parameter ADDR_WIDTH = 5
)(
    input rclk,
    input wclk,
    input [WIDTH-1:0] din,
    input [ADDR_WIDTH-1:0] waddr,
    input we,
    input re,
    input [ADDR_WIDTH-1:0] raddr,
    output [WIDTH-1:0] dout
);

    localparam DEPTH = 1 << ADDR_WIDTH;
    (* ramstyle = "mlab" *) reg [WIDTH-1:0] mem[0:DEPTH-1];

    reg [WIDTH-1:0] dout_r;
```

```
    always @(posedge wclk) begin
         if (we)
              mem[waddr] <= din;
    end
    always @(posedge rclk) begin
         if (re)
              dout_r <= mem[raddr];
    end
    assign dout = dout_r;

endmodule


module synchronizer_ff_r2 #(
    parameter WIDTH = 8
)(
    input din_clk,
    input [WIDTH-1:0] din,
    input dout_clk,
    output [WIDTH-1:0] dout
);

    reg [WIDTH-1:0] ff_launch = {WIDTH {1'b0}}
    /* synthesis preserve dont_replicate */;
    always @(posedge din_clk) begin
         ff_launch <= din;
    end

    reg [WIDTH-1:0] ff_meta = {WIDTH {1'b0}}
    /* synthesis preserve dont_replicate */;
    always @(posedge dout_clk) begin
         ff_meta <= ff_launch;
    end

    reg [WIDTH-1:0] ff_sync = {WIDTH {1'b0}}
   /* synthesis preserve dont_replicate */;
    always @(posedge dout_clk) begin
         ff_sync <= ff_meta;
    end

    assign dout = ff_sync;
endmodule
```

2.4.4.2. Dual Clock FIFO Timing Constraints

If you choose to code your own dual clock FIFO, you must also create appropriate timing constraints in Synopsis Design Constraints format (`.sdc`).

Typically, you set the read and write clock domains asynchronous to each other by using the `set_clock_groups` SDC command. You typically specify the `set_clock_groups` command in a top-level `.sdc` file.

Constrain the read and write pointer clock domain crossings with skew and net delay constraints.

A skew constraint ensures the gray-coded pointer values transfer correctly between the clock domains. A net delay constraint bounds the wire delay between the two clock domains, to help reduce latency through the FIFO.

In the RTL example above, the pointers cross clock domains at the `ff_launch` to `ff_meta` register path, in two instances of the `synchronizer_ff_r2` entity.

The following example constraints are appropriate for the RTL above. You can customize the `-from` and `-to` names as necessary for your implementation.

```
# Skew from read to write domain
set_max_skew -from rd2wr|ff_launch[*] -to rd2wr|ff_meta[*] \
     -get_skew_value_from_clock_period
src_clock_period -skew_value_multiplier 0.8
# Skew from write to read domain
set_max_skew -from wr2rd|ff_launch[*] -to wr2rd|ff_meta[*] \
     -get_skew_value_from_clock_period
src_clock_period -skew_value_multiplier 0.8
# Net delay from read to write domain
set_net_delay -from rd2wr|ff_launch[*] -to rd2wr|ff_meta[*] \
     -get_value_from_clock_period
dst_clock_period -value_multiplier 0.8 -max
# Net delay from write to read domain
set_net_delay -from wr2rd|ff_launch[*] -to wr2rd|ff_meta[*] \
     -get_value_from_clock_period
dst_clock_period -value_multiplier 0.8 -max
```

After writing the skew and net delay constraints in the `.sdc` file, specify an entity-bound `.sdc` file `.qsf` assignment to apply the constraints to the synchronizer register paths in all instances of your FIFO.

Use the name of the `.sdc` file containing these constraints in the entity-bound `.sdc` file assignment in your `.qsf`. Also provide the name of the FIFO entity to which the constraints apply.

The following `.qsf` assignment example assumes that you save the constraints in `fifo_synchronizer.sdc` in your project directory, and that the constraints therein apply to the `dcfifo_example` entity:

```
set_global_assignment -name SDC_ENTITY_FILE fifo_synchronizer.sdc \
     -entity dcfifo_example
```

2.5. Register and Latch Coding Guidelines

This section provides device-specific coding recommendations for Altera registers and latches. Understanding the architecture of the target Altera device helps ensure that your RTL produces the expected results and achieves the optimal quality of results.

2.5.1. Register Power-Up Values

Registers in the device core power-up to a low (`0`) logic level on all Altera FPGA devices. However, for designs that specify a power-up level other than `0`, synthesis tools can implement logic that directs registers to behave as if they were powering up to a high (`1`) logic level.

For designs that use `preset` signals, but the target device does not support presets in the register architecture, synthesis may convert the `preset` signal to a `clear` signal, which requires to perform a NOT gate push-back optimization. NOT gate push-back adds an inverter to the input and the output of the register, so that the reset and power-up conditions appear high, and the device operates as expected. In this case, the synthesis tool may issue a message about the power-up condition. The register itself powers up low, but since the register output inverts, the signal that arrives at all destinations is high.

Due to these effects, if you specify a non-zero reset value, the synthesis tool may use the asynchronous clear (`aclr`) signals available on the registers to implement the high bits with NOT gate push-back. In that case, the registers look as though they power-up to the specified reset value.

When an asynchronous load (`aload`) signal is available in the device registers, the synthesis tools can implement a reset of `1` or `0` value by using an asynchronous load of `1` or `0`. When the synthesis tool uses a `load` signal, it is not performing NOT gate push-back, so the registers power-up to a `0` logic level. For additional details, refer to the appropriate device family handbook.

Optionally you can force all registers into their appropriate values after reset through an explicit reset signal. This technique allows to reset the device after power-up to restore the proper state.

Synchronizing the device architecture's external or combinational logic before driving the register's asynchronous control ports allows for more stable designs and avoids potential glitches.

Related Information

Recommended Design Practices on page 72

2.5.1.1. Specifying a Power-Up Value

Options available in synthesis tools allow you to specify power-up conditions for the design. Quartus Prime Pro Edition synthesis provides the **Power-Up Level** logic option.

You can also specify the power-up level with an `altera_attribute` assignment in the source code. This attribute forces synthesis to perform NOT gate push-back, because synthesis tools cannot change the power-up states of core registers.

You can apply the **Power-Up Level** logic option to a specific register, or to a design entity, module, or sub design. When you assign this option, every register in that block receives the value. Registers power up to `0` by default. Therefore, you can use this assignment to force all registers to power-up to `1` using NOT gate push-back.

Setting the **Power-Up Level** to a logic level of **high** for a large design entity could degrade the quality of results due to the number of inverters that requires. In some situations, this design style causes issues due to `enable` signal inference or secondary control logic inference. It may also be more difficult to migrate this type of designs.

Some synthesis tools can also read the default or initial values for registered signals and implement this behavior in the device. For example, Quartus Prime Pro Edition synthesis converts default values for registered signals into **Power-Up Level** settings. When the Quartus Prime software reads the default values, the synthesized behavior matches the power-up state of the HDL code during a functional simulation.

Example 37. Verilog Register with High Power-Up Value

```
reg q = 1'b1; //q has a default value of '1'

always @ (posedge clk)
```

```
begin
    q <= d;
end
```

Example 38. VHDL Register with High Power-Up Level

```
SIGNAL q : STD_LOGIC := '1'; -- q has a default value of '1'

PROCESS (clk, reset)
BEGIN
    IF (rising_edge(clk)) THEN
        q <= d;
    END IF;
END PROCESS;
```

Your design may contain undeclared default power-up conditions based on signal type. If you declare a VHDL register signal as an integer, Quartus Prime synthesis uses the left end of the integer range as the power-up value. For the default signed integer type, the default power-up value is the highest magnitude negative integer (`100…001`). For an unsigned integer type, the default power-up value is `0`.

Note: If the target device architecture does not support two asynchronous control signals, such as `aclr` and `aload`, you cannot set a different power-up state and reset state. If the NOT gate push-back algorithm creates logic to set a register to `1`, that register powers-up high. If you set a different power-up condition through a synthesis attribute or initial value, synthesis ignores the power-up level.

2.5.2. Secondary Register Control Signals Such as Clear and Clock Enable

The registers in Altera FPGAs provide a number of secondary control signals. Use these signals to implement control logic for each register without using extra logic cells. Altera FPGA device families vary in their support for secondary signals, so consult the device family data sheet to verify which signals are available in your target device.

To make the most efficient use of the signals in the device, ensure that HDL code matches the device architecture as closely as possible. The control signals have a certain priority due to the nature of the architecture. Your HDL code must follow that priority where possible.

Your synthesis tool can emulate any control signals using regular logic, so achieving functionally correct results is always possible. However, if your design requirements allow flexibility in controlling use and priority of control signals, match your design to the target device architecture to achieve the most efficient results. If the priority of the signals in your design is not the same as that of the target architecture, you may require extra logic to implement the control signals. This extra logic uses additional device resources, and can cause additional delays for the control signals.

In certain cases, using logic other than the dedicated control logic in the device architecture can have a larger impact. For example, the `clock enable` signal has priority over the synchronous `reset` or `clear` signal in the device architecture. The `clock enable` turns off the clock line in the LAB, and the `clear` signal is synchronous. Therefore, in the device architecture, the synchronous clear takes effect only when a clock edge occurs.

If you define a register with a synchronous `clear` signal that has priority over the `clock enable` signal, Quartus Prime synthesis emulates the clock enable functionality using data inputs to the registers. You cannot apply a Clock Enable Multicycle constraint, because the emulated functionality does not use the `clock enable` port of the register. In this case, using a different priority causes unexpected results with an assignment to the `clock enable` signal.

The signal order is the same for all Altera FPGA device families. However, not all device families provide every signal. The priority order is:

1. Asynchronous Clear (`clrn`)—highest priority
2. Enable (`ena`)
3. Synchronous Clear (`sclr`)
4. Synchronous Load (`sload`)
5. Data In (`data`)—lowest priority

The priority order for secondary control signals in Altera FPGA devices differs from the order for other vendors' FPGA devices. If your design requirements are flexible regarding priority, verify that the secondary control signals meet design performance requirements when migrating designs between FPGA vendors. To achieve the best results. try to match your target device architecture.

Example 39. Verilog D-type Flipflop bus with Secondary Signals

This module uses all Arria 10 DFF secondary signals: `clrn`, `ena`, `sclr`, and `sload`. Note that it instantiates 8-bit bus of DFFs rather than a single DFF, because synthesis infers some secondary signals only if there are multiple DFFs with the same secondary signal.

```
module top(clk, clrn, sclr, sload, ena, data, sdata, q);
    input clk, clrn, sclr, sload, ena;
    input [7:0] data, sdata;
    output [7:0] q;
    reg [7:0] q;
    always @ (posedge clk or posedge clrn)
        begin
        if (clrn)
            q <= 8'b0;
        else if (ena)
            begin
            if (sclr)
                q <= 8'b0;
            else if (!sload)
                q <= data;
            else
                q <= sdata;
            end
        end
endmodule
```

Related Information

Quartus® Prime Timing Analyzer Cookbook

2.5.3. Latches

A latch is a small combinational loop that holds the value of a signal until a new value is assigned. Synthesis tools can infer latches from HDL code when you did not intend to use a latch. If you do intend to infer a latch, it is important to infer it correctly to guarantee correct device operation.

Note: Design without the use of latches whenever possible.

Related Information

Avoid Unintended Latch Inference on page 75

2.5.3.1. Avoid Unintentional Latch Generation

When you design combinational logic, certain coding styles can create an unintentional latch. For example, when `CASE` or `IF` statements do not cover all possible input conditions, synthesis tools can infer latches to hold the output if a new output value is not assigned. Check your synthesis tool messages for references to inferred latches.

If your code unintentionally creates a latch, modify your RTL to remove the latch:

- Synthesis infers a latch when HDL code assigns a value to a signal outside of a clock edge (for example, with an asynchronous `reset`), but the code does not assign a value in an edge-triggered design block.
- Unintentional latches also occur when HDL code assigns a value to a signal in an edge-triggered design block, but synthesis optimizations remove that logic. For example, when a `CASE` or `IF` statement tests a condition that only evaluates to FALSE, synthesis removes any logic or signal assignment in that statement during optimization. This optimization may result in the inference of a latch for the signal.
- Omitting the final `ELSE` or `WHEN OTHERS` clause in an `IF` or `CASE` statement can also generate a latch. Don't care (`X`) assignments on the default conditions are useful in preventing latch generation. For the best logic optimization, assign the default `CASE` or final `ELSE` value to don't care (`X`) instead of a logic value.

In Verilog HDL designs, use the `full_case` attribute to treat unspecified cases as don't care values (`X`). However, since the `full_case` attribute is synthesis-only, it can cause simulation mismatches, because simulation tools still treat the unspecified cases as latches.

Example 40. VHDL Code Preventing Unintentional Latch Creation

Without the final `ELSE` clause, the following code creates unintentional latches to cover the remaining combinations of the `SEL` inputs. When you are targeting a Stratix series device with this code, omitting the final `ELSE` condition can cause synthesis tools to use up to six LEs, instead of the three it uses with the `ELSE` statement. Additionally, assigning the final `ELSE` clause to `1` instead of `X` can result in slightly more LEs, because synthesis tools cannot perform as much optimization when you specify a constant value as opposed to a don't care value.

```
LIBRARY ieee;
USE IEEE.std_logic_1164.all;

ENTITY nolatch IS
    PORT (a,b,c: IN STD_LOGIC;
        sel: IN STD_LOGIC_VECTOR (4 DOWNTO 0);
```

```
        oput: OUT STD_LOGIC);
END nolatch;

ARCHITECTURE rtl OF nolatch IS
BEGIN
    PROCESS (a,b,c,sel) BEGIN
        IF sel = "00000" THEN
            oput <= a;
        ELSIF sel = "00001" THEN
            oput <= b;
        ELSIF sel = "00010" THEN
            oput <= c;
        ELSE                    --- Prevents latch inference
            oput <= 'X'; --/
        END IF;
    END PROCESS;
END rtl;
```

2.5.3.2. Inferring Latches Correctly

Synthesis tools can infer a latch that does not exhibit the glitch and timing hazard problems typically associated with combinational loops. Quartus Prime Pro Edition software reports latches that synthesis inferred in the **User-Specified and Inferred Latches** section of the Compilation Report. This report indicates whether the latch presents a timing hazard, and the total number of user-specified and inferred latches.

Note: In some cases, timing analysis does not completely model latch timing. As a best practice, avoid latches unless required by the design and you fully understand the impact.

If latches or combinational loops in the design do not appear in the **User Specified and Inferred Latches** section, then Quartus Prime synthesis did not infer the latch as a safe latch, so the latch is not considered glitch-free.

All combinational loops listed in the **Analysis & Synthesis Logic Cells Representing Combinational Loops** table in the Compilation Report are at risk of timing hazards. These entries indicate possible problems with the design that require further investigation. However, correct designs can include combinational loops. For example, it is possible that the combinational loop cannot be sensitized. This occurs when there is an electrical path in the hardware, but either:

- The designer knows that the circuit never encounters data that causes that path to be activated, or
- The surrounding logic is set up in a mutually exclusive manner that prevents that path from ever being sensitized, independent of the data input.

For 6-input LUT-based devices, Quartus Prime synthesis implements all latch inputs with a single adaptive look-up table (ALUT) in the combinational loop. Therefore, all latches in the **User-Specified and Inferred Latches** table are free of timing hazards when a single input changes.

If Quartus Prime synthesis report lists a latch as a safe latch, other optimizations, such as physical synthesis netlist optimizations in the Fitter, maintain the hazard-free performance. To ensure hazard-free behavior, only one control input can change at a time. Changing two inputs simultaneously, such as deasserting `set` and `reset` at the same time, or changing data and enable at the same time, can produce incorrect behavior in any latch.

Quartus Prime synthesis infers latches from `always` blocks in Verilog HDL and `process` statements in VHDL. However, Quartus Prime synthesis does not infer latches from continuous assignments in Verilog HDL, or concurrent signal assignments in VHDL. These rules are the same as for register inference. The Quartus Prime synthesis infers registers or flipflops only from `always` blocks and `process` statements.

Example 41. Verilog HDL Set-Reset Latch

```
module simple_latch (
   input SetTerm,
   input ResetTerm,
   output reg LatchOut
   );
   always @ (SetTerm or ResetTerm) begin
      if (SetTerm)
         LatchOut = 1'b1;
      else if (ResetTerm)
         LatchOut = 1'b0;
   end
endmodule
```

Example 42. VHDL Data Type Latch

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.all;
ENTITY simple_latch IS
    PORT (
        enable, data    : IN STD_LOGIC;
        q               : OUT STD_LOGIC
    );
END simple_latch;
ARCHITECTURE rtl OF simple_latch IS
BEGIN
    latch : PROCESS (enable, data)
        BEGIN
        IF (enable = '1') THEN
            q <= data;
        END IF;
    END PROCESS latch;
END rtl;
```

The following example shows a Verilog HDL continuous assignment that does not infer a latch in the Quartus Prime software:

Example 43. Verilog Continuous Assignment Does Not Infer Latch

```
assign latch_out = (~en & latch_out) | (en & data);
```

The behavior of the assignment is similar to a latch, but it may not function correctly as a latch, and its timing is not analyzed as a latch. Quartus Prime Pro Edition synthesis also creates safe latches when possible for instantiations of a latch IP core. The Latch IP allows you to define a latch with any combination of data, enable, set, and `reset` inputs. The same limitations apply for creating safe latches as for inferring latches from HDL code.

Inferring the Latch IP core in another synthesis tool ensures that Quartus Prime synthesis also recognizes the implementation as a latch. If a third-party synthesis tool implements a latch using the Latch IP core, Quartus Prime Pro Edition synthesis reports the latch in the **User-Specified and Inferred Latches** table, in the same

manner as it lists latches you define in HDL source code. The coding style necessary to produce an Latch IP core implementation depends on the synthesis tool. Some third-party synthesis tools list the number of Latch IP cores that are inferred.

The Fitter uses global routing for control signals, including signals that synthesis identifies as latch enables. In some cases, the global insertion delay decreases timing performance. If necessary, you can turn off the **Quartus Prime Global Signal** logic option to manually prevent the use of global signals. The **Global & Other Fast Signals** table in the Compilation Report reports Global latch enables.

2.6. General Coding Guidelines

This section describes how coding styles impact synthesis of HDL code into the target Altera FPGA devices. You can improve your design efficiency and performance by following these recommended coding styles, and designing logic structures to match the appropriate device architecture.

2.6.1. Tri-State Signals

Use tri-state signals only when they are attached to top-level bidirectional or output pins.

Avoid lower-level bidirectional pins. Also avoid using the `Z` logic value unless it is driving an output or bidirectional pin. Even though some synthesis tools implement designs with internal tri-state signals correctly in Altera FPGA devices using multiplexer logic, do not use this coding style for Altera FPGA designs.

Note: In hierarchical block-based design flows, a hierarchical boundary cannot contain any bidirectional ports, unless the lower-level bidirectional port is connected directly through the hierarchy to a top-level output pin without connecting to any other design logic. If you use boundary tri-states in a lower-level block, synthesis software must push the tri-states through the hierarchy to the top level to make use of the tri-state drivers on output pins of Altera FPGA devices. Because pushing tri-states requires optimizing through hierarchies, lower-level tri-states are restricted with block-based design methodologies.

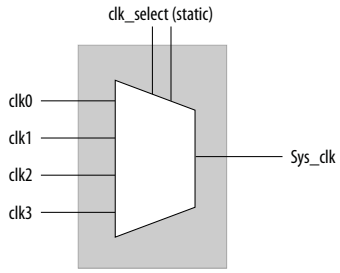
2.6.2. Clock Multiplexing

Clock multiplexing is sometimes used to operate the same logic function with different clock sources. This type of logic can introduce glitches that create functional problems. The delay inherent in the combinational logic can also lead to timing problems. Clock multiplexers trigger warnings from a wide range of design rule check and timing analysis tools.

Use dedicated hardware to perform clock multiplexing when it is available, instead of using multiplexing logic. For example, you can use the Clock Switchover feature or the Clock Control Block available in certain Altera FPGA devices. These dedicated hardware blocks avoid glitches, ensure that you use global low-skew routing lines, and avoid any possible hold time problems on the device due to logic delay on the clock line. Altera FPGA devices also support dynamic PLL reconfiguration, which is the safest and most robust method of changing clock rates during device operation.

If your design has too many clocks to use the clock control block, or if dynamic reconfiguration is too complex for your design, you can implement a clock multiplexer in logic cells. However, if you use this implementation, consider simultaneous toggling inputs and ensure glitch-free transitions.

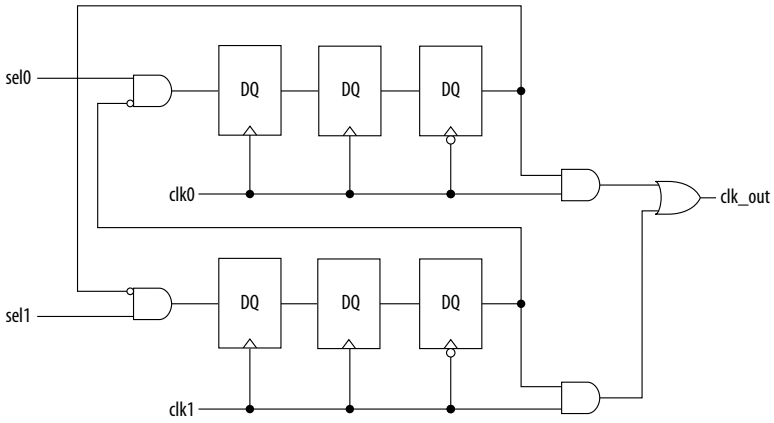
Figure 3. Simple Clock Multiplexer in a 6-Input LUT



Each device datasheet describes how LUT outputs can glitch during a simultaneous toggle of input signals, independent of the LUT function. Even though the 4:1 MUX function does not generate detectable glitches during simultaneous data input toggles, some cell implementations of multiplexing logic exhibit significant glitches, so this clock mux structure is not recommended. An additional problem with this implementation is that the output behaves erratically during a change in the `clk_select` signals. This behavior could create timing violations on all registers fed by the system clock and result in possible metastability.

A more sophisticated clock select structure can eliminate the simultaneous toggle and switching problems.

Figure 4. Glitch-Free Clock Multiplexer Structure



You can generalize this structure for any number of clock channels. The design ensures that no clock activates until all others are inactive for at least a few cycles, and that activation occurs while the clock is low. The design applies a `synthesis_keep` directive to the AND gates on the right side, which ensures there are no simultaneous toggles on the input of the `clk_out` OR gate.

Note: Switching from clock A to clock B requires that clock A continue to operate for at least a few cycles. If clock A stops immediately, the design sticks. The select signals are implemented as a "one-hot" control in this example, but you can use other encoding if you prefer. The input side logic is asynchronous and is not critical. This design can tolerate extreme glitching during the switch process.

Example 44. Verilog HDL Clock Multiplexing Design to Avoid Glitches

This example works with Verilog-2001.

```
module clock_mux (clk,clk_select,clk_out);

    parameter num_clocks = 4;

    input [num_clocks-1:0] clk;
    input [num_clocks-1:0] clk_select; // one hot
    output clk_out;

    genvar i;

    reg [num_clocks-1:0] ena_r0;
    reg [num_clocks-1:0] ena_r1;
    reg [num_clocks-1:0] ena_r2;
    wire [num_clocks-1:0] qualified_sel;

    // A look-up-table (LUT) can glitch when multiple inputs
    // change simultaneously. Use the keep attribute to
    // insert a hard logic cell buffer and prevent
    // the unrelated clocks from appearing on the same LUT.

    wire [num_clocks-1:0] gated_clks /* synthesis keep */;

    initial begin
        ena_r0 = 0;
        ena_r1 = 0;
        ena_r2 = 0;
    end

    generate
        for (i=0; i<num_clocks; i=i+1)
        begin : lp0
            wire [num_clocks-1:0] tmp_mask;
            assign tmp_mask = {num_clocks{1'b1}} ^ (1 << i);

            assign qualified_sel[i] = clk_select[i] & (~|(ena_r2 & tmp_mask));

            always @(posedge clk[i]) begin
                ena_r0[i] <= qualified_sel[i];
                ena_r1[i] <= ena_r0[i];
            end

            always @(negedge clk[i]) begin
                ena_r2[i] <= ena_r1[i];
            end

            assign gated_clks[i] = clk[i] & ena_r2[i];
        end
    endgenerate

    // These will not exhibit simultaneous toggle by construction
    assign clk_out = |gated_clks;

endmodule
```

2.6.3. Adder Trees

Structuring adder trees appropriately to match your targeted Altera FPGA device architecture can provide significant improvements in your design's efficiency and performance.

A good example of an application using a large adder tree is a finite impulse response (FIR) correlator. Using a pipelined binary or ternary adder tree appropriately can greatly improve the quality of your results.

2.6.3.1. Architectures with 6-Input LUTs in Adaptive Logic Modules

In Altera FPGA device families with 6-input LUT in their basic logic structure, ALMs can simultaneously add three bits. Take advantage of this feature by restructuring your code for better performance.

Although code targeting 4-input LUT architectures compiles successfully for 6-input LUT devices, the implementation can be inefficient. For example, to take advantage of the 6-input adaptive ALUT, you must rewrite large pipelined binary adder trees designed for 4-input LUT architectures. By restructuring the tree as a ternary tree, the design becomes much more efficient, significantly improving density utilization.

Example 45. Verilog HDL Pipelined Ternary Tree

The example shows a pipelined adder, but partitioning your addition operations can help you achieve better results in non-pipelined adders as well. If your design is not pipelined, a ternary tree provides much better performance than a binary tree. For example, depending on your synthesis tool, the HDL code `sum = (A + B + C) + (D + E)` is more likely to create the optimal implementation of a 3-input adder for `A + B + C` followed by a 3-input adder for `sum1 + D + E` than the code without the parentheses. If you do not add the parentheses, the synthesis tool may partition the addition in a way that is not optimal for the architecture.

```
module ternary_adder_tree (a, b, c, d, e, clk, out);
    parameter width = 16;
    input [width-1:0] a, b, c, d, e;
    input    clk;
    output [width-1:0] out;

    wire [width-1:0] sum1, sum2;
    reg [width-1:0] sumreg1, sumreg2;
    // registers

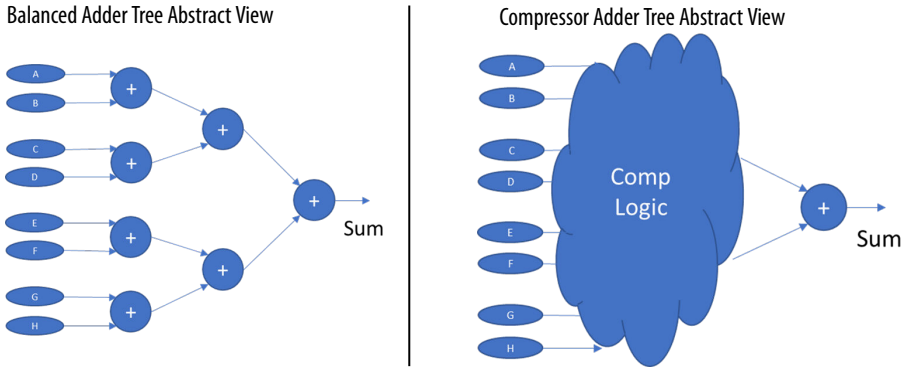
    always @ (posedge clk)
        begin
            sumreg1 <= sum1;
            sumreg2 <= sum2;
        end

    // 3-bit additions
    assign sum1 = a + b + c;
    assign sum2 = sumreg1 + d + e;
    assign out = sumreg2;
endmodule
```

2.6.3.2. Change Adder Tree Styles

Because ALMs can implement functions of up to six inputs, you can improve the performance of certain designs by using a compressor implementation for adder trees, rather than the default balanced binary tree implementation. The expected downside tradeoff of the compressor implementation is the use of more ALM logic resources. However the overall logic depth is lower, and the final timing characteristics improve.

Figure 5. Balanced Binary Versus Compressor Style Adder Trees



For designs that may benefit, you can apply the **Use Compressor Implementation** (`USE_COMPRESSOR_IMPLEMENTATION`) global, entity, or instance assignment to specify whether the Compiler synthesizes adder trees as balanced binary trees, or as compressor style trees.

You can specify this assignment in the Assignment Editor, or with the following assignment in the `.qsf`.

```
set_instance_assignment -name USE_COMPRESSOR_IMPLEMENTATION ALWAYS -to <foo>
```

The following options are available for this assignment:

Table 2. Use Compressor Implementation Assignment Options

Option	Description
Always	The Compiler always synthesizes all adder trees with this assignment as compressor style trees. There is a limit of at least 2 non-constant operands before this triggers (otherwise synthesis implements a binary add or a pure-LUT implementation depending on size).
Never	The Compiler never synthesizes the assigned adder tree as a compressor. The Compiler synthesizes the adder as either a balanced binary tree, or if sufficiently small, in pure LUTs.
Auto	This setting currently behaves the same as the **Never** setting. The Compiler synthesizes the adder as either a balanced binary tree, or if sufficiently small, in pure LUTs. This setting never uses compressor style adder trees.

2.6.4. State Machine HDL Guidelines

Synthesis tools can recognize and encode Verilog HDL and VHDL state machines during synthesis. This section presents guidelines to secure the best results when you use state machines.

Synthesis tools that can recognize a piece of code as a state machine can perform optimizations that improve the design area and performance. For example, the tool can recode the state variables to improve the quality of results, or optimize other parts of the design through known properties of state machines.

To achieve the best results, synthesis tools often use one-hot encoding for FPGA devices and minimal-bit encoding for CPLD devices, although the choice of implementation can vary for different state machines and different devices. Refer to the synthesis tool documentation for techniques to control the encoding of state machines.

To ensure proper recognition and inference of state machines and to improve the quality of results, observe the following guidelines for both Verilog HDL and VHDL:

- Assign default values to outputs derived from the state machine so that synthesis does not generate unwanted latches.
- Separate state machine logic from all arithmetic functions and datapaths, including assigning output values.
- For designs in which more than one state perform the same operation, define the operation outside the state machine, and direct the output logic of the state machine to use this value.
- Ensure a defined power-up state with a simple asynchronous or synchronous `reset`. In designs where the state machine contains more elaborate `reset` logic, such as both an asynchronous `reset` and an asynchronous load, the Quartus Prime software infers regular logic rather than a state machine.

If a state machine enters an illegal state due to a problem with the device, the design likely ceases to function correctly until the next `reset` of the state machine. Synthesis tools do not provide for this situation by default. The same issue applies to any other registers if there is some fault in the system. A `default` or `when others` clause does not affect this operation, assuming that the design never deliberately enters this state. Synthesis tools remove any logic generated by a default state if it is not reachable by normal state machine operation.

Many synthesis tools (including Quartus Prime synthesis) have an option to implement a safe state machine. The Quartus Prime software inserts extra logic to detect illegal states and force the state machine's transition to the `reset` state. Safe state machines are useful when the state machine can enter an illegal state, for example, when a state machine has control inputs that originate in another clock domain, such as the control logic for a dual-clock FIFO.

This option protects state machines by forcing them into the `reset` state. All other registers in the design are not protected this way. As a best practice for designs with asynchronous inputs, use a synchronization register chain instead of relying on the safe state machine option.

2.6.4.1. State Machine Processing

The default state machine encoding uses one-hot encoding for FPGA devices and minimal-bits encoding for CPLDs. These settings achieve the best results on average, but another encoding style might be more appropriate for your design.

Use the **State Machine Processing** option (**Assignments ➤ Settings ➤ Compiler Settings ➤ Advanced Settings (Synthesis)**) to control the encoding style of your design.

One-Hot State Machine Encoding

For one-hot encoding, the Quartus Prime software does not guarantee that each state has one bit set to one and all other bits set to zero. Quartus Prime synthesis creates one-hot register encoding with standard one-hot encoding and then inverts the first bit. This results in an initial state with all zero values, and the remaining states have two 1 values. Quartus Prime synthesis encodes the initial state with all zeros for the state machine power-up because all device registers power up to a low value.

This encoding has the same properties as true one-hot encoding: the software recognizes each state by the value of one bit. For example, in a one-hot-encoded state machine with five states, including an initial or reset state, the software uses the following register encoding:

```
State 0    0 0 0 0 0
State 1    0 0 0 1 1
State 2    0 0 1 0 1
State 3    0 1 0 0 1
State 4    1 0 0 0 1
```

User-Encoded State Machine Encoding

If you set the **State Machine Processing** logic option to **User-Encoded** in a Verilog HDL design, the software starts with the original design values for the state constants. For example, a Verilog HDL design can contain the following declaration:

```
parameter S0 = 4'b1010, S1 = 4'b0101, ...
```

If the software infers the states `S0, S1,...` the software uses the encoding `4'b1010, 4'b0101,...`. If necessary, the software inverts bits in a user-encoded state machine to ensure that all bits of the reset state of the state machine are zero.

Remember: You can view the state machine encoding from the Compilation Report under the State Machines of the Analysis & Synthesis Report.

To assign your own state encoding with the **User-Encoded** setting of the **State Machine Processing** option in a VHDL design, you must apply specific binary encoding to the elements of an enumerated type because enumeration literals have no numeric values in VHDL. Use the `syn_encoding` synthesis attribute to apply your encoding values.

2.6.4.2. State Machine Power-Up

In Stratix 10 devices, registers do not necessarily power-up in the same clock cycle if they are not in the same sector. This fact can cause issues with state machines if the state machine enters an undefined state.

One-hot encoded state machines are especially susceptible to this issue, as the number of undefined states is large compared to the number of legal states. Retiming also increases the risk of this issue because when state registers retime across logic or routing, it becomes more likely that the different state registers of one state machine are in different sectors.

To mitigate this risk, the Compiler automatically uses **Safe State Machine** for any state machine of 6 or less states for Stratix 10 designs. This **Safe State Machine** setting forces the state machines back into the reset state if they enter an undefined state. The Compiler does not automatically use **Safe State Machine** for state machines of more than 6 states, or for Arria 10 or Cyclone 10 GX devices, because the effect on the quality of results can be significant.

2.6.4.3. Verilog HDL State Machines

To ensure proper recognition and inference of Verilog HDL state machines, observe the following additional Verilog HDL guidelines.

Refer to your synthesis tool documentation for specific coding recommendations. If the synthesis tool doesn't recognize and infer the state machine, the tool implements the state machine as regular logic gates and registers, and the state machine doesn't appear as a state machine in the **Analysis & Synthesis** section of the Quartus Prime Compilation Report. In this case, Quartus Prime synthesis does not perform any optimizations specific to state machines.

- If you are using the SystemVerilog standard, use enumerated types to describe state machines.
- Represent the states in a state machine with the parameter data types in Verilog-1995 and Verilog-2001, and use the parameters to make state assignments. This parameter implementation makes the state machine easier to read and reduces the risk of errors during coding.
- Do not directly use integer values for state variables, such as `next_state <= 0`. However, using an integer does not prevent inference in the Quartus Prime software.
- Quartus Prime software doesn't infer a state machine if the state transition logic uses arithmetic similar to the following example:

```
case (state)
    0: begin
        if (ena) next_state <= state + 2;
        else next_state <= state + 1;
        end
    1: begin
    ...
endcase
```

- Quartus Prime software doesn't infer a state machine if the state variable is an output.
- Quartus Prime software doesn't infer a state machine for signed variables.

2.6.4.3.1. Verilog-2001 State Machine Coding Example

The following module `verilog_fsm` is an example of a typical Verilog HDL state machine implementation. This state machine has five states.

The asynchronous reset sets the variable `state` to `state_0`. The sum of `in_1` and `in_2` is an output of the state machine in `state_1` and `state_2`. The difference (`in_1` – `in_2`) is also used in `state_1` and `state_2`. The temporary variables `tmp_out_0` and `tmp_out_1` store the sum and the difference of `in_1` and `in_2`. Using these temporary variables in the various states of the state machine ensures proper resource sharing between the mutually exclusive states.

Example 46. Verilog-2001 State Machine

```
module verilog_fsm (clk, reset, in_1, in_2, out);
    input clk, reset;
    input [3:0] in_1, in_2;
    output [4:0] out;
    parameter state_0 = 3'b000;
    parameter state_1 = 3'b001;
    parameter state_2 = 3'b010;
    parameter state_3 = 3'b011;
    parameter state_4 = 3'b100;

    reg [4:0] tmp_out_0, tmp_out_1, tmp_out_2;
    reg [2:0] state, next_state;

    always @ (posedge clk or posedge reset)
    begin
        if (reset)
            state <= state_0;
        else
            state <= next_state;
    end
    always @ (*)
    begin
        tmp_out_0 = in_1 + in_2;
        tmp_out_1 = in_1 - in_2;
        case (state)
            state_0: begin
               tmp_out_2 = in_1 + 5'b00001;
               next_state = state_1;
            end
            state_1: begin
                if (in_1 < in_2) begin
                    next_state = state_2;
                    tmp_out_2 = tmp_out_0;
                end
                else begin
                    next_state = state_3;
                    tmp_out_2 = tmp_out_1;
                end
            end
            state_2: begin
                tmp_out_2 = tmp_out_0 - 5'b00001;
                next_state = state_3;
            end
            state_3: begin
                tmp_out_2 = tmp_out_1 + 5'b00001;
                next_state = state_0;
            end
            state_4:begin
                tmp_out_2 = in_2 + 5'b00001;
                next_state = state_0;
            end
            default:begin
                tmp_out_2 = 5'b00000;
                next_state = state_0;
            end
        endcase
```

```
    end
    assign out = tmp_out_2;
endmodule
```

You can achieve an equivalent implementation of this state machine by using `‘define` instead of the `parameter` data type, as follows:

```
‘define state_0 3'b000
‘define state_1 3'b001
‘define state_2 3'b010
‘define state_3 3'b011
‘define state_4 3'b100
```

In this case, you assign `` `state_x `` instead of `state_x` to `state` and `next_state`, for example:

```
next_state <= ‘state_3;
```

Note: Although Altera supports the `‘define` construct, use the `parameter` data type, because it preserves the state names throughout synthesis.

2.6.4.3.2. SystemVerilog State Machine Coding Example

Use the following coding style to describe state machines in SystemVerilog.

Example 47. SystemVerilog State Machine Using Enumerated Types

The module `enum_fsm` is an example of a SystemVerilog state machine implementation that uses enumerated types.

In Quartus Prime Pro Edition synthesis, the enumerated type that defines the states for the state machine must be of an unsigned integer type. If you do not specify the enumerated type as `int unsigned`, synthesis uses a signed `int` type by default. In this case, the Quartus Prime software synthesizes the design, but does not infer or optimize the logic as a state machine.

```
module enum_fsm (input clk, reset, input int data[3:0], output int o);
enum int unsigned { S0 = 0, S1 = 2, S2 = 4, S3 = 8 } state, next_state;
always_comb begin : next_state_logic
      next_state = S0;
      case(state)
        S0: next_state = S1;
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = S3;
      endcase
end
always_comb begin
      case(state)
         S0: o = data[3];
         S1: o = data[2];
         S2: o = data[1];
         S3: o = data[0];
      endcase
end
always_ff@(posedge clk or negedge reset) begin
      if(~reset)
         state <= S0;
      else
         state <= next_state;
end
endmodule
```

2.6.4.4. VHDL State Machines

To ensure proper recognition and inference of VHDL state machines, represent the different states with enumerated types, and use the corresponding types to make state assignments.

This implementation makes the state machine easier to read, and reduces the risk of errors during coding. If your RTL does not represent states with an enumerated type, Quartus Prime synthesis (and other synthesis tools) do not recognize the state machine. Instead, synthesis implements the state machine as regular logic gates and registers. Consequently, and the state machine does not appear in the state machine list of the Quartus Prime Compilation Report, **Analysis & Synthesis** section. Moreover, Quartus Prime synthesis does not perform any of the optimizations that are specific to state machines.

2.6.4.4.1. VHDL State Machine Coding Example

The following state machine has five states. The asynchronous reset sets the variable `state` to `state_0`.

The sum of `in1` and `in2` is an output of the state machine in `state_1` and `state_2`. The difference (`in1 - in2`) is also used in `state_1` and `state_2`. The temporary variables `tmp_out_0` and `tmp_out_1` store the sum and the difference of `in1` and `in2`. Using these temporary variables in the various states of the state machine ensures proper resource sharing between the mutually exclusive states.

Example 48. VHDL State Machine

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;
ENTITY vhdl_fsm IS
   PORT(
      clk: IN STD_LOGIC;
      reset: IN STD_LOGIC;
      in1: IN UNSIGNED(4 downto 0);
      in2: IN UNSIGNED(4 downto 0);
      out_1: OUT UNSIGNED(4 downto 0)
      );
END vhdl_fsm;
ARCHITECTURE rtl OF vhdl_fsm IS
   TYPE Tstate IS (state_0, state_1, state_2, state_3, state_4);
   SIGNAL state: Tstate;
   SIGNAL next_state: Tstate;
BEGIN
   PROCESS(clk, reset)
   BEGIN
      IF reset = '1' THEN
            state <=state_0;
      ELSIF rising_edge(clk) THEN
            state <= next_state;
      END IF;
   END PROCESS;
PROCESS (state, in1, in2)
      VARIABLE tmp_out_0: UNSIGNED (4 downto 0);
      VARIABLE tmp_out_1: UNSIGNED (4 downto 0);
   BEGIN
      tmp_out_0 := in1 + in2;
      tmp_out_1 := in1 - in2;
      CASE state IS
         WHEN state_0 =>
            out_1 <= in1;
```

```
            next_state <= state_1;
         WHEN state_1 =>
            IF (in1 < in2) then
               next_state <= state_2;
               out_1 <= tmp_out_0;
            ELSE
               next_state <= state_3;
               out_1 <= tmp_out_1;
            END IF;
         WHEN state_2 =>
            IF (in1 < "0100") then
               out_1 <= tmp_out_0;
            ELSE
               out_1 <= tmp_out_1;
            END IF;
               next_state <= state_3;
         WHEN state_3 =>
               out_1 <= "11111";
               next_state <= state_4;
         WHEN state_4 =>
               out_1 <= in2;
               next_state <= state_0;
         WHEN OTHERS =>
               out_1 <= "00000";
               next_state <= state_0;
      END CASE;
   END PROCESS;
END rtl;
```

2.6.5. Multiplexer HDL Guidelines

Multiplexers form a large portion of the logic utilization in many FPGA designs. By optimizing your multiplexer logic, you ensure the most efficient implementation.

This section addresses common problems and provides design guidelines to achieve optimal resource utilization for multiplexer designs. The section also describes various types of multiplexers, and how they are implemented.

For more information, refer to the *Advanced Synthesis Cookbook*.

2.6.5.1. Quartus Prime Software Option for Multiplexer Restructuring

Quartus Prime Pro Edition synthesis provides the **Restructure Multiplexers** logic option that extracts and optimizes buses of multiplexers during synthesis. The default **Auto** for this option setting uses the optimization whenever beneficial for your design. You can turn the option on or off specifically to have more control over use.

Even with this Quartus Prime-specific option turned on, it is beneficial to understand how your coding style can be interpreted by your synthesis tool, and avoid the situations that can cause problems in your design.

2.6.5.2. Multiplexer Types

This section addresses how Quartus Prime synthesis creates multiplexers from various types of HDL code.

State machines, `CASE` statements, and `IF` statements are all common sources of multiplexer logic in designs. These HDL structures create different types of multiplexers, including binary multiplexers, selector multiplexers, and priority multiplexers.

The first step toward optimizing multiplexer structures for best results is to understand how Quartus Prime infers and implements multiplexers from HDL code.

2.6.5.2.1. Binary Multiplexers

Binary multiplexers select inputs based on binary-encoded selection bits.

Device families featuring 6-input look up tables (LUTs) are perfectly suited for 4:1 multiplexer building blocks (4 data and 2 select inputs). The extended input mode facilitates implementing 8:1 blocks, and the fractured mode handles residual 2:1 multiplexer pairs.

Example 49. Verilog HDL Binary-Encoded Multiplexers

```
case (sel)
   2'b00: z = a;
   2'b01: z = b;
   2'b10: z = c;
   2'b11: z = d;
endcase
```

2.6.5.2.2. Selector Multiplexers

Selector multiplexers have a separate select line for each data input. The select lines for the multiplexer are one-hot encoded. Quartus Prime commonly builds selector multiplexers as a tree of AND and OR gates.

Even though the implementation of a tree-shaped, N-input selector multiplexer is slightly less efficient than a binary multiplexer, in many cases the select signal is the output of a decoder. Quartus Prime synthesis combines the selector and decoder into a binary multiplexer.

Example 50. Verilog HDL One-Hot-Encoded `CASE` Statement

```
case (sel)
   4'b0001: z = a;
   4'b0010: z = b;
   4'b0100: z = c;
   4'b1000: z = d;
   default: z = 1'bx;
endcase
```

2.6.5.2.3. Priority Multiplexers

In priority multiplexers, the select logic implies a priority. The options to select the correct item must be checked in a specific order based on signal priority.

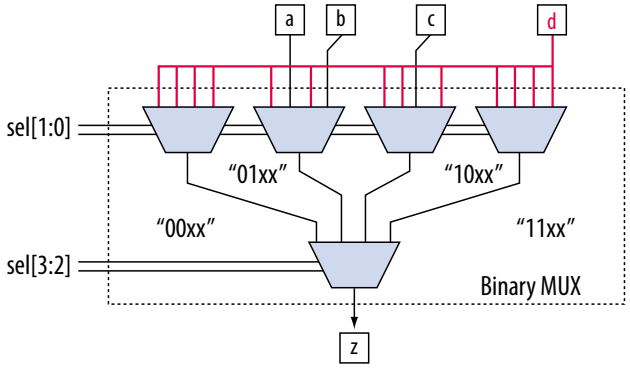
Synthesis tools commonly infer these structures from `IF`, `ELSE`, `WHEN`, `SELECT`, and `?:` statements in VHDL or Verilog HDL.

Example 51. VHDL IF Statement Implying Priority

The multiplexers form a chain, evaluating each condition or select bit sequentially.

```
IF cond1 THEN z <= a;
ELSIF cond2 THEN z <= b;
ELSIF cond3 THEN z <= c;
ELSE z <= d;
END IF;
```

Figure 6. Priority Multiplexer Implementation of an IF Statement



Depending on the number of multiplexers in the chain, the timing delay through this chain can become large, especially for device families with 4-input LUTs.

To improve the timing delay through the multiplexer, avoid priority multiplexers if priority is not required. If the order of the choices is not important to the design, use a `CASE` statement to implement a binary or selector multiplexer instead of a priority multiplexer. If delay through the structure is important in a multiplexed design requiring priority, consider recoding the design to reduce the number of logic levels to minimize delay, especially along your critical paths.

2.6.5.3. Implicit Defaults in `IF` Statements

`IF` statements in Verilog HDL and VHDL can simplify expressing conditions that do not easily lend themselves to a `CASE`-type approach. However, `IF` statements can result in complex multiplexer trees that are not easy for synthesis tools to optimize. In particular, all `IF` statements have an `ELSE` condition, even when not specified in the code. These implicit defaults can cause additional complexity in multiplexed designs.

You can simplify multiplexed logic and remove unneeded defaults with multiple methods. The optimal method is recoding the design, so the logic takes the structure of a 4:1 `CASE` statement. Alternatively, if priority is important, you can restructure the code to reduce default cases and flatten the multiplexer. Examine whether the default "`ELSE IF`" conditions are don't care cases. You can add a default `ELSE` statement to make the behavior explicit. Avoid unnecessary default conditions in the multiplexer logic to reduce the complexity and logic utilization that the design implementation requires.

2.6.5.4. `default` or `OTHERS` `CASE` Assignment

To fully specify the cases in a `CASE` statement, include a `default` (Verilog HDL) or `OTHERS` (VHDL) assignment.

This assignment is especially important in one-hot encoding schemes where many combinations of the select lines are unused. Specifying a case for the unused select line combinations gives the synthesis tool information about how to synthesize these cases, and is required by the Verilog HDL and VHDL language specifications.

For some designs you do not need to consider the outcome in the unused cases, because these cases are unreachable. For these types of designs, you can specify any value for the `default` or `OTHERS` assignment. However, the assignment value you choose can have a large effect on the logic utilization required to implement the design.

To obtain best results, explicitly define invalid `CASE` selections with a separate `default` or `OTHERS` statement, instead of combining the invalid cases with one of the defined cases.

If the value in the invalid cases is not important, specify those cases explicitly by assigning the `X` (don't care) logic value instead of choosing another value. This assignment allows your synthesis tool to perform the best area optimizations.

2.6.6. Cyclic Redundancy Check Functions

CRC computations are used heavily by communications protocols and storage devices to detect any corruption of data. These functions are highly effective; there is a very low probability that corrupted data can pass a 32-bit CRC check

CRC functions typically use wide XOR gates to compare the data. The way synthesis tools flatten and factor these XOR gates to implement the logic in FPGA LUTs can greatly impact the area and performance results for the design. XOR gates have a cancellation property that creates an exceptionally large number of reasonable factoring combinations, so synthesis tools cannot always choose the best result by default.

The 6-input ALUT has a significant advantage over 4-input LUTs for these designs. When properly synthesized, CRC processing designs can run at high speeds in devices with 6-input ALUTs.

The following guidelines help you improve the quality of results for CRC designs in Altera FPGA devices.

2.6.6.1. If Performance is Important, Optimize for Speed

To minimize area and depth of levels of logic, synthesis tools flatten XOR gates.

By default, Quartus Prime Pro Edition synthesis targets area optimization for XOR gates. Therefore, for more focus on depth reduction, set the synthesis optimization technique to speed.

Note: Flattening for depth sometimes causes a significant increase in area.

2.6.6.2. Use Separate CRC Blocks Instead of Cascaded Stages

Some designs optimize CRC to use cascaded stages (for example, four stages of 8 bits). In such designs, Quartus Prime synthesis uses intermediate calculations (such as the calculations after 8, 24, or 32 bits) depending on the data width.

This design is not optimal for FPGA devices. The XOR cancellations that Quartus Prime synthesis performs in CRC designs mean that the function does not require all the intermediate calculations to determine the final result. Therefore, forcing the use of intermediate calculations increases the area required to implement the function, as

well as increasing the logic depth because of the cascading. It is typically better to create full separate CRC blocks for each data width that you require in the design, and then multiplex them together to choose the appropriate mode at a given time

2.6.6.3. Use Separate CRC Blocks Instead of Allowing Blocks to Merge

Synthesis tools often attempt to optimize CRC designs by sharing resources and extracting duplicates in two different CRC blocks because of the factoring options in the XOR logic.

CRC logic allows significant reductions, but this works best when the Compiler optimizes CRC function separately. Check for duplicate extraction behavior if for designs with different CRC functions that are driven by common data signals or that feed the same destination signals.

For designs with poor quality results that have two CRC functions sharing logic you can ensure that the blocks are synthesized independently with one of the following methods:

- Define each CRC block as a separate design partition in a hierarchical compilation design flow.
- Synthesize each CRC block as a separate project in a third-party synthesis tool and then write a separate Verilog Quartus Mapping (`.vqm`) or EDIF netlist file for each.

2.6.6.4. Take Advantage of Latency if Available

If your design can use more than one cycle to implement the CRC functionality, adding registers and retiming the design can help reduce area, improve performance, and reduce power utilization.

If your synthesis tool offers a retiming feature (such as the Quartus Prime software **Perform gate-level register retiming** option), you can insert an extra bank of registers at the input and allow the retiming feature to move the registers for better results. You can also build the CRC unit half as wide and alternate between halves of the data in each clock cycle.

2.6.6.5. Save Power by Disabling CRC Blocks When Not in Use

CRC designs are heavy consumers of dynamic power because the logic toggles whenever there is a change in the design.

To save power, use clock enables to disable the CRC function for every clock cycle that the logic is not required. Some designs don't check the CRC results for a few clock cycles while other logic is performing. It is valuable to disable the CRC function even for this short amount of time.

2.6.6.6. Initialize the Device with the Synchronous Load (sload) Signal

CRC designs often require the data to be initialized to 1's before operation. In devices that support the `sload` signal, you can use this signal to set all registers in the design to 1's before operation.

To enable the `sload` signal, follow the coding guidelines in this chapter. After compilation you can check the register equations in the Chip Planner to ensure that the signal behaves as expected.

If you must force a register implementation using an `sload` signal, refer to *Designing with Low-Level Primitives User Guide* to see how you can use low-level device primitives.

Related Information

Secondary Register Control Signals Such as Clear and Clock Enable on page 42

2.6.7. Comparator HDL Guidelines

This section provides information about the different types of implementations available for comparators (<, >, or ==), and provides suggestions on how you can code the design to encourage a specific implementation. Synthesis tools, including Quartus Prime Pro Edition synthesis, use device and context-specific implementation rules, and select the best one for the design.

Synthesis tools implement the == comparator in general logic cells and the < comparison in either the carry chain or general logic cells. In devices with 6-input ALUTs, the carry chain can compare up to three bits per cell. Carry chain implementation tends to be faster than general logic on standalone benchmark test cases, but can result in lower performance on larger designs due to increased restrictions on the Fitter. The area requirement is similar for most input patterns. The synthesis tools select an appropriate implementation based on the input pattern.

You can guide the Quartus Prime Synthesis engine by choosing specific coding styles. To select a carry chain implementation explicitly, rephrase the comparison in terms of addition.

For example, the following coding style allows the synthesis tool to select the implementation, which is most likely using general logic cells in modern device families:

```
wire [6:0] a,b;
wire alb = a<b;
```

In the following coding style, the synthesis tool uses a carry chain (except for a few cases, such as when the chain is very short, or the signals `a` and `b` minimize to the same signal):

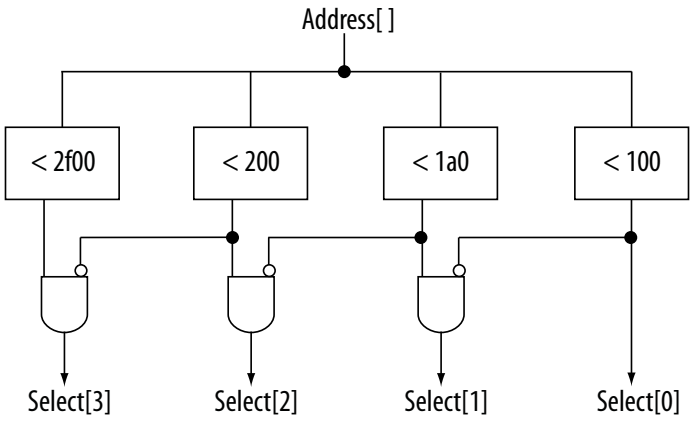
```
wire [6:0] a,b;
wire [7:0] tmp = a - b;
wire alb = tmp[7]
```

This second coding style uses the top bit of the `tmp` signal, which is 1 in two's complement logic if *a* is less than *b*, because the subtraction *a* - *b* results in a negative number.

If you have any information about the range of the input, you can use "don't care" values to optimize the design. This information is not available to the synthesis tool, so specific hand implementation of the logic can reduce the device area required to implement the comparator.

The following logic structure, which occurs frequently in address decoders, allows you to check whether a bus value is within a constant range with a small amount of logic area:

Figure 7. Example Logic Structure for Using Comparators to Check a Bus Value Range



2.6.8. Counter HDL Guidelines

The Quartus Prime synthesis engine implements counters in HDL code as an adder followed by registers, and makes available register control signals such as enable (`ena`), synchronous clear (`sclr`), and synchronous load (`sload`). For best area utilization, ensure that the up and down control or controls are expressed in terms of one addition operator, instead of two separate addition operators.

If you use the following coding style, your synthesis engine may implement two separate carry chains for addition:

```
out <= count_up ? out + 1 : out - 1;
```

For simple designs, the synthesis engine identifies this coding style and optimizes the logic. However, in complex designs, or designs with preserve pragmas, the Compiler cannot optimize all logic, so more careful coding becomes necessary.

The following coding style requires only one adder along with some other logic:

```
out <= out + (count_up ? 1 : -1);
```

This style makes more efficient use of resources and area, since it uses only one carry chain adder, and the –1 constant logic is implemented in the LUT before the adder.

2.7. Designing with Low-Level Primitives

Low-level HDL design is the practice of using low-level primitives and assignments to dictate a particular hardware implementation for a piece of logic. Low-level primitives are small architectural building blocks that assist you in creating your design.

With the Quartus Prime software, you can use low-level HDL design techniques to force a specific hardware implementation that can help you achieve better resource utilization or faster timing results.

Note: Using low-level primitives is an optional advanced technique to help with specific design challenges. For many designs, synthesizing generic HDL source code and Altera FPGA IP cores give you the best results.

Low-level primitives allow you to use the following types of coding techniques:

- Instantiate the logic cell or `LCELL` primitive to prevent Quartus Prime Pro Edition synthesis from performing optimizations across a logic cell
- Instantiate registers with specific control signals using `DFF` primitives
- Specify the creation of LUT functions by identifying the LUT boundaries
- Use I/O buffers to specify I/O standards, current strengths, and other I/O assignments
- Use I/O buffers to specify differential pin names in your HDL code, instead of using the automatically-generated negative pin name for each pair

For details about and examples of using these types of assignments, refer to the *Designing with Low-Level Primitives User Guide*.

2.8. Cross-Module Referencing (XMR) in HDL Code

Cross Module Referencing (XMR), also known as hierarchical reference, is enabled by default. It is a mechanism built into Verilog, SystemVerilog, and VHDL to globally reference nets in any hierarchy across modules, which means you can refer to any net of a particular module in a different module (irrespective of the hierarchy) directly without going through the ports. Hence, XMR can be a downward reference or an upward reference.

Note: XMR also works in mixed language designs.

You can use XMR to read from nets or write to a net in different hierarchies. XMR is helpful in debugging and verification, for example:

- Override or force any signal. For more information, refer to Using force Statements in HDL Code on page 67.
- Write cover points for functional coverage.
- Tap into any signal from anywhere in the entire design.

Consider the following hierarchy within the instance `top` of module `a`:

```
module a
  net x
  instance p of module b
    net x
    instance m of module d
      net x
  instance q of module c
    net x
    instance n of module e
      net x
  instance r of module b
     net x
     instance m of module d
       net x
```

In the above scenario, all modules consist of a net named `x`. By using full-path-based XMR, you can globally reference each net `x` anywhere within the hierarchy, as follows:

- `top.x`
- `top.p.x`
- `top.p.m.x`
- `top.q.x`
- `top.q.n.x`
- `top.r.x`
- `top.r.m`

XMR starts with searching within the current module. Then, it hierarchically searches downwards through child instances. If XMR is unresolved, it searches one step above in the hierarchy (parent module) and hierarchically downwards. It keeps going further in the hierarchy until it gets resolved. To avoid unexpected behavior, Altera recommends always using the hierarchy path where possible.

XMR Use Case Examples

Example 52. XMR of Signals in Higher Modules from Lower Modules

In the following example, the signal `d` in the `sub` module is assigned to the value `a` from the `top` module:

```
module top (input a, input b, output c, output d);
  sub inst1 (.a(a), .b(b), .c(c), .d(d));
endmodule

module sub (input a, input b, output c, output d);
  assign c = a & b;
  assign d = top.a;
endmodule
```

Example 53. XMR of Signals in Lower Modules from Higher Modules

In the following example, the `sub` module's value `a` assigns the output value `d`, given the complete path in the `top` module:

```
module top (input a, input b, output c, output d);
  sub inst1 (.a(a), .b(b), .c(c));
  assign d = inst1.a;
endmodule
module sub (input a, input b, output c);
  assign c = a & b;
endmodule
```

Example 54. XMR of Signals in `generate` Block

Consider the following example with a `generate` block where you write the value to `temp` at the top module and read the `out2` value from the `top` module:

```
module top (input [3:0] in1, in2, input clk, output [3:0] out1, out2);
  generate
    begin:blk1 sub inst (in1, clk, out1, temp);
  end:blk1
  endgenerate
```

```
//XMR read
assign out2 = top.blk1.inst.temp;
endmodule
```

Example 55. XMR From Inside an `always` Block

Consider the following example of XMR of signals within the `always_comb` construct, where the output value `d` in the `top` module is assigned from the `sub` module value `a`:

```
module top (input logic a, input logic b, output logic c, output logic d);
  sub inst1 (.a(a), .b(b), .c(c));
  always_comb d = inst1.a;

endmodule

module sub (input logic a, input logic b, output logic c);
  assign c = b & a;
endmodule
```

Limitations of XMR

The following are some limitations of XMR in the Quartus Prime software:

- XMR must be within the same design partition. For example, if there are partitions A and B, then in partition B, there cannot be any XMR to anything in partition A and vice versa.

  Note: The same partition requirement applies for global signals that are specified in a package and can be used in any module.
- Multiple-driven nets are not supported. So, you cannot use XMR to drive a net already driven by another signal.
- You can use XMR only in Verilog, SystemVerilog, and VHDL. It does not work for Text Design File (TDF) or Block Design File (BDF).

  Note: Altera does not recommend using XMR on SystemVerilog interfaces since they are prone to errors.
- You cannot use XMR to refer to signals inside an Altera IP core or a Signal Tap partition that is automatically created during synthesis.

Related Information

Synplify Pro for Microsemi Edition Language Support Reference Manual

2.9. Using `force` Statements in HDL Code

`force` statement in SystemVerilog is a continuous procedural assignment on a net or a variable. Applying a `force` statement to a net or variable overrides all other drivers to that net or variable. In simulation, you can use a `force` statement in conjunction with a `release` statement. However, Quartus Prime software synthesis supports using only the `force` statement to override the drivers of a net (gate outputs, module outputs, and continuous assignments) and previous assignments made on a particular net or a net bus.

Note: Synthesis supports using a `force` statement only inside an initial block.

Examples of `force` Statements in Synthesis

The following are some examples of `force` statements that the Quartus Prime software synthesis supports:

Example 56. Using a `force` Statement to Set Counter `enable` to 0

The following is an example of how you can use a `force` statement to tie the `en` port of the `counter` instance `u1` to logic `0`:

```
module top(clk, rst, enable, dout);
  input clk, rst, enable;
  output [3:0] dout;
  counter u1(.clk(clk), .reset(rst), .en(enable), .q(dout));

  initial  begin
    force u1.en = 1'b0
  end
endmodule
```

You can observe that the `force` statement overrides the other driver of the `en` port, which is the `enable` port of the `top` module.

Example 57. Using a `force` Statement to Change Connections

The following example shows how you can use a `force` statement to change the connections in the design:

```
module top(input [3:0] din, din1, output logic [3:0] dout, dout1, input clk,
rst);
  dff i0(.din(din), .dout(dout), .clk(clk), .rst(rst) );
  dff i1(.din(din1), .dout(dout1), .clk(clk), .rst(rst) );
endmodule

module top_modified(input [3:0] din, din1, output logic [3:0] dout, dout1, input
clk, rst);
  top i_top(.*);
  initial
  begin
    force i_top.i1.din = i_top.din;
  end
endmodule
```

In this example, the design's `top` module instantiates two instances of the `dff` module. `din` and `din1` ports of the `top` module drive the `din` port of `i0` and `i1` instances.

Suppose you want to change the connections in the `top` module without changing the RTL inside the `top` module. In this situation, you can use a `force` statement within a wrapper module (`top_modified`), which becomes the new `top` module. In the new `top` module, use the `force` statement to modify the connections in the `top` module such that the `din` port of both `i0` and `i1` instances is driven by the same `din` port of the `top` module. The `force` statement uses a cross-module reference (XMR) to access signals in a hierarchy below it. For more information about XMR, refer to Cross-Module Referencing (XMR) in HDL Code on page 65.

Note: For this example, instead of creating a wrapper `top_modified` that instantiates the `top` module, you can also create a secondary top-level entity and make the `force` assignment in it, as shown in the following:

```
module secondary_top(input [3:0] din, din1, output logic [3:0] dout, dout1,
input clk, rst);
initial begin
  force top.i1.din = top.din;
end
endmodule
```

2.10. Recommended HDL Coding Styles Revision History

The following revisions history applies to this chapter:

Document Version	Quartus Prime Version	Changes
2025.04.14	25.1	• Applied Altera rebranding throughout as necessary. • Updated throughout for Agilex 3 support. • Updated *Inferring RAM with Byte-Enable Signals* topic for support of all legal bytewidths. • Added *Inferring Simple Quad-Port RAM* topic for new support.
2024.09.30	24.3	• Added synchronous and asynchronous reset examples to *Specifying Initial Memory Contents at Power-Up* topic.
2024.07.08	24.2	• Added *State Machine Processing*
2023.10.02	23.1	• Made minor updates to code snippets in *Using force Statements in HDL Code* and *Cross-Module Referencing (XMR) in HDL Code*.
2023.04.03	23.1	• Updated product family name to "Intel Agilex 7."
2022.09.26	22.3	• Added *Using force Statements in HDL Code*. • Added *Cross-Module Referencing (XMR) in HDL Code*.
2021.10.04	21.3	• Added new *Inferring FIFOs in HDL Code* topic and linked to *FIFO Intel FPGA IP User Guide*. • Added new *Dual Clock FIFO Example in Verilog HDL* topic. • Added new *Dual Clock FIFO Timing Constraints* topic.
2021.06.21	21.2	• Updated *Inferring Shift Registers in HDL Code* for Stratix 10 and Intel® Agilex 7 devices. • Updated wording of *Controlling RAM Inference and Implementation* for clarity.
2019.09.30	19.3	• Updated Simple Dual-Port Synchronous RAM with Byte Enable examples. • Updated True Dual-Port Synchronous RAM examples. • Updated Verilog HDL Single-Bit Wide Shift Register example from 64 to 69 bits. • Updated VHDL Single-Bit Wide Shift Register example from 67 to 69 bits. • Updated Verilog HDL 8-Bit Wide Shift Register with Evenly Spaced Taps from 64 to 254 bits.
2018.09.24	18.1	• Added "State Machine Power-Up" topic. • Updated "Designing with Low-Level Primitives" to remove support for carry and cascade chains using `CARRY`, `CARRY_SUM`, and `CASCADE` primitives. • Renamed topic: "Use the Device Synchronous Load (sload) Signal to Initialize" to "Initialize the Device with the Synchronous Load (sload) Signal"

continued...

Document Version	Quartus Prime Version	Changes
2017.11.06	17.1	• Described new no_ram synthesis attribute.
2017.05.08	17.0	• Updated example: Verilog HDL Multiply-Accumulator • Updated information about use of safe state machine. • Revised Check Read-During-Write Behavior. • Revised Controlling RAM Inference and Implementation. • Revised Single-Clock Synchronous RAM with Old Data Read-During-Write Behavior. • Revised Single-Clock Synchronous RAM with New Data Read-During-Write Behavior. • Updated and moved template for VHDL Single-Clock Simple Dual Port Synchronous RAM with New Data Read-During-Write Behavior. • Revised Inferring ROM Functions from HDL Code. • Removed example: VHDL 8-Bit Wide, 64-Bit Long Shift Register with Evenly Spaced Taps. • Removed example: Verilog HDL D-Type Flipflop (Register) With ena, aclr, and aload Control Signals • Removed example: VHDL D-Type Flipflop (Register) With ena, aclr, and aload Control Signals • Added example: Verilog D-type Flipflop bus with Secondary Signals • Removed references to 4-input LUT-based devices. • Removed references to Integrated Synthesis. • Created example: Avoid this VHDL Coding Style.
2016.10.31	16.1	• Provided corrected Verilog HDL Pipelined Binary Tree and Ternary Tree examples. • Implemented Intel rebranding.
2016.05.03	16.0	• Added information about use of safe state machine. • Updated example code templates with latest coding styles.
2015.11.02	15.1	• Changed instances of *Quartus II* to *Quartus Prime*.
2015.05.04	15.0	Added information and reference about ramstyle attribute for sift register inference.
2014.12.15	14.1	Updated location of Fitter Settings, Analysis & Synthesis Settings, and Physical Optimization Settings to Compiler Settings.
2014.08.18	14.0.a10	• Added recommendation to use register pipelining to obtain high performance in DSP designs.
2014.06.30	14.0	Removed obsolete MegaWizard Plug-In Manager support.
November 2013	13.1	Removed HardCopy device support.
June 2012	12.0	• Revised section on inserting Altera templates. • Code update for Example 11-51. • Minor corrections and updates.
November 2011	11.1	• Updated document template. • Minor updates and corrections.
December 2010	10.1	• Changed to new document template. • Updated Unintentional Latch Generation content. • Code update for Example 11-18.
July 2010	10.0	• Added support for mixed-width RAM • Updated support for no_rw_check for inferring RAM blocks • Added support for byte-enable

continued...

Document Version	Quartus Prime Version	Changes
November 2009	9.1	• Updated support for Controlling Inference and Implementation in Device RAM Blocks • Updated support for Shift Registers
March 2009	9.0	• Corrected and updated several examples • Added support for Arria II GX devices • Other minor changes to chapter
November 2008	8.1	Changed to 8-1/2 x 11 page size. No change to content.
May 2008	8.0	Updates for the Quartus Prime software version 8.0 release, including: • Added information to "RAM • Functions—Inferring ALTSYNCRAM and ALTDPRAM Megafunctions from HDL Code" on page 6–13 • Added information to "Avoid Unsupported Reset and Control Conditions" on page 6–14 • Added information to "Check Read-During-Write Behavior" on page 6–16 • Added two new examples to "ROM Functions—Inferring ALTSYNCRAM and LPM_ROM Megafunctions from HDL Code" on page 6–28: Example 6–24 and Example 6–25 • Added new section: "Clock Multiplexing" on page 6–46 • Added hyperlinks to references within the chapter • Minor editorial updates

3. Recommended Design Practices

This chapter provides design recommendations for Altera FPGA devices.

Current FPGA applications have reached the complexity and performance requirements of ASICs. In the development of complex system designs, design practices have an enormous impact on the timing performance, logic utilization, and system reliability of a device. Well-coded designs behave in a predictable and reliable manner even when retargeted to different families or speed grades. Good design practices also aid in successful design migration between FPGA and ASIC implementations for prototyping and production.

For optimal performance, reliability, and faster time-to-market when designing with Altera FPGA devices, you should adhere to the following guidelines:

- Understand the impact of synchronous design practices
- Follow recommended design techniques, including hierarchical design partitioning, and timing closure guidelines
- Take advantage of the architectural features in the targeted device

3.1. Following Synchronous FPGA Design Practices

The first step in good design methodology is to understand the implications of your design practices and techniques. This section outlines the benefits of optimal synchronous design practices and the hazards involved in other approaches.

Good synchronous design practices can help you meet your design goals consistently. Problems with other design techniques can include reliance on propagation delays in a device, which can lead to race conditions, incomplete timing analysis, and possible glitches.

In a synchronous design, a clock signal triggers every event. If you ensure that all the timing requirements of the registers are met, a synchronous design behaves in a predictable and reliable manner for all process, voltage, and temperature (PVT) conditions. You can easily migrate synchronous designs to different device families or speed grades.

3.1.1. Implementing Synchronous Designs

In a synchronous design, the clock signal controls the activities of all inputs and outputs.

On every active edge of the clock (usually the rising edge), the data inputs of registers are sampled and transferred to outputs. Following an active clock edge, the outputs of combinational logic feeding the data inputs of registers change values. This change triggers a period of instability due to propagation delays through the logic as the

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

signals go through several transitions and finally settle to new values. Changes that occur on data inputs of registers do not affect the values of their outputs until after the next active clock edge.

Because the internal circuitry of registers isolates data outputs from inputs, instability in the combinational logic does not affect the operation of the design if you meet the following timing requirements:

- Before an active clock edge, you must ensure that the data input has been stable for at least the setup time of the register.
- After an active clock edge, you must ensure that the data input remains stable for at least the hold time of the register.

  When you specify all your clock frequencies and other timing requirements, the Quartus Prime Timing Analyzer reports actual hardware requirements for the setup times (t_{SU}) and hold times (t_H) for every pin in your design. By meeting these external pin requirements and following synchronous design techniques, you ensure that you satisfy the setup and hold times for all registers in your device.

  Tip: To meet setup and hold time requirements on all input pins, any inputs to combinational logic that feed a register should have a synchronous relationship with the clock of the register. If signals are asynchronous, you can register the signals at the inputs of the device to help prevent a violation of the required setup and hold times.

  When you violate the setup or hold time of a register, you might oscillate the output, or set the output to an intermediate voltage level between the high and low levels called a metastable state. In this unstable state, small perturbations such as noise in power rails can cause the register to assume either the high or low voltage level, resulting in an unpredictable valid state. Various undesirable effects can occur, including increased propagation delays and incorrect output states. In some cases, the output can even oscillate between the two valid states for a relatively long period of time.

3.1.2. Asynchronous Design Hazards

Asynchronous design techniques, such as ripple counters or pulse generators, can work as "short cuts" to save device resources. However, asynchronous techniques have inherent problems. For example, relying on propagation delays can result in incomplete timing constraints and possible glitches and spikes, because propagation delay varies with temperature and voltage fluctuations.

Asynchronous design structures that depend on the relative propagation delays can present race conditions. Race conditions arise when the order of signal changes affect the output of the logic. The same logic design can have varying timing delays with each compilation, depending on placement and routing. The number of possible variations make it impossible to determine the timing delay associated with a particular block of logic. As devices become faster due to process improvements, delays in asynchronous designs may decrease, resulting in designs that do not function as expected. Relying on a particular delay also makes asynchronous designs difficult to migrate to other architectures, devices, or speed grades.

The timing of asynchronous design structures is often difficult or impossible to model with timing assignments and constraints. If you do not have complete or accurate timing constraints, the timing-driven algorithms that synthesis and place-and-route tools use may not be able to perform the best optimizations, and the reported results may be incomplete.

Additionally, asynchronous design structures can generate glitches, which are pulses that are very short compared to clock periods. Combinational logic is the main cause of glitches. When the inputs to the combinational logic change, the outputs exhibit several glitches before settling to their new values. Glitches can propagate through combinational logic, leading to incorrect values on the outputs in asynchronous designs. In synchronous designs, glitches on register's data inputs have no negative consequences, because data processing waits until the next clock edge.

3.2. HDL Design Guidelines

When designing with HDL code, consider how synthesis tools interpret different HDL design techniques and what results to expect.

Design style can affect logic utilization and timing performance, as well as the design's reliability. This section describes basic design techniques that ensure optimal synthesis results for designs that target Altera FPGA devices while avoiding common causes of unreliability and instability. As a best practice, consider potential problems when designing combinational logic, and pay attention to clocking schemes so that the design maintains synchronous functionality and avoids timing issues.

3.2.1. Considerations for the Hyperflex FPGA Architecture

The Hyperflex FPGA architecture and the Hyper-Retimer require a review of the best design practices to achieve the highest clock rates possible.

While most common techniques of high-speed design apply to designing for the Hyperflex architecture, you must use some new approaches to achieve the highest performance. Follow these general RTL design guidelines to enable the Hyper-Retimer to optimize design performance:

- Design in a way that facilitates register retiming by the Hyper-Retimer.
- Use a latency-insensitive design that supports the addition of pipeline stages at clock domain boundaries, top-level I/Os, and at the boundaries of functional blocks.
- Restructure RTL to avoid performance-limiting loops.

For more information about best design practices targeting Stratix 10 devices, refer to the *Stratix 10 High-Performance Design Handbook*.

Related Information

Hyperflex™ Architecture High-Performance Design Handbook

3.2.2. Optimizing Combinational Logic

Combinational logic structures consist of logic functions that depend only on the current state of the inputs. In Altera FPGAs, these functions are implemented in the look-up tables (LUTs) with either logic elements (LEs) or adaptive logic modules (ALMs).

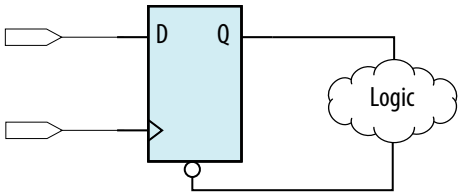
For cases where combinational logic feeds registers, the register control signals can implement part of the logic function to save LUT resources. By following the recommendations in this section, you can improve the reliability of your combinational design.

3.2.2.1. Avoid Combinational Loops

Combinational loops are among the most common causes of instability and unreliability in digital designs. Combinational loops generally violate synchronous design principles by establishing a direct feedback loop that contains no registers.

Avoid combinational loops whenever possible. In a synchronous design, feedback loops should include registers. For example, a combinational loop occurs when the left-hand side of an arithmetic expression also appears on the right-hand side in HDL code. A combinational loop also occurs when you feed back the output of a register to an asynchronous pin of the same register through combinational logic.

Figure 8. Combinational Loop Through Asynchronous Control Pin



Tip: Use recovery and removal analysis to perform timing analysis on asynchronous ports, such as `clear` or `reset` in the Quartus Prime software.

Combinational loops are inherently high-risk design structures for the following reasons:

- Combinational loop behavior generally depends on relative propagation delays through the logic involved in the loop. As discussed, propagation delays can change, which means the behavior of the loop is unpredictable.
- In many design tools, combinational loops can cause endless computation loops . Most tools break open combinational loops to process the design. The various tools used in the design flow may open a given loop differently, and process it in a way inconsistent with the original design intent.

3.2.2.2. Avoid Unintended Latch Inference

Avoid using latches to ensure that you can completely analyze the timing performance and reliability of your design. A latch is a small circuit with combinational feedback that holds a value until a new value is assigned. You can implement latches with the Quartus Prime Text Editor or Block Editor.

A common mistake in HDL code is unintended latch inference; Quartus Prime Synthesis issues a warning message if this occurs. Unlike other technologies, a latch in FPGA architecture is not significantly smaller than a register. However, the architecture is not optimized for latch implementation and latches generally have slower timing performance compared to equivalent registered circuitry.

Latches have a transparent mode in which data flows continuously from input to output. A positive latch is in transparent mode when the enable signal is high (low for a negative latch). In transparent mode, glitches on the input can pass through to the output because of the direct path created. This presents significant complexity for timing analysis. Typical latch schemes use multiple enable phases to prevent long transparent paths from occurring. However, timing analysis cannot identify these safe applications.

The Timing Analyzer analyzes latches as synchronous elements clocked on the falling edge of the positive latch signal by default. It allows you to treat latches as having nontransparent start and end points. Be aware that even an instantaneous transition through transparent mode can lead to glitch propagation. The Timing Analyzer cannot perform cycle-borrowing analysis.

Due to various timing complexities, latches have limited support in formal verification tools. Therefore, you should not rely on formal verification for a design that includes latches.

Related Information

Avoid Unintentional Latch Generation on page 44

3.2.2.3. Avoid Delay Chains in Clock Paths

Delays in PLD designs can change with each placement and routing cycle. Effects such as rise and fall time differences and on-chip variation mean that delay chains, especially those placed on clock paths, can cause significant problems in your design. Avoid using delay chains to prevent these kinds of problems.

You require delay chains when you use two or more consecutive nodes with a single fan-in and a single fan-out to cause delay. Inverters are often chained together to add delay. Delay chains are sometimes used to resolve race conditions created by other asynchronous design practices.

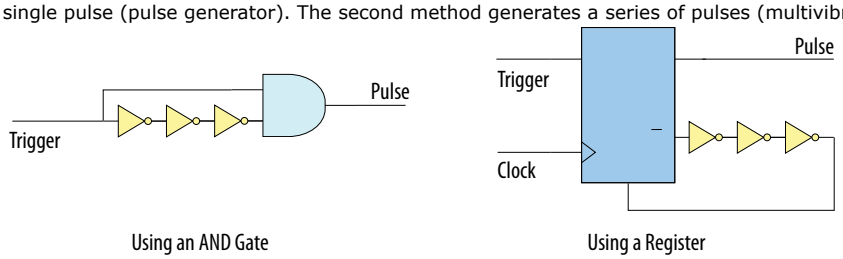
In some ASIC designs, delays are used for buffering signals as they are routed around the device. This functionality is not required in FPGA devices because the routing structure provides buffers throughout the device.

3.2.2.4. Use Synchronous Pulse Generators

Use synchronous techniques to design pulse generators.

Figure 9. Asynchronous Pulse Generators

The figure shows two methods for asynchronous pulse generation. The first method uses a delay chain to generate a single pulse (pulse generator). The second method generates a series of pulses (multivibrators).



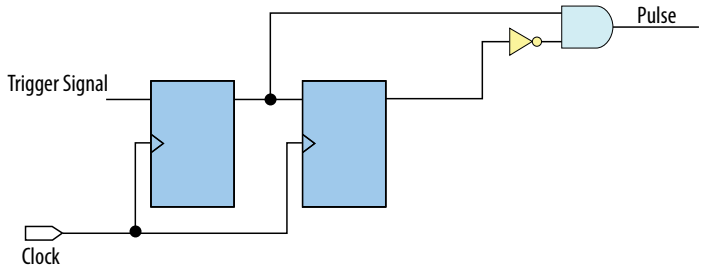
In the first method, a trigger signal feeds both inputs of a 2-input AND gate, and the design adds inverters to one of the inputs to create a delay chain. The width of the pulse depends on the time differences between the path that feeds the gate directly and the path that goes through the delay chain. This is the same mechanism responsible for the generation of glitches in combinational logic following a change of input values. This technique artificially increases the width of the glitch.

In the second method, a register's output drives its asynchronous reset signal through a delay chain. The register resets itself asynchronously after a certain delay. The Compiler can determine the pulse width only after placement and routing, when

routing and propagation delays are known. You cannot reliably create a specific pulse width when creating HDL code, and it cannot be set by EDA tools. The pulse may not be wide enough for the application under all PVT conditions. Also, the pulse width changes if you change to a different device. Additionally, verification is difficult because static timing analysis cannot verify the pulse width.

Multivibrators use a glitch generator to create pulses, together with a combinational loop that turns the circuit into an oscillator. This method creates additional problems because of the number of pulses involved. Additionally, when the structures generate multiple pulses, they also create a new artificial clock in the design that must be analyzed by design tools.

Figure 10. Recommended Synchronous Pulse-Generation Technique



The pulse width is always equal to the clock period. This pulse generator is predictable, can be verified with timing analysis, and is easily moved to other architectures, devices, or speed grades.

3.2.3. Optimizing Clocking Schemes

Like combinational logic, clocking schemes have a large effect on the performance and reliability of a design.

Altera recommends avoiding the use of internally generated clocks (other than PLLs) wherever possible because they can cause functional and timing problems in the design. If not carefully designed, clocks generated with or passing through combinational logic can introduce glitches that create functional problems, and the delay inherent in combinational logic can lead to timing problems. Refer to the sections listed below for information on some common scenarios of using combinational logic in a clock path (for example, clock mux) and design considerations to prevent unexpected failures.

Tip: Specify all clock relationships in the Quartus Prime software to allow for the best timing-driven optimizations during fitting and to allow correct timing analysis. Use clock setting assignments on any derived or internal clocks to specify their relationship to the base clock.

Use global device-wide, low-skew dedicated routing for all internally-generated clocks, instead of routing clocks on regular routing lines.

Avoid data transfers between different clocks wherever possible. If you require a data transfer between different clocks, use FIFO circuitry. You can use the clock uncertainty features in the Quartus Prime software to compensate for the variable delays between clock domains. Consider setting a clock setup uncertainty and clock hold uncertainty value of 10% to 15% of the clock delay.

The following sections provide specific examples and recommendations for avoiding clocking scheme problems:

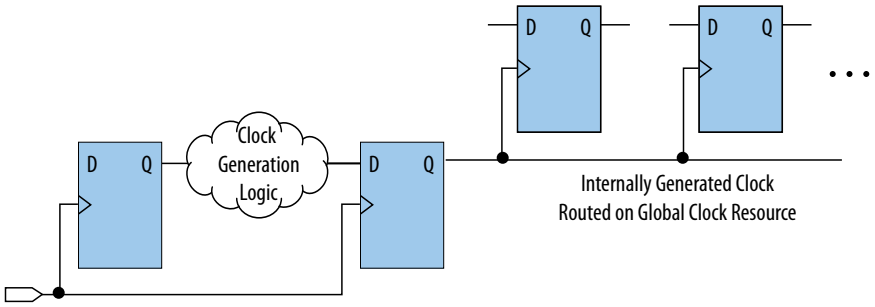
3.2.3.1. Register Combinational Logic Outputs

If you use the output from combinational logic as a clock signal or as an asynchronous reset signal, you can expect to see glitches in your design. In a synchronous design, glitches on data inputs of registers are normal events that have no consequences. However, a glitch or a spike on the clock input (or an asynchronous input) to a register can have significant consequences.

Narrow glitches can violate the register's minimum pulse width requirements. Setup and hold requirements might also be violated if the data input of the register changes when a glitch reaches the clock input. Even if the design does not violate timing requirements, the register output can change value unexpectedly and cause functional hazards elsewhere in the design.

To avoid these problems, you should always register the output of combinational logic before you use it as a clock signal.

Figure 11. Recommended Clock-Generation Technique



Registering the output of combinational logic ensures that glitches generated by the combinational logic are blocked at the data input of the register.

3.2.3.2. Avoid Asynchronous Clock Division

Designs often require clocks that you create by dividing a master clock. Most Altera FPGAs provide dedicated phase-locked loop (PLL) circuitry for clock division. Using dedicated PLL circuitry can help you avoid many of the problems that can be introduced by asynchronous clock division logic.

When you must use logic to divide a master clock, always use synchronous counters or state machines. Additionally, create your design so that registers always directly generate divided clock signals, and route the clock on global clock resources. To avoid glitches, do not decode the outputs of a counter or a state machine to generate clock signals.

3.2.3.3. Avoid Ripple Counters

To simplify verification, avoid ripple counters in your design. In the past, FPGA designers implemented ripple counters to divide clocks by a power of two because the counters are easy to design and may use fewer gates than their synchronous counterparts.

Ripple counters use cascaded registers, in which the output pin of one register feeds the clock pin of the register in the next stage. This cascading can cause problems because the counter creates a ripple clock at each stage. These ripple clocks must be handled properly during timing analysis, which can be difficult and may require you to make complicated timing assignments in your synthesis and placement and routing tools.

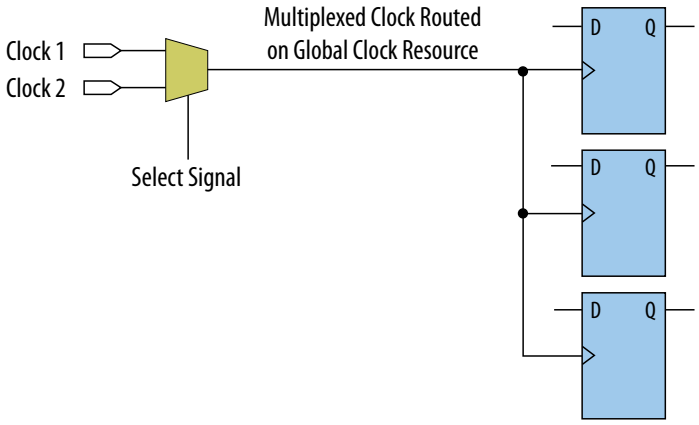
You can often use ripple clock structures to make ripple counters out of the smallest amount of logic possible. However, in all Altera devices supported by the Quartus Prime software, using a ripple clock structure to reduce the amount of logic used for a counter is unnecessary because the device allows you to construct a counter using one logic element per counter bit. You should avoid using ripple counters completely.

3.2.3.4. Use Multiplexed Clocks

Use clock multiplexing to operate the same logic function with different clock sources. In these designs, multiplexing selects a clock source.

For example, telecommunications applications that deal with multiple frequency standards often use multiplexed clocks.

Figure 12. Multiplexing Logic and Clock Sources



Adding multiplexing logic to the clock signal can create the problems addressed in the previous sections, but requirements for multiplexed clocks vary widely, depending on the application. Clock multiplexing is acceptable when the clock signal uses global clock routing resources and if the following criteria are met:

- The clock multiplexing logic does not change after initial configuration
- The design uses multiplexing logic to select a clock for testing purposes
- Registers are always reset when the clock switches
- A temporarily incorrect response following clock switching has no negative consequences

If the design switches clocks in real time with no reset signal, and your design cannot tolerate a temporarily incorrect response, you must use a synchronous design so that there are no timing violations on the registers, no glitches on clock signals, and no race conditions or other logical problems. By default, the Quartus Prime software optimizes and analyzes all possible paths through the multiplexer and between both

internal clocks that may come from the multiplexer. This may lead to more restrictive analysis than required if the multiplexer is always selecting one particular clock. If you do not require the more complete analysis, you can assign the output of the multiplexer as a base clock in the Quartus Prime software, so that all register-to-register paths are analyzed using that clock.

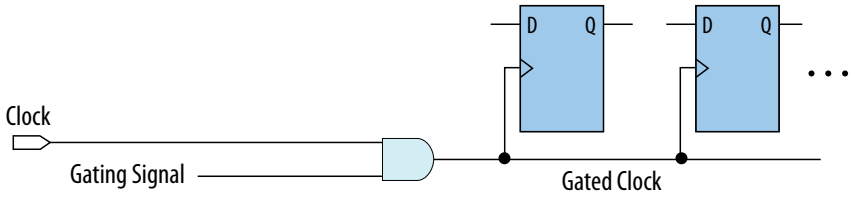
Tip: Use dedicated hardware to perform clock multiplexing when it is available, instead of using multiplexing logic. For example, you can use the clock-switchover feature or clock control block available in certain Altera FPGA devices. These dedicated hardware blocks ensure that you use global low-skew routing lines and avoid any possible hold time problems on the device due to logic delay on the clock line.

Note: For device-specific information about clocking structures, refer to the appropriate device data sheet or handbook.

3.2.3.5. Use Gated Clocks

Gated clocks turn a clock signal on and off using an enable signal that controls gating circuitry. When a clock is turned off, the corresponding clock domain is shut down and becomes functionally inactive.

Figure 13. Gated Clock



You can use gated clocks to reduce power consumption in some device architectures by effectively shutting down portions of a digital circuit when they are not in use. When a clock is gated, both the clock network and the registers driven by it stop toggling, thereby eliminating their contributions to power consumption. However, gated clocks are not part of a synchronous scheme and therefore can significantly increase the effort required for design implementation and verification. Gated clocks contribute to clock skew and make device migration difficult. These clocks are also sensitive to glitches, which can cause design failure.

Use dedicated hardware to perform clock gating rather than an AND or OR gate. For example, you can use the clock control block in newer Altera FPGA devices to shut down an entire clock network. Dedicated hardware blocks ensure that you use global routing with low skew, and avoid any possible hold time problems on the device due to logic delay on the clock line.

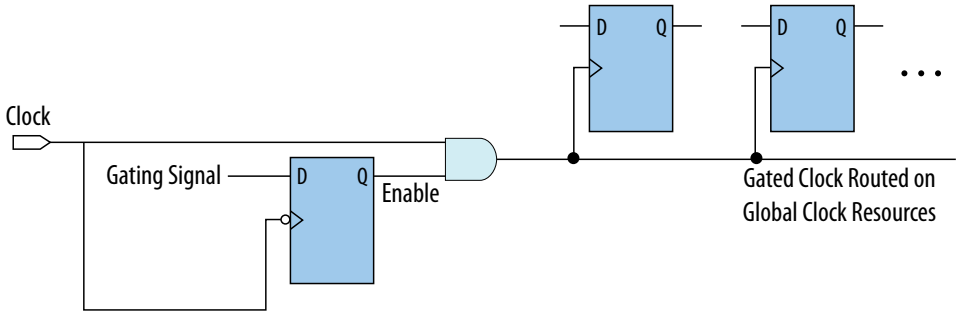
From a functional point of view, you can shut down a clock domain in a purely synchronous manner using a synchronous clock enable signal. However, when using a synchronous clock enable scheme, the clock network continues toggling. This practice does not reduce power consumption as much as gating the clock at the source does. In most cases, use a synchronous scheme.

3.2.3.5.1. Recommended Clock-Gating Methods

Use gated clocks only when your target application requires power reduction and gated clocks provide the required reduction in your device architecture. If you must use clocks gated by logic, follow a robust clock-gating methodology and ensure the gated clock signal uses dedicated global clock routing.

You can gate a clock signal at the source of the clock network, at each register, or somewhere in between. Since the clock network contributes to switching power consumption, gate the clock at the source whenever possible to shut down the entire clock network instead of further along.

Figure 14. Recommended Clock-Gating Technique for Clock Active on Rising Edge



To generate a gated clock with the recommended technique, use a register that triggers on the inactive edge of the clock. With this configuration, only one input of the gate changes at a time, preventing glitches or spikes on the output. If the clock is active on the rising edge, use an AND gate. Conversely, for a clock that is active on the falling edge, use an OR gate to gate the clock and register

Pay attention to the delay through the logic generating the enable signal, because the enable command must be ready in less than one-half the clock cycle. This might cause problems if the logic that generates the enable command is particularly complex, or if the duty cycle of the clock is severely unbalanced. However, careful management of the duty cycle and logic delay may be an acceptable solution when compared with problems created by other methods of gating clocks.

In the Timing Analyzer, ensure to apply a clock setting to the output of the AND gate. Otherwise, the timing analyzer might analyze the circuit using the clock path through the register as the longest clock path and the path that skips the register as the shortest clock path, resulting in artificial clock skew.

In certain cases, converting the gated clocks to clock enable pins may help reduce glitch and clock skew, and eventually produce a more accurate timing analysis. You can set the Quartus Prime software to automatically convert gated clocks to clock enable pins by turning on the **Auto Gated Clock Conversion** option. The conversion applies to two types of gated clocking schemes: single-gated clock and cascaded-gated clock.

Related Information

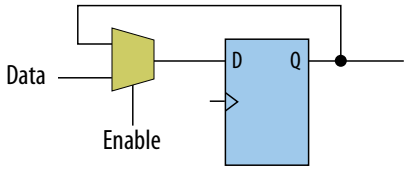
Auto Gated Clock Conversion logic option
In *Quartus Prime Help*

3.2.3.6. Use Synchronous Clock Enables

To turn off a clock domain in a synchronous manner, use a synchronous clock enable signal. FPGAs efficiently support clock enable signals because there is a dedicated clock enable signal available on all device registers.

This scheme does not reduce power consumption as much as gating the clock at the source because the clock network keeps toggling, and performs the same function as a gated clock by disabling a set of registers. Insert a multiplexer in front of the data input of every register to either load new data, or copy the output of the register.

Figure 15. Synchronous Clock Enable



When designing for Stratix 10 devices, consider that high fan-out clock enable signals can limit the performance achievable by the Hyper- Retimer. For specific recommendations, refer to the *Stratix 10 High-Performance Design Handbook*.

Related Information

Hyperflex™ Architecture High-Performance Design Handbook

3.2.4. Optimizing Physical Implementation and Timing Closure

This section provides design and timing closure techniques for high speed or complex core logic designs with challenging timing requirements. These techniques may also be helpful for low or medium speed designs.

3.2.4.1. Planning Physical Implementation

When planning a design, consider the following elements of physical implementation:

- The number of unique clock domains and their relationships
- The amount of logic in each functional block
- The location and direction of data flow between blocks
- How data routes to the functional blocks between I/O interfaces

Interface-wide control or status signals may have competing or opposing constraints. For example, when a functional block's control or status signals interface with physical channels from both sides of the device. In such cases you must provide enough pipeline register stages to allow these signals to traverse the width of the device. In addition, you can structure the hierarchy of the design into separate logic modules for each side of the device. The side modules can generate and use registered control signals per side. This simplifies floorplanning, particularly in designs with transceivers, by placing per-side logic near the transceivers.

When adding register stages to pipeline control signals, turn off **Auto Shift Register Replacement** in the **Assignment Editor** (**Assignments ➤ Assignment Editor**) for each register as needed. By default, chains of registers can be converted to a RAM-

based implementation based on performance and resource estimates. Since pipelining helps meet timing requirements over long distance, this assignment ensures that control signals are not converted.

3.2.4.2. Planning FPGA Resources

Your design requirements impact the use of FPGA resources. Plan functional blocks with appropriate global, regional, and dual-regional network signals in mind.

In general, after allocating the clocks in a design, use global networks for the highest fan-out control signals. When a global network signal distributes a high fan-out control signal, the global signal can drive logic anywhere in the device. Similarly, when using a regional network signal, the driven logic must be in one quadrant of the device, or half the device for a dual-regional network signal. Depending on data flow and physical locations of the data entry and exit between the I/Os and the device, restricting a functional block to a quadrant or half the device may not be practical for performance or resource requirements.

When floorplanning a design, consider the balance of different types of device resources, such as memory, logic, and DSP blocks in the main functional blocks. For example, if a design is memory intensive with a small amount of logic, it may be difficult to develop an effective floorplan. Logic that interfaces with the memory would have to spread across the chip to access the memory. In this case, it is important to use enough register stages in the data and control paths to allow signals to traverse the chip to access the physically disparate resources needed.

3.2.4.3. Optimizing for Timing Closure

To achieve timing closure for your design, you can enable compilation settings in the Quartus Prime software, or you can directly modify your timing constraints.

Compilation Settings for Timing Closure

Note: Changes in project settings can significantly increase compilation time. You can view the performance gain versus runtime cost by reviewing the Fitter messages after design processing.

Table 3. Compilation Settings that Impact Timing Closure

Setting	Location	Effect on Timing Closure
Allow Register Duplication	**Assignments ➤ Settings ➤ Compiler Settings ➤ Advanced Settings (Fitter)**	This technique is most useful where registers have high fan-out, or where the fan-out is in physically distant areas of the device. Review the netlist optimizations report and consider manually duplicating registers automatically added by physical synthesis. You can also locate the original and duplicate registers in the Chip Planner. Compare their locations, and if the fan-out is improved, modify the code and turn off register duplication to save compile time.
Prevent Register Retiming	**Assignments ➤ Settings ➤ Compiler Settings ➤ Advanced Settings (Fitter)**	Useful if some combinatorial paths between registers exceed the timing goal while other paths fall short. If a design is already heavily pipelined, register retiming is less likely to provide significant performance gains, since there should not be significantly unbalanced levels of logic across pipeline stages.

Guidelines for Optimizing Timing Closure using Timing Constraints

Appropriate timing constraints are essential to achieving timing closure. Use the following general guidelines in applying timing constraints:

- Apply multicycle constraints in your design wherever single-cycle timing analysis is not necessary.
- Apply False Path constraints to all asynchronous clock domain crossings or resets in the design. This technique prevents overconstraining and the Fitter focuses only on critical paths to reduce compile time. However, overconstraining timing critical clock domains can sometimes provide better timing results and lower compile times than physical synthesis.
- Overconstrain rather than using physical synthesis when the slack improvement from physical synthesis is near zero. Overconstrain the frequency requirement on timing critical clock domains by using setup uncertainty.
- When evaluating the effect of constraint changes on performance and runtime, compile the design with at least three different seeds to determine the average performance and runtime effects. Different constraint combinations produce various results. Three samples or more establish a performance trend. Modify your constraints based on performance improvement or decline.
- Leave settings at the default value whenever possible. Increasing performance constraints can increase the compile time significantly. While those increases may be necessary to close timing on a design, using the default settings whenever possible minimizes compile time.

Related Information

Quartus Prime Pro Edition User Guide: Design Optimization

3.2.4.4. Optimizing Critical Timing Paths

To close timing in high speed designs, review paths with the largest timing failures. Correcting a single, large timing failure can result in a very significant timing improvement.

Review the register placement and routing paths by clicking **Tools ➤ Chip Planner**. Large timing failures on high fan-out control signals can be caused by any of the following conditions:

- Sub-optimal use of global networks
- Signals that traverse the chip on local routing without pipelining
- Failure to correct high fan-out by register duplication

For high-speed and high-bandwidth designs, optimize speed by reducing bus width and wire usage. To reduce wire usage, move the data as little as possible. For example, if a block of logic functions on a few bits of a word, store inactive bits in a FIFO or memory. Memory is cheaper and denser than registers, and reduces wire usage.

Related Information

Quartus Prime Pro Edition User Guide: Design Optimization

3.2.5. Optimizing Power Consumption

The total FPGA power consumption is comprised of I/O power, core static power, and core dynamic power. Knowledge of the relationship between these components is fundamental in calculating the overall total power consumption.

You can use various optimization techniques and tools to minimize power consumption when applied during FPGA design implementation. The Quartus Prime software offers power-driven compilation features to fully optimize device power consumption. Power-driven compilation focuses on reducing your design's total power consumption using power-driven synthesis and power-driven placement and routing.

Related Information

Quartus Prime Pro Edition User Guide: Power Analysis and Optimization
In *Quartus Prime Pro Edition User Guide: Power Analysis and Optimization*

3.2.6. Managing Design Metastability

In FPGA designs, synchronization of asynchronous signals can cause metastability. You can use the Quartus Prime software to analyze the mean time between failures (MTBF) due to metastability. A high metastability MTBF indicates a more robust design.

Related Information

- Managing Metastability with the Quartus Prime Software on page 125
- Quartus Prime Pro Edition User Guide: Design Optimization

3.3. Use Clock and Register-Control Architectural Features

In addition to following general design guidelines, you must code your design with the device architecture in mind. FPGAs provide device-wide clocks and register control signals that can improve performance.

3.3.1. Use Global Reset Resources

ASIC designs may use local resets to avoid long routing delays. Take advantage of the device-wide asynchronous reset pin available on most FPGAs to eliminate these problems. This reset signal provides low-skew routing across the device.

The following are three types of resets used in synchronous circuits:

- Synchronous Reset
- Asynchronous Reset
- Synchronized Asynchronous Reset—preferred when designing an FPGA circuit

3.3.1.1. Use Synchronous Resets

The synchronous reset ensures that the circuit is fully synchronous. You can easily time the circuit with the Quartus Prime Timing Analyzer.

Because clocks that are synchronous to each other launch and latch the reset signal, the data arrival and data required times are easily determined for proper slack analysis. The synchronous reset is easier to use with cycle-based simulators.

There are two methods by which a reset signal can reach a register; either by being gated in with the data input, or by using an LAB-wide control signal (`synclr`). If you use the first method, you risk adding an additional gate delay to the circuit to accommodate the reset signal, which causes increased data arrival times and negatively impacts setup slack. The second method relies on dedicated routing in the LAB to each register, but this is slower than an asynchronous reset to the same register.

Figure 16. Synchronous Reset

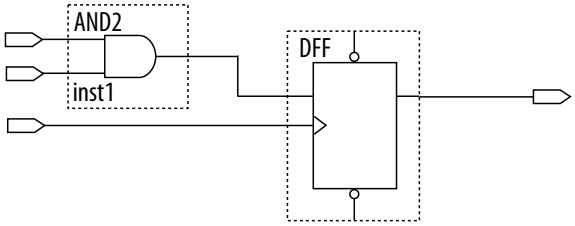
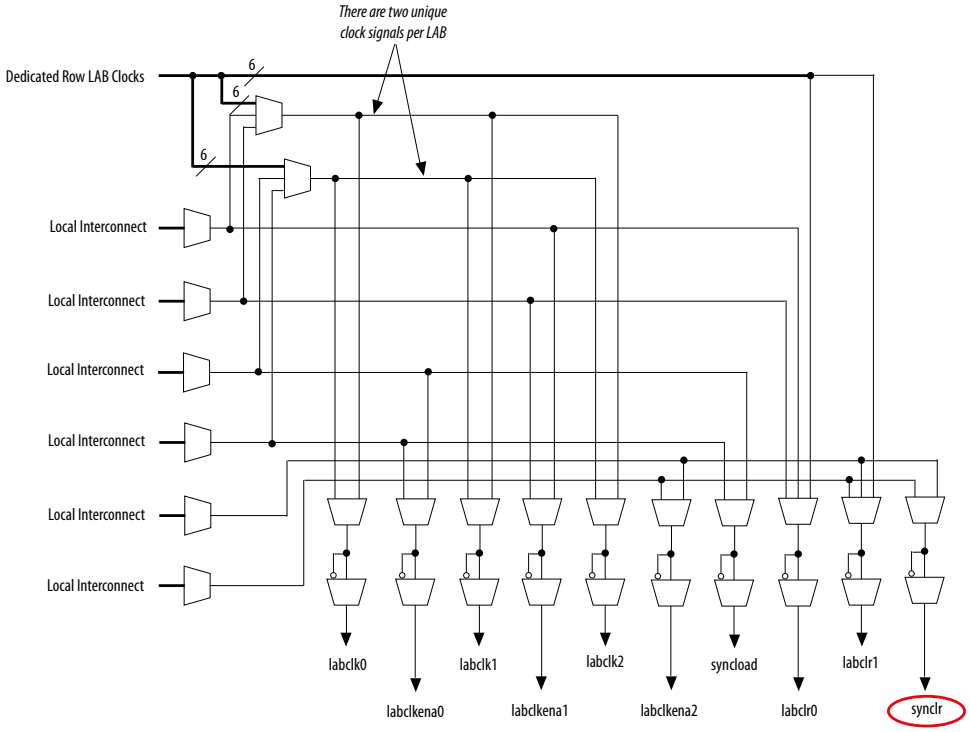
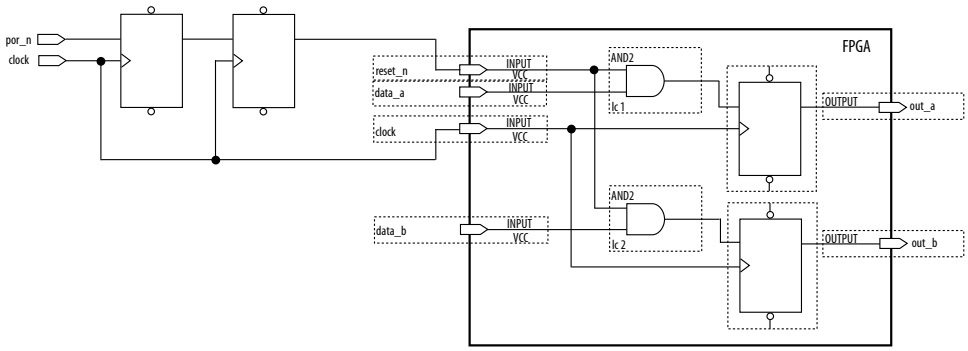


Figure 17. LAB-Wide Control Signals



Consider two types of synchronous resets when you examine the timing analysis of synchronous resets—externally synchronized resets and internally synchronized resets. Externally synchronized resets are synchronized to the clock domain outside the FPGA, and are not very common. A power-on asynchronous reset is dual-rank synchronized externally to the system clock and then brought into the FPGA. Inside the FPGA, gate this reset with the data input to the registers to implement a synchronous reset.

Figure 18. Externally Synchronized Reset



The following example shows the Verilog HDL equivalent of the schematic. When you use synchronous resets, the reset signal is not put in the sensitivity list.

The following example shows the necessary modifications that you should make to the internally synchronized reset.

Example 58. Verilog HDL Code for Externally Synchronized Reset

```
module sync_reset_ext (
         input   clock,
         input   reset_n,
         input   data_a,
         input   data_b,
         output  out_a,
         output  out_b
         );
reg      reg1, reg2
assign   out_a  = reg1;
assign   out_b  = reg2;
always @ (posedge clock)
begin
     if (!reset_n)
     begin
         reg1     <= 1'b0;
         reg2     <= 1'b0;
    end
    else
    begin
         reg1     <= data_a;
         reg2     <= data_b;
    end
end
endmodule     //  sync_reset_ext
```

The following example shows the constraints for the externally synchronous reset. Because the external reset is synchronous, you only need to constrain the `reset_n` signal as a normal input signal with `set_input_delay` constraint for `–max` and `–min`.

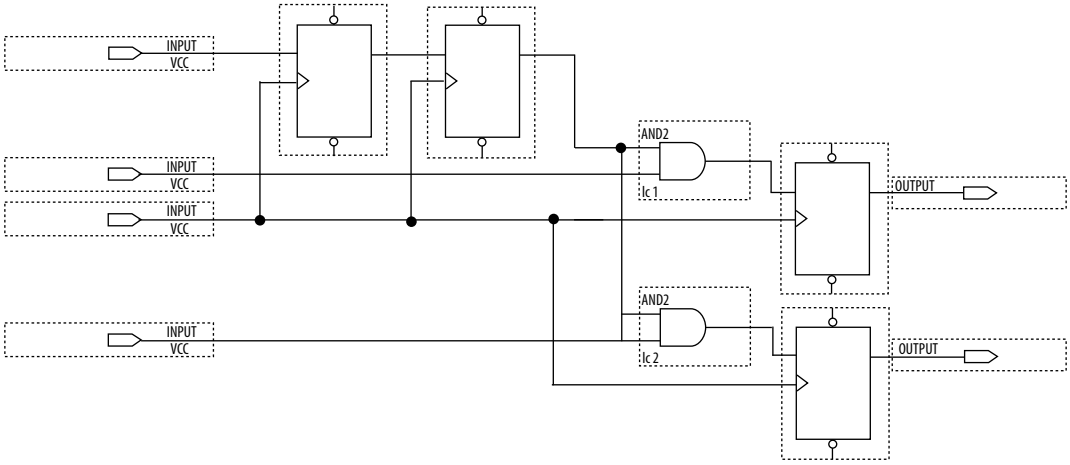
Example 59. SDC Constraints for Externally Synchronized Reset

```
# Input clock - 100 MHz
create_clock [get_ports {clock}] \
        -name {clock} \
        -period 10.0 \
```

```
        -waveform {0.0 5.0}
# Input constraints on low-active reset
# and data
set_input_delay 7.0 \
        -max \
        -clock [get_clocks {clock}] \
        [get_ports {reset_n data_a data_b}]
set_input_delay 1.0 \
        -min \
        -clock [get_clocks {clock}] \
        [get_ports {reset_n data_a data_b}]
```

More often, resets coming into the device are asynchronous, and must be synchronized internally before being sent to the registers.

Figure 19. Internally Synchronized Reset



The following example shows the Verilog HDL equivalent of the schematic. Only the clock edge is in the sensitivity list for a synchronous reset.

Example 60. Verilog HDL Code for Internally Synchronized Reset

```
module sync_reset (
        input clock,
        input reset_n,
        input data_a,
        input data_b,
        output out_a,
        output out_b
        );
reg     reg1, reg2
reg     reg3, reg4

assign     out_a = reg1;
assign     out_b = reg2;
assign     rst_n = reg4;

always @ (posedge clock)
begin
    if (!rst_n)
    begin
        reg1 <= 1'bo;
        reg2 <= 1'b0;
    end
    else
```

```
    begin
        reg1 <= data_a;
        reg2 <= data_b;
    end
end

always @ (posedge clock)
begin
    reg3 <= reset_n;
    reg4 <= reg3;
end
endmodule // sync_reset
```

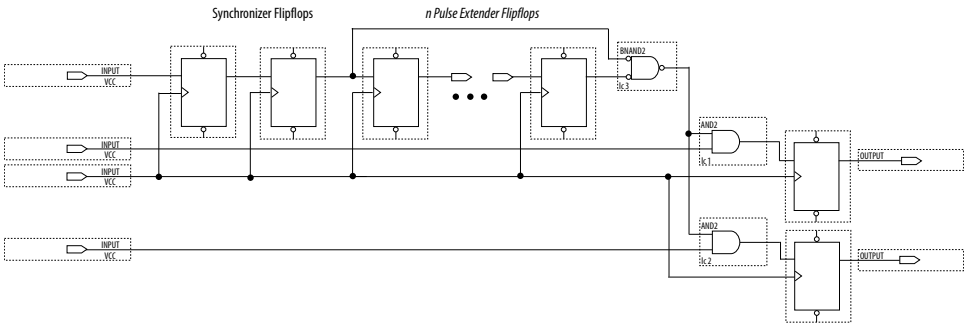
The SDC constraints are similar to the external synchronous reset, except that the input reset cannot be constrained because it is asynchronous. Cut the input path with a `set_false_path` statement to avoid these being considered as unconstrained paths.

Example 61. SDC Constraints for Internally Synchronized Reset

```
# Input clock - 100 MHz
create_clock [get_ports {clock}] \
        -name {clock} \
        -period 10.0 \
        -waveform {0.0 5.0}
# Input constraints on data
set_input_delay 7.0 \
        -max \
        -clock [get_clocks {clock}] \
        [get_ports {data_a data_b}]
set_input_delay 1.0 \
        -min \
        -clock [get_clocks {clock}] \
        [get_ports {data_a data_b}]
# Cut the asynchronous reset input
set_false_path \
        -from [get_ports {reset_n}] \
        -to [all_registers]
```

An issue with synchronous resets is their behavior with respect to short pulses (less than a period) on the asynchronous input to the synchronizer flipflops. This can be a disadvantage because the asynchronous reset requires a pulse width of at least one period wide to guarantee that it is captured by the first flipflop. However, this can also be viewed as an advantage in that this circuit increases noise immunity. Spurious pulses on the asynchronous input have a lower chance of being captured by the first flipflop, so the pulses do not trigger a synchronous reset. In some cases, you might want to increase the noise immunity further and reject any asynchronous input reset that is less than `n` periods wide to debounce an asynchronous input reset.

Figure 20. Internally Synchronized Reset with Pulse Extender



Junction dots indicate the number of stages. You can have more flipflops to get a wider pulse that spans more clock cycles.

Many designs have more than one clock signal. In these cases, use a separate reset synchronization circuit for each clock domain in the design. When you create synchronizers for PLL output clocks, these clock domains are not reset until you lock the PLL and the PLL output clocks are stable. If you use the reset to the PLL, this reset does not have to be synchronous with the input clock of the PLL. You can use an asynchronous reset for this. Using a reset to the PLL further delays the assertion of a synchronous reset to the PLL output clock domains when using internally synchronized resets.

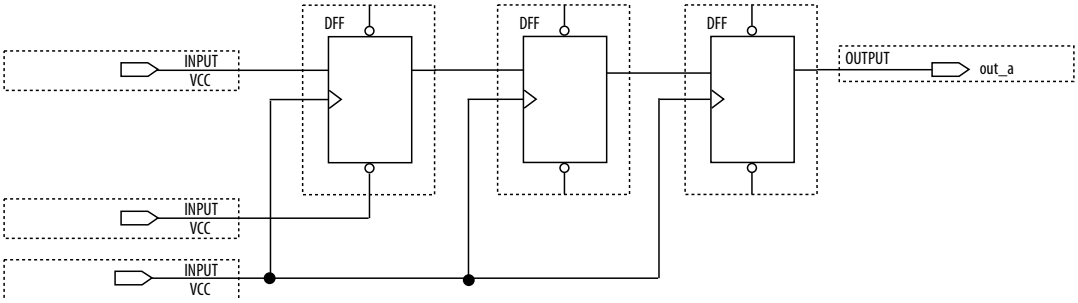
3.3.1.2. Using Asynchronous Resets

Asynchronous resets are the most common form of reset in circuit designs, as well as the easiest to implement. Typically, you can insert the asynchronous reset into the device, turn on the global buffer, and connect to the asynchronous reset pin of every register in the device.

This method is only advantageous under certain circumstances—you do not need to always reset the register. Unlike the synchronous reset, the asynchronous reset is not inserted in the datapath, and does not negatively impact the data arrival times between registers. Reset takes effect immediately, and as soon as the registers receive the reset pulse, the registers are reset. The asynchronous reset is not dependent on the clock.

However, when the reset is deasserted and does not pass the recovery (μt_{SU}) or removal (μt_{H}) time check (the Timing Analyzer recovery and removal analysis checks both times), the edge is said to have fallen into the metastability zone. Additional time is required to determine the correct state, and the delay can cause the setup time to fail to register downstream, leading to system failure. To avoid this, add a few follower registers after the register with the asynchronous reset and use the output of these registers in the design. Use the follower registers to synchronize the data to the clock to remove the metastability issues. You should place these registers close to each other in the device to keep the routing delays to a minimum, which decreases data arrival times and increases MTBF. Ensure that these follower registers themselves are not reset, but are initialized over a period of several clock cycles by “flushing out” their current or initial state.

Figure 21. Asynchronous Reset with Follower Registers



The following example shows the equivalent Verilog HDL code. The active edge of the reset is now in the sensitivity list for the procedural block, which infers a clock enable on the follower registers with the inverse of the reset signal tied to the clock enable. The follower registers should be in a separate procedural block as shown using non-blocking assignments.

Example 62. Verilog HDL Code of Asynchronous Reset with Follower Registers

```
module async_reset (
        input   clock,
        input   reset_n,
        input   data_a,
        output   out_a,
        );
reg     reg1, reg2, reg3;
assign  out_a  = reg3;
always @ (posedge clock, negedge reset_n)
begin
    if (!reset_n)
        reg1    <= 1'b0;
    else
        reg1    <= data_a;
end
always @ (posedge clock)
begin
    reg2    <= reg1;
    reg3    <= reg2;
end
endmodule  //  async_reset
```

You can easily constrain an asynchronous reset. By definition, asynchronous resets have a non-deterministic relationship to the clock domains of the registers they are resetting. Therefore, static timing analysis of these resets is not possible and you can use the `set_false_path` command to exclude the path from timing analysis. Because the relationship of the reset to the clock at the register is not known, you cannot run recovery and removal analysis in the Timing Analyzer for this path. Attempting to do so even without the false path statement results in no paths reported for recovery and removal.

Example 63. SDC Constraints for Asynchronous Reset

```
# Input clock - 100 MHz
create_clock [get_ports {clock}] \
        -name {clock} \
        -period 10.0 \
        -waveform {0.0 5.0}
```

```
# Input constraints on data
set_input_delay 7.0 \
        -max \
        -clock [get_clocks {clock}]\
        [get_ports {data_a}]
set_input_delay 1.0 \
        -min \
        -clock [get_clocks {clock}] \
        [get_ports {data_a}]
# Cut the asynchronous reset input
set_false_path \
        -from [get_ports {reset_n}] \
        -to [all_registers]
```

The asynchronous reset is susceptible to noise, and a noisy asynchronous reset can cause a spurious reset. You must ensure that the asynchronous reset is debounced and filtered. You can easily enter into a reset asynchronously, but releasing a reset asynchronously can lead to potential problems (also referred to as "reset removal") with metastability, including the hazards of unwanted situations with synchronous circuits involving feedback.

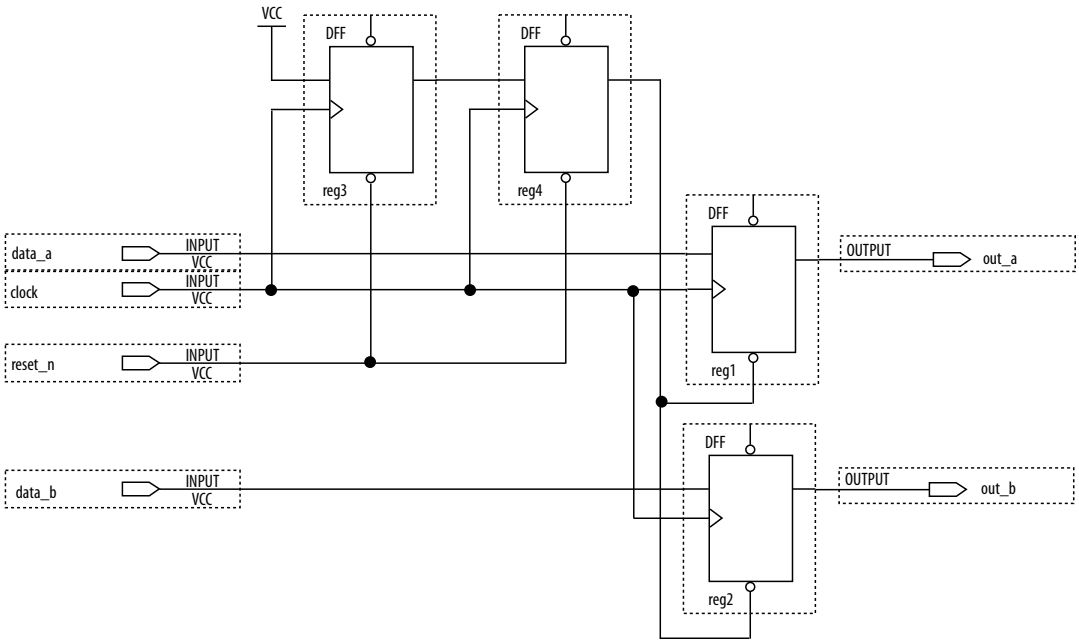
3.3.1.3. Use Synchronized Asynchronous Reset

To avoid potential problems associated with purely synchronous resets and purely asynchronous resets, you can use synchronized asynchronous resets. Synchronized asynchronous resets combine the advantages of synchronous and asynchronous resets.

These resets are asynchronously asserted and synchronously deasserted. This takes effect almost instantaneously, and ensures that no datapath for speed is involved. Also, the circuit is synchronous for timing analysis and is resistant to noise.

The following example shows a method for implementing the synchronized asynchronous reset. You should use synchronizer registers in a similar manner as synchronous resets. However, the asynchronous reset input is gated directly to the `CLRN` pin of the synchronizer registers and immediately asserts the resulting reset. When the reset is deasserted, logic "1" is clocked through the synchronizers to synchronously deassert the resulting reset.

Figure 22. Schematic of Synchronized Asynchronous Reset



The following example shows the equivalent Verilog HDL code. Use the active edge of the reset in the sensitivity list for the blocks.

Example 64. Verilog HDL Code for Synchronized Asynchronous Reset

```
module sync_async_reset (
        input    clock,
        input    reset_n,
        input    data_a,
        input    data_b,
        output   out_a,
        output   out_b
        );
reg     reg1, reg2;
reg     reg3, reg4;
assign  out_a    = reg1;
assign  out_b    = reg2;
assign  rst_n    = reg4;
always @ (posedge clock, negedge reset_n)
begin
    if (!reset_n)
    begin
       reg3     <= 1'b0;
       reg4     <= 1'b0;
    end
    else
    begin
       reg3     <= 1'b1;
       reg4     <= reg3;
    end
end
always @ (posedge clock, negedge rst_n)
begin
    if (!rst_n)
    begin
       reg1     <= 1'b0;
```

```
        reg2     <= 1;b0;
    end
    else
    begin
        reg1     <= data_a;
        reg2     <= data_b;
    end
end
endmodule  // sync_async_reset
```

To minimize the metastability effect between the two synchronization registers, and to increase the MTBF, the registers should be located as close as possible in the device to minimize routing delay. If possible, locate the registers in the same logic array block (LAB). The input reset signal (`reset_n`) must be excluded with a `set_false_path` command:

```
set_false_path -from [get_ports {reset_n}] -to [all_registers]
```

The `set_false_path` command used with the specified constraint excludes unnecessary input timing reports that would otherwise result from specifying an input delay on the reset pin.

The instantaneous assertion of synchronized asynchronous resets is susceptible to noise and runt pulses. If possible, you should debounce the asynchronous reset and filter the reset before it enters the device. The circuit ensures that the synchronized asynchronous reset is at least one full clock period in length. To extend this time to *n* clock periods, you must increase the number of synchronizer registers to *n* + 1. You must connect the asynchronous input reset (`reset_n`) to the `CLRN` pin of all the synchronizer registers to maintain the asynchronous assertion of the synchronized asynchronous reset.

3.3.2. Use Global Clock Network Resources

Altera FPGAs provide device-wide global clock routing resources and dedicated inputs. Use the FPGA's low-skew, high fan-out dedicated routing where available.

By assigning a clock input to one of these dedicated clock pins or with an Quartus Prime assignment to assign global routing, you can take advantage of the dedicated routing available for clock signals.

In an ASIC design, you must balance the clock delay distributed across the device. Because Altera FPGAs provide device-wide global clock routing resources and dedicated inputs, there is no need to manually balance delays on the clock network.

Limit the number of clocks in the design to the number of dedicated global clock resources available in the FPGA. Clocks feeding multiple locations that do not use global routing may exhibit clock skew across the device leading to timing problems. In addition, generating internal clocks with combinational logic adds delays on the clock path. Delay on a clock line can result in a clock skew greater than the data path length between two registers. If the clock skew is greater than the data delay, you violate the timing parameters of the register (such as hold time requirements) and the design does not function correctly.

FPGAs offer low-skew global routing resources to distribute high fan-out signals. These resources help with the implementation of large designs with multiple clock domains. Many large FPGA devices provide dedicated global clock networks, regional clock networks, and dedicated fast regional clock networks. These clocks are organized into a hierarchical clock structure that allows multiple clocks in each device region with low

skew and delay. There are typically several dedicated clock pins to drive either global or regional clock networks, and both PLL outputs and internal clocks can drive various clock networks.

Stratix 10 devices have a newer architecture. You can configure Stratix 10 clocking resources to create efficiently balanced clock trees of various sizes, ranging from a single clock sector to the entire device. By default, the Quartus Prime Software automatically determines the size and location of the clock tree. Alternatively, you can directly constrain the clock tree size and location either with a Clock Region assignment or by Logic Lock Regions.

To reduce clock skew in a given clock domain and ensure that hold times are met in that clock domain, assign each clock signal to one of the global high fan-out, low-skew clock networks in the FPGA device. The Quartus Prime software automatically assigns global routing resources for high fan-out control signals, PLL outputs, and signals feeding the global clock pins on the device. To direct the software to assign global routing for a signal, turn on the **Global Signal** option in the Assignment Editor.

Note: Global Signal assignments only controls whether a signal is promoted using the specified dedicated resources or not, but does not control which or how many resources are used.

To take full advantage of the routing resources in a design, make sure that the sources of clock signals (input clock pins or internally-generated clocks) drive only the clock input ports of registers. In older Altera device families, if a clock signal feeds the data ports of a register, the signal may not be able to use dedicated routing, which can lead to decreased performance and clock skew problems. In general, allowing clock signals to drive the data ports of registers is not considered synchronous design and can complicate timing closure.

3.3.3. Use Clock Region Assignments to Optimize Clock Constraints

The Quartus Prime software determines how clock regions are assigned. You can override these assignments with Clock Region assignments to specify that a signal routed with global routing paths must use the specified clock region.

Clock Region assignments allow you to control the placement of the clock region for floorplanning reasons. For example, use a Clock Region assignment to ensure that a certain area of the device has access to a global signal, throughout your design iterations. A Clock Region assignment can also be used in cases of congestion involving global signal resources. By specifying a smaller clock region size, the assignment prevents a signal using spine clock resources in the excluded sectors that may be encountering clock-related congestion.

You can specify Clock Region assignments in the assignment editor.

3.3.3.1. Clock Region Assignments in Stratix 10 Devices

In Stratix 10 devices, clock networks are constructed using programmable clock routing. As with other Altera device families, you can use Clock Region assignments for floorplanning, controlling the size and location of each clock tree.

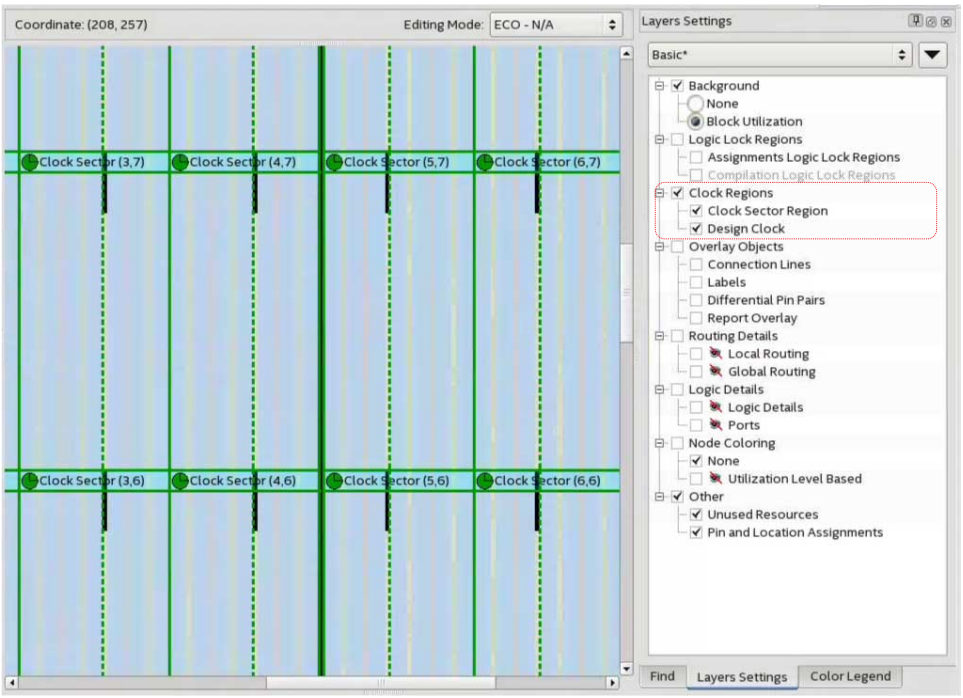
Although the Quartus Prime Pro Edition software generates balanced clock trees, there are sources of timing variation, such as process variation and jitter, which prevents clock trees from being perfectly skew balanced. Longer paths, with higher insertion

delay, have more timing variation. However, the Timing Analyzer can account for and eliminate some sources of variation in timing along common clock paths. In practice, this means that the size of the clock region has a significant impact on the worst-case skew of the clock tree; a larger clock tree experiences higher insertion delay and worst-case clock skew when compared to a smaller clock region. The distance between the clock region and the clock source also increases insertion delay, but the impact of distance on worst-case clock skew is much smaller than the impact of the size of the clock region.

One case to consider is when a design contains high-speed clock domains that are expected to grow during the design process. Specifying a clock region constraint to create a larger clock region than the compiler generates automatically helps ensure that timing closure is robust with higher clock insertion delays and clock skews.

An additional design consideration is the minimum pulse width constraint on clock signals. For a clock signal to propagate correctly on the Stratix 10 clock network, a minimum delay must be met between the rising edge and falling edge of the clock pulse. If the Timing Analyzer cannot guarantee that this constraint is met, the clock signal may not propagate as expected under all operating conditions. This can happen when the delay variation on a clock path becomes too great. This situation does not normally occur, but may arise if clock signals are routed through core logic elements or core routing resources.

In designs that target Stratix 10 devices, clock regions can be constrained to a rectangle whose dimensions are defined by the sector grid, as seen in the Clock Sector Region layer of the Chip Planner.



This assignment specifies the bottom left and top right coordinates of the rectangle in the format "`SX# SY# SX# SY#`". For example, "`SX0 SY0 SX1 SY1`" constrains the clock to a 2x2 region, from the bottom left of sector (0,0) to the top right of sector (1,1). For a constraint spanning only one sector, it is sufficient to specify the location

of that sector, for example "`SX1 SY1`". The bounding rectangle can also be specified by the bottom left and top right corners in chip coordinates, for example, "`X37 Y181 X273 Y324`". However, such a constraint should be sector aligned (using sector coordinates guarantees this) or the Fitter automatically snaps to the smallest sector aligned rectangle that still encompasses the original assignment. The "`SX# SY# SX# SY#`"|"`X# Y# X# Y#`" strings are case-insensitive.

3.3.3.2. Clock Region Assignments in Arria 10 and Older Device Families

In device families with dedicated clock network resources and predefined clock regions, this assignment takes as its value the names of those Global, Regional, Periphery or Spine Clock regions. These region names are visible in Chip Planner by enabling the appropriate Clock Region layer in the **Layers Settings** dialog box. Examples of valid values include `Regional Clock Region 1` or `Periphery Clock Region 1`.

When constraining a global signal to a smaller than normal region, for example, to avoid clock congestion, you may specify a clock region of a different type than the global resources being used. For example, a signal with a Global Signal assignment of `Global Clock`, but a Clock Region assignment of `Regional Clock Region 0`, constrains the clock to use global network routing resources, but only to the region covered by `Regional Clock Region 0`. To provide a finer level of control, you can also list multiple smaller clock regions, separated by commas. For example: `Periphery Clock Region 0, Periphery Clock Region 1` constrains a signal to only the area reachable by those two periphery clock networks.

3.3.4. Avoid Asynchronous Register Control Signals

Avoid using an asynchronous load signal if the design target device architecture does not include registers with dedicated circuitry for asynchronous loads. Also, avoid using both asynchronous clear and preset if the architecture provides only one of these control signals.

Some Altera devices directly support an asynchronous clear function, but not a preset or load function. When the target device does not directly support the signals, the synthesis or placement and routing software must use combinational logic to implement the same functionality. In addition, if you use signals in a priority other than the inherent priority in the device architecture, combinational logic may be required to implement the necessary control signals. Combinational logic is less efficient and can cause glitches and other problems; it is best to avoid these implementations.

3.4. Implementing Embedded RAM

Altera's dedicated memory architecture offers many advanced features that you can enable with Altera-provided IP cores. Use synchronous memory blocks for your design, so that the blocks can be mapped directly into the device dedicated memory blocks.

You can use single-port, dual-port, or three-port RAM with a single- or dual-clocking method. You should not infer the asynchronous memory logic as a memory block or place the asynchronous memory logic in the dedicated memory block, but implement the asynchronous memory logic in regular logic cells.

Altera memory blocks have different read-during-write behaviors, depending on the targeted device family, memory mode, and block type. Read-during-write behavior refers to read and write from the same memory address in the same clock cycle; for example, you read from the same address to which you write in the same clock cycle.

You should check how you specify the memory in your HDL code when you use read-during-write behavior. The HDL code that describes the read returns either the old data stored at the memory location, or the new data being written to the memory location.

In some cases, when the device architecture cannot implement the memory behavior described in your HDL code, the memory block is not mapped to the dedicated RAM blocks, or the memory block is implemented using extra logic in addition to the dedicated RAM block. Implement the read-during-write behavior using single-port RAM in Arria GX devices and the Cyclone and Stratix series of devices to avoid this extra logic implementation.

In many synthesis tools, you can specify that the read-during-write behavior is not important to your design; if, for example, you never read and write from the same address in the same clock cycle.

Related Information

Inferring RAM functions from HDL Code on page 10

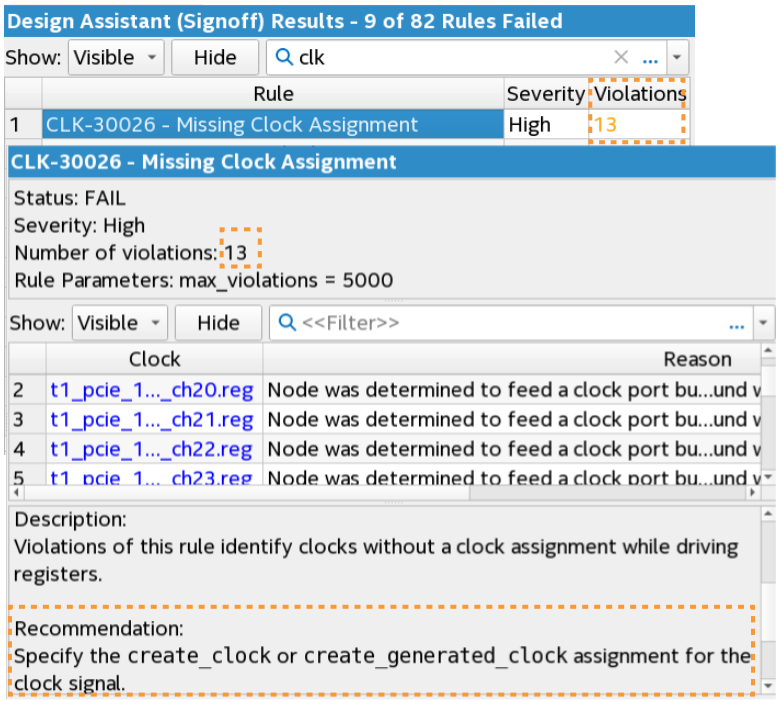
3.5. Design Assistant Design Rule Checking

The Quartus Prime Design Assistant increases productivity by reducing the total number of design iterations for design closure, and by minimizing the time in each iteration with targeted rule checks and guidance at each stage of compilation.

The Design Assistant detects and helps you to resolve design rule violations by providing recommendations for correction and pathways to the violation source. Avoiding design rule violations improves the reliability, timing performance, and logic utilization of your design.

When enabled, Design Assistant automatically reports any violations against a standard set of recommended design guidelines [(1)]. You can run Design Assistant automatically during compilation, and report violations detected throughout the compilation process.

Figure 23. Design Assistant Recommends Corrections for Design Rule Violations



Alternatively, you can run Design Assistant in analysis mode, which allows you to launch Design Assistant checks from other Quartus Prime tools, such as Chip Planner. For some rules, Design Assistant supports cross-probing to the Timing Analyzer and Quartus Prime design visualization tools for root cause analysis and correction.

You can specify which rules Design Assistant checks, thus eliminating the rule checks that are unimportant for your design.

---

(1) A set of default rules ensures design health without significant runtime increase.

Related Information

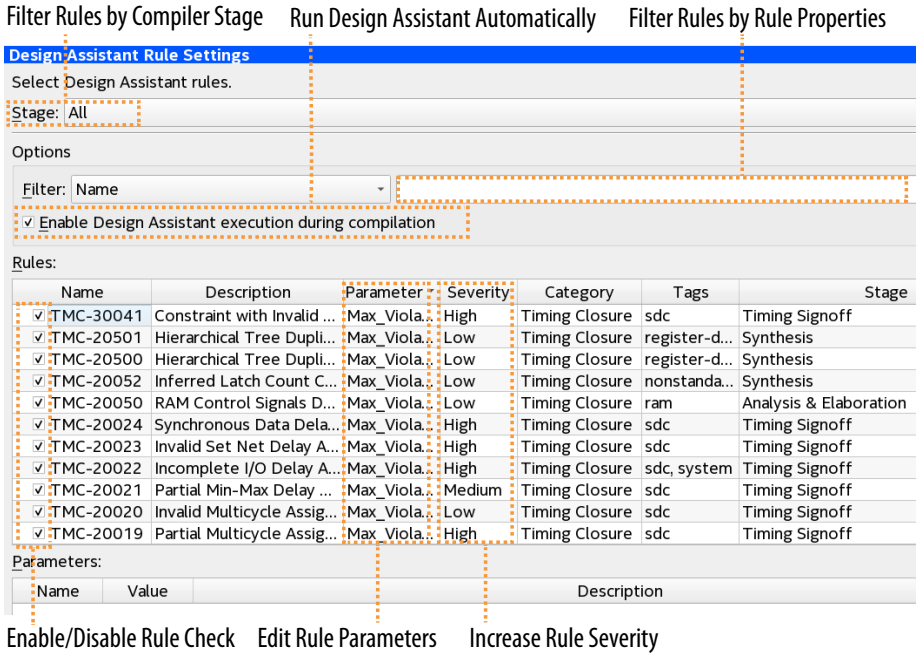
AN 919: Improving Quality of Results with Design Assistant

3.5.1. Setting Up Design Assistant

Customize the Design Assistant for individual design characteristics and reporting requirements. For example, you can disable rules for specific stages of compilation, change the threshold for violation reporting, and other options. Follow these steps to specify initial options for running Design Assistant:

1. Open an Quartus Prime project.
2. Click **Assignments ➤ Settings ➤ Design Assistant Rule Settings**.

Figure 24. Design Assistant Rule Settings



3. Use the default settings or specify any of the following options:

Table 4. Design Assistant Rule Settings

Option	Description
Stage filter	Filters the **Rules** list by **All**, **Analysis & Elaboration**, **Synthesis**, **Plan**, **Place**, **Finalize** or **Timing Signoff** Compiler stages.
Text **Filter**	Filters the **Rules** list by matching text and the **Name**, **Description**, **Parameter**, **Severity**, **Category**, or **Tags** of the rule.
Enable Design Assistant execution during compilation	Runs Design Assistant automatically during compilation. Alternatively, enable this setting with `FLOW_ENABLE_DESIGN_ASSISTANT` in the `.qsf`. The settings in this dialog have no impact when this setting is disabled.
Rules	Lists all available Design Assistant rules and properties. Enable or disable analysis for the rule by enabling or disabling the rule **checkbox**.

continued...

Option	Description
Name Column	Specifies the alphanumeric rule ID. Rules that apply to more than one Compiler stage have sub-rules for each stage.
Description Column	Summary rule description.
Parameter Column	Lists rule parameters or "Multiple Values" for rules that support multiple Compiler stages. Select any rule to edit parameter values. Specify parameters on a per-stage basis by specifying parameters for the stage subrule.
Severity Column	Specifies **Low**, **Medium**, **High**, **Critical**, or **Fatal** as the rule **Severity** for reporting. You can increase the **Severity** level of rules.
Category Column	Specifies the rule class, such as **Timing Closure**, **Reset**, and others.
Tags	Specifies one or more additional facet of the rule for search and filtering purposes. For example, **global-signal** tag for design rule checks related to global signals. *Design Assistant Tags* defines the meaning of each tag.
Stage Column	Specifies the Compiler stages to which the rule applies. Rules for **Analysis & Elaboration**, **Synthesis**, **Plan**, **Place**, **Finalize**, and **Timing Signoff** stages are available. Enable or disable the rule on a per-stage basis by enabling or disabling the **checkbox** option for the stage subrule.
Parameters for rule Column	Allows you to specify parameters for rules that support parameters. Specify parameters on a per-stage basis by specifying parameters for the stage subrule.

Related Information

- Managing Design Assistant Rules on page 110
- Design Assistant Tags on page 119

3.5.1.1. Design Assistant Rule Severity Levels

Design Assistant designates each rule violation with a severity level. You can increase the severity level for any rule to match your particular design requirements.

Table 5. Design Assistant Rule Severity Levels

Severity	Description	Severity Level Color
Fatal	Failure condition that stops the Compiler flow after violation.	Red
Critical	A critical issue that requires correction for sign-off.	Red
High	Potentially causes functional failure. May indicate missing or incorrect design data.	Yellow
Medium	Potentially impacts quality of results for f_{MAX} or resource utilization.	Brown
Low	Optional suggestions that can reflect FPGA design best practices and may have small impact on device performance and utilization in typical designs.	Blue

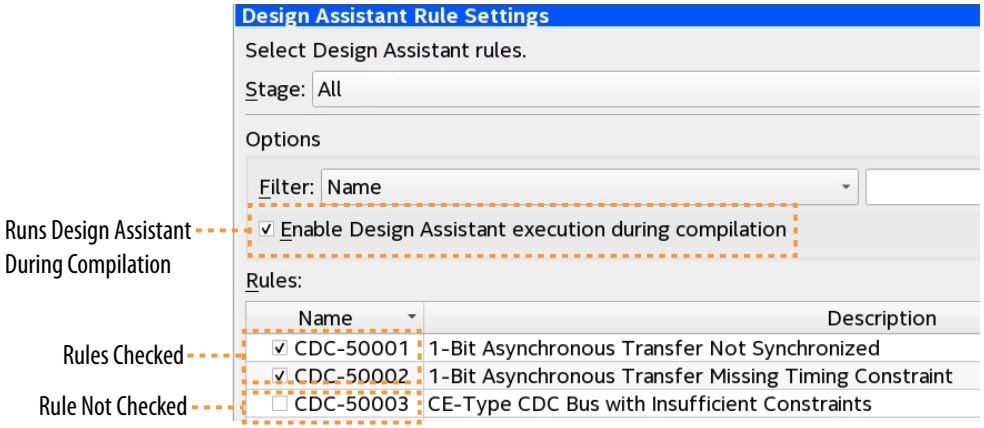
3.5.2. Running Design Assistant During Compilation

When enabled, Design Assistant runs automatically during compilation and reports design rule violations in the Compilation Report.

When you enable or specify parameters for a rule check in compilation mode, those specifications apply by default to running Design Assistant in compilation mode. If you change the rule settings for analysis mode, those settings are independent from the rule settings in compilation mode.

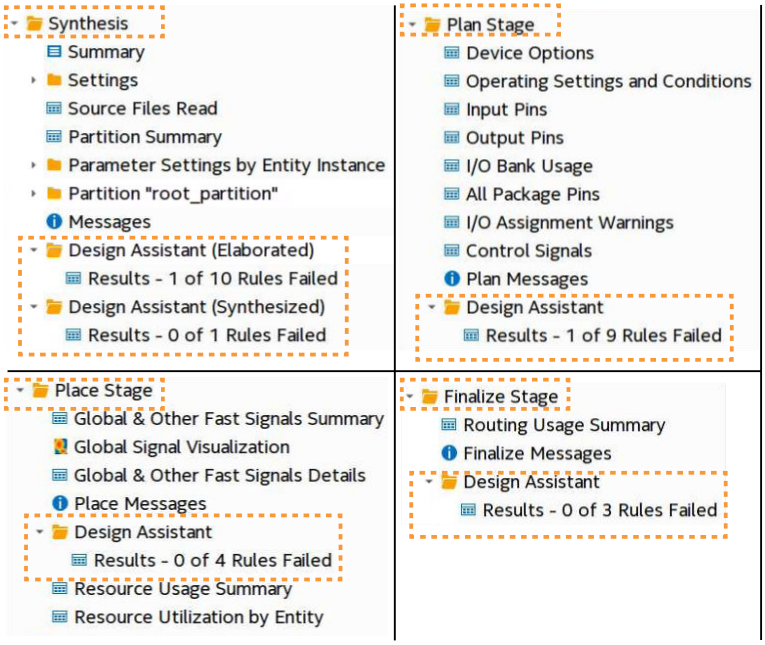
1. To run Design Assistant checking during compilation flows, ensure that **Enable Design Assistant execution during compilation** is on.
2. To enable or disable specific design rule checks, turn on or off the checkbox for that rule in the **Name** column. If the rule is unchecked, Design Assistant does not report violations for the rule.
3. In the **Parameters** field, consider changing default values for rules you enable.

Figure 25. Design Assistant Rule Settings



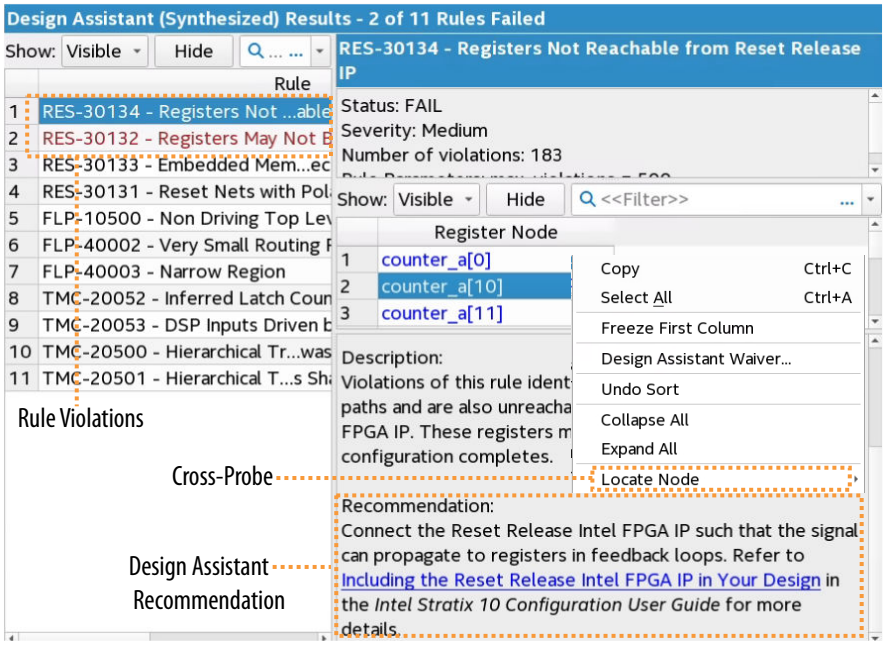
4. To run Design Assistant during compilation, run one or more stages of the Compiler from the Processing menu or Compilation Dashboard.

Figure 26. Example Design Assistant Results in Compilation Reports



5. To view the results for each rule, click the rule in the **Rules** list. A description of the rule and design recommendations for correction appear.

Figure 27. Design Assistant Rule Violation Recommendation

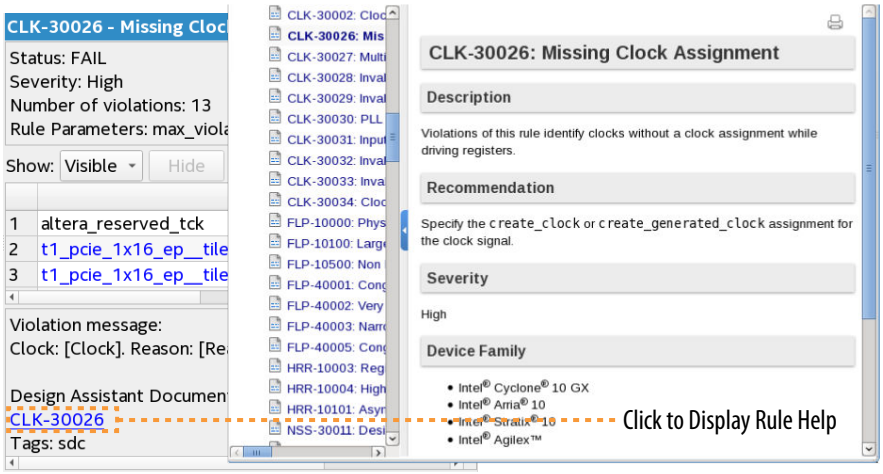


3.5.2.1. Opening Design Assistant Rule Help

Each rule report contains a link to the rule's Quartus Prime Help topic. Help includes full rule descriptions and diagrams. To link to the Design Assistant rule Help:

1. In the Design Assistant report, click any rule in the **Rule** list. The right pane shows the rule description.

Figure 28. Linking in Design Assistant Reports to Rule Help



2. Click the rule ID link under **Design Assistant Document**. The rule displays in Help.

3.5.3. Running Design Assistant in Analysis Mode

You can launch Design Assistant in analysis mode directly from the Timing Analyzer or Chip Planner to rapidly run the specific rule checks that relate to those tools. For example, when you launch Design Assistant from the Chip Planner, Design Assistant is preset to check only a subset of the FLP (floorplanning) Design Assistant rules.

Similarly, when you launch Design Assistant from the Timing Analyzer, Design Assistant is preset to check only a subset of rules that are helpful during timing analysis. You can cross-probe to the Timing Analyzer and design visualization tools to determine the root cause of violations.

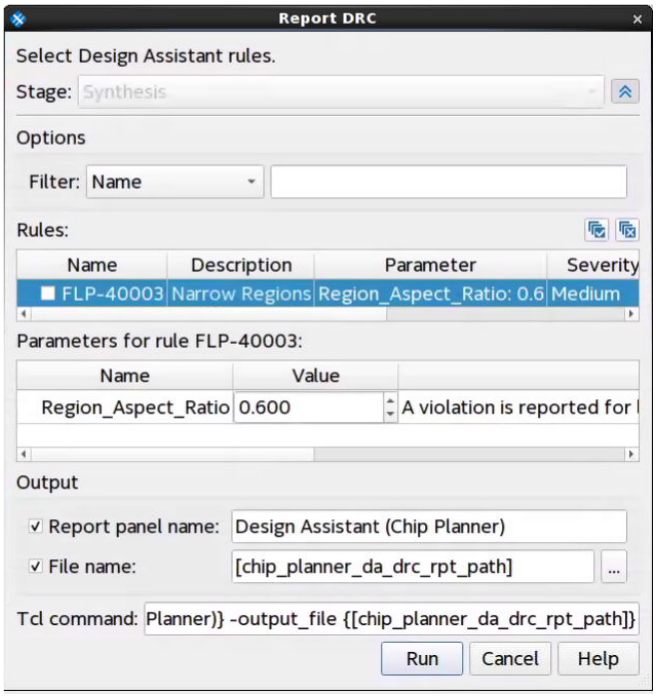
When you enable or specify parameters for a rule check in analysis mode, those specifications do not apply to running Design Assistant in compilation mode. The rule settings for analysis mode are independent from the rule settings in compilation mode.

3.5.3.1. Launching Design Assistant from Chip Planner

You can run Design Assistant directly from Chip Planner to assist when optimizing the floorplan in the tool. When you launch Design Assistant from the Chip Planner, Design Assistant is preset to check only the FLP (floorplanning) Design Assistant rules. Follow these steps to run the Design Assistant from the Chip Planner:

1. Run any stage of the Compiler. You must run at least the Analysis & Elaboration stage before running Design Assistant from Chip Planner.
2. Click **Tools ➤ Chip Planner**.

Figure 29. Report DRC Dialog Box in Chip Planner



3. In Chip Planner **Tasks** pane, click **Report DRC** under **Design Assistant**. The **Report DRC** (design rule check) dialog box appears.
4. Under **Rules**, disable any rules that are not important to your analysis by removing the check mark.
5. Consider whether to adjust rule parameter values in the **Parameters** field.
6. Under **Output**, confirm the **Report panel name** and optionally specify an output **File name**.
7. Click **Run**. The Results reports generate and appear in the **Report** pane and in the Compilation Report.

Figure 30. Rule Violations in Chip Planner Reports Pane

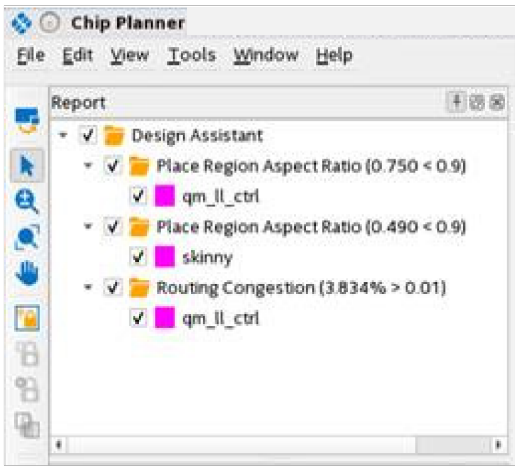
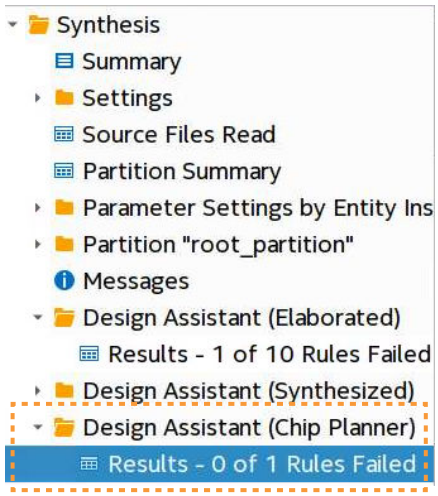


Figure 31. Chip Planner Rule Violations in Main Compilation Report



3.5.3.2. Launching Design Assistant from Timing Analyzer

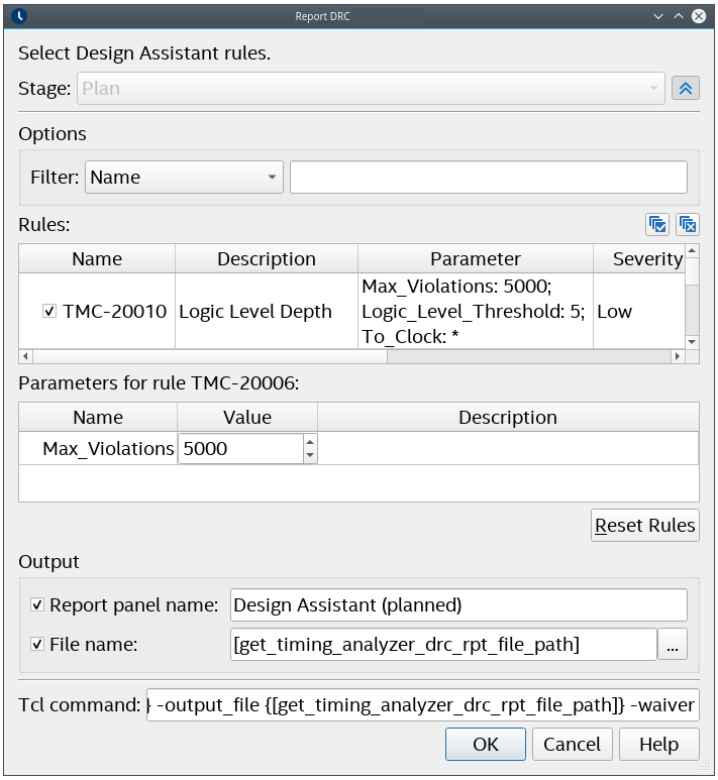
You can run Design Assistant directly from the Timing Analyzer to assist when optimizing timing paths and other timing conditions. When you launch Design Assistant from the Timing Analyzer, Design Assistant only checks rules that relate to timing analysis.

Follow these steps to run the Design Assistant from the Timing Analyzer:

1. Compile the design through at least the Compiler's Plan stage.
2. Open the Timing Analyzer for the Compiler stage from the Compilation Dashboard.
3. In the Timing Analyzer, click **Reports ➤ Design Assistant ➤ Report DRC**. The **Report DRC** (design rule check) dialog box opens.
4. Under **Rules**, disable any rules that are not important to your analysis by removing the check mark.

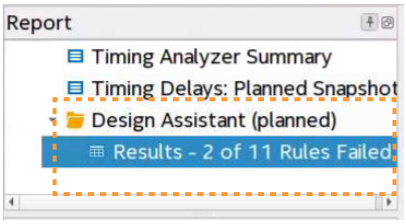
5. Consider whether to adjust rule parameter values in the **Parameters** field.

Figure 32. Report DRC (Design Rule Check) Dialog Box



6. Confirm the **Report panel name** and optionally specify an output **File name**.
7. Click **Run**. The Results reports generate and appear in the **Report** pane, as well as the main Compilation Report.

Figure 33. Design Assistant Reports in Timing Analyzer Report Pane



3.5.4. Cross-Probing from Design Assistant

In addition to the Locate Node command, Design Assistant allows you to cross-probe individual design objects relevant to the violation. For select high-value rules, Design Assistant provides full violation cross-probing ability into the Quartus Prime Timing Analyzer and other design visualization tools.

For example, for rule TMC-20210 - Paths Failing Setup Analysis with High Routing Delay Added for Hold, you can right-click the violation, and then click **Report Timing (Extra Info)** to locate the path in the Timing Analyzer GUI.

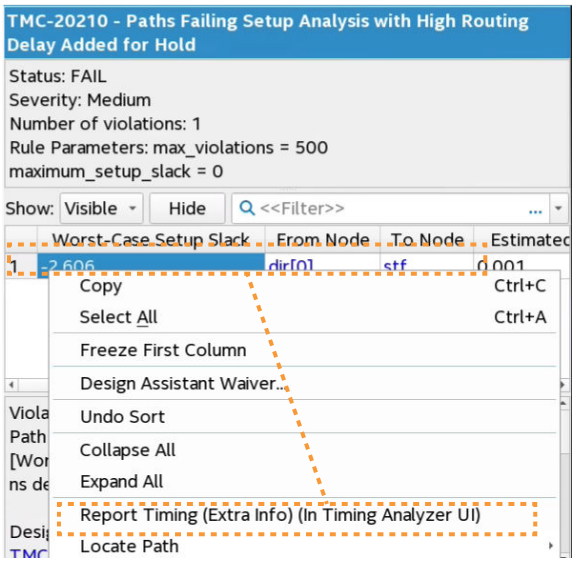
You can also locate from a rule violation instance to the source of the violation in Quartus Prime design visualization tools, such as RTL Viewer, Resource Property Viewer, Technology Map Viewer, and Chip Planner. You can also locate to the violation source in the design file.

Cross-probing with Design Assistant can help you to more rapidly identify the root cause and resolve any rule violations negatively impacting your design.

3.5.4.1. Cross-Probing from Design Assistant to Timing Analyzer

Some Design Assistant rule violations allow cross-probing into Timing Analyzer. For example, for a path that Design Assistant flags with a setup analysis violation due to delay added for hold, you can cross-probe into the Timing Analyzer to view more information on the affected path and edge.

Figure 34. Cross Probing from Design Assistant Rule TMC-20210 Violations to Timing Analyzer



Follow these steps to cross-probe from such Design Assistant rule violations to the Timing Analyzer:

1. Compile the design through at least the Compiler's Plan stage.
2. Locate a rule violation in the Design Assistant folder of the Compilation Report.
3. Right-click the rule violation to display any **Report Timing** commands available for the violation.
4. Click the **Report Timing** command. The Timing Analyzer opens and reports the timing data for the violation path. **Report Timing (Extra Info)** includes Estimated Delay Added for Hold and Route Stage Congestion Impact extra data.

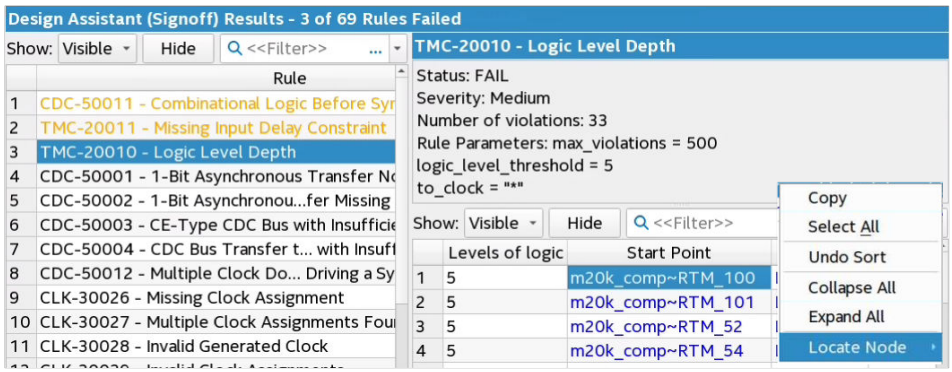
3.5.4.2. Cross-Probing from Design Assistant to Visualization Tools

Design Assistant can cross-probe from rule violations to the source in various Quartus Prime design visualization tools. The following example demonstrates expanding from the cross-probing location for violation analysis.

The following example illustrates cross probing for the TMC-20010 Logic Level Depth rule violation to the RTL Viewer:

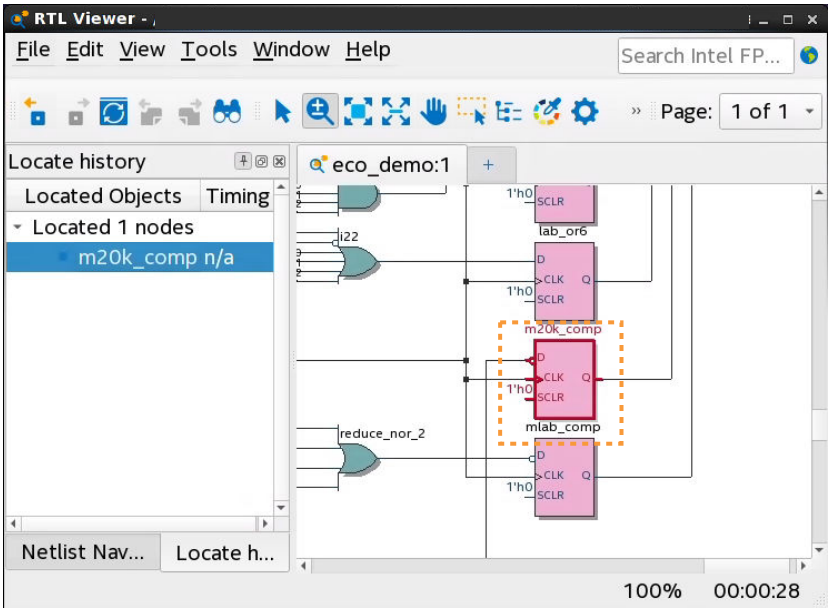
1. When Design Assistant reports FAIL status for rule TMC-20010, you can right-click any of the rule violations in the Design Assistant report, and then click **Locate Node ➤ Locate in RTL Viewer**.

Figure 35. Locate in RTL Viewer



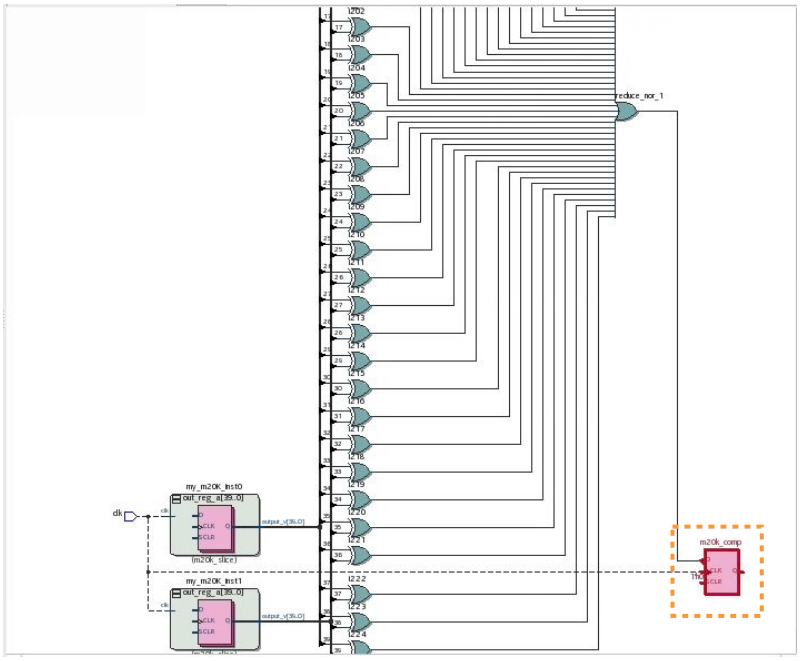
Cross-probing allows you to locate the driver register in the RTL Viewer.

Figure 36. Driver Register in RTL Viewer



2. To then fully visualize the logic level depth, right-click the register and click **Filter** to display **Sources and Destinations** of the register.

Figure 37. Expanded Connections



3.5.5. Managing Design Assistant Rules

The Design Assistant provides functions to help you manage the rules that are important for your design characteristics. You can use these features to help ensure that you are only checking rules that are important for your design at the relevant stage of compilation.

Design Assistant provides the following functionality to help you manage rule checking and reporting:

- Waiving Design Assistant Rules on page 112
- Enabling Rules for Specific Compiler Stages on page 111
- Specifying Rule Parameters for a Specific Compiler Stage on page 111
- Modifying Rule Severity Levels on page 112
- Changing the Default Number of Violations per Rule on page 110

3.5.5.1. Changing the Default Number of Violations per Rule

The default number of violations per rule is set to 5000 to limit runtime that rule processing incurs. The default limit is set to 5000 to help ensure that every violation is presented in all but the most exceptional cases. In typical designs, most rule violations do not approach this limit.

You can change the default number of violations that Design Assistant reports per rule, to show more or less violations per rule. Specify the following assignment in the project `.qsf` to change the default number of violations per rule:

```
set_global_assignment -name DESIGN_ASSISTANT_MAX_VIOLATIONS_PER_RULE <number>
```

To specify no limit on the number of violations per rule, at the possible expense of increased runtimes, enter `-1` for the `<number>` of the `DESIGN_ASSISTANT_MAX_VIOLATIONS_PER_RULE` assignment.

Change the default number of violations for individual rules in the **Design Assistant Settings** page, as Setting Up Design Assistant on page 100 describes.

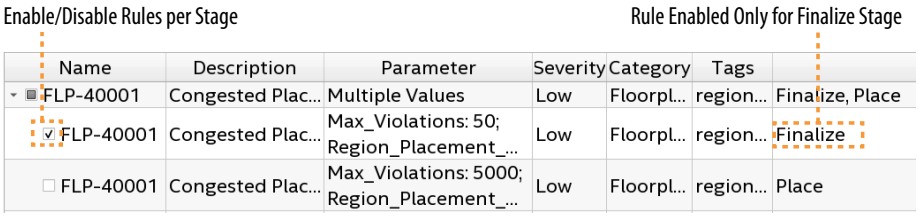
3.5.5.2. Enabling Rules for Specific Compiler Stages

You can enable or disable checking of Design Assistant rules on a per stage basis. This feature allows you to disable rule checks that are not important during one or more stage of compilation, while leaving the rule enabled for other stages.

To enable or disable rules for specific Compiler stages, follow these steps:

1. Specify initial Design Assistant Settings, as Setting Up Design Assistant on page 100 describes.
2. In the **Design Assistant Rule Settings** page, click the arrow next to a rule that supports multiple Compiler stages to expand the subrules for each stage.
3. For the subrule, enable or disable the **checkbox** to enable or disable checking for that rule during that Compiler stage.

Figure 38. Enable or Disable Rules per Stage



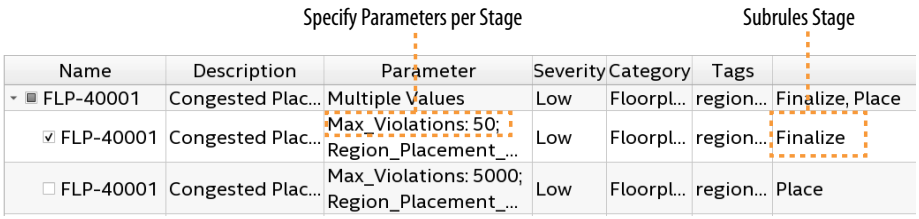
3.5.5.3. Specifying Rule Parameters for a Specific Compiler Stage

You can specify different parameters for Design Assistant rules for each Compiler stages. This feature allows you apply a set of rule parameters to a specific stage of compilation, and have a different set of rule parameters for another stage.

To enable or disable parameters for specific Compiler stages, follow these steps:

1. Specify initial Design Assistant Settings, as Setting Up Design Assistant on page 100 describes.
2. In the **Design Assistant Rule Settings** page, click the arrow next to a rule that supports multiple Compiler stages to expand the subrules for each stage.
3. Select the subrule and then specify the parameters in **Parameters for rule**.

Figure 39. Specifying Parameters Per Stage



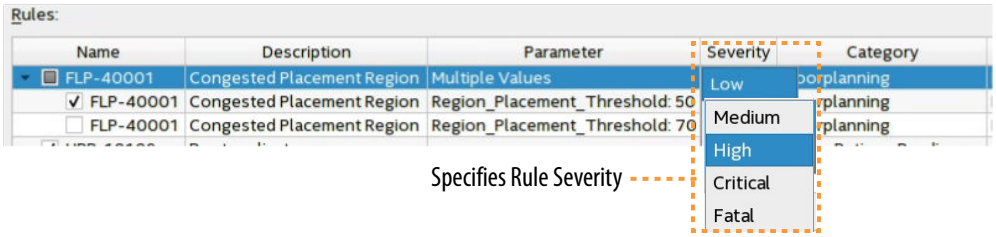
3.5.5.4. Modifying Rule Severity Levels

You can increase the severity level of a Design Assistant rule to match the importance of the rule for your design. You cannot decrease the severity level below the default. Design Assistant messages and reports reflect the rule severity level. You can filter and hide rule messages based on the severity level that you specify.

To customize rule severity level, follow these steps:

1. Specify initial Design Assistant Settings, as Setting Up Design Assistant on page 100 describes.
2. In the **Design Assistant Rule Settings** page, select the rule with a **Severity** that you want to change. You can only change the severity level of parent rules. Subrules for each stage must reflect the parent rule **Severity** level.
3. Click the **Severity** cell and select **Low**, **Medium**, **High**, **Critical**, or **Fatal**. Design Assistant reports the severity level you specify for the rule violations. Fatal violations cause failure of the Compiler stage.

Figure 40. Modifying Rule Severity Level



Note: Fatal violations indicate conditions that cause design failure and therefore cause the Compiler stage to be unsuccessful. You must correct the fatal condition, reduce the rule **Severity**, or create a rule waiver before proceeding to the next Compiler stage.

Figure 41. Messages Report Fatal Rule Violation Causes Compiler Failure

```
Design Assistant Results: 1 of 1 Fatal severity rules issued violations in snapshot 'final'.
Design Assistant Results: 2 of 32 High severity rules issued violations in snapshot 'final'.
Design Assistant Results: 0 of 26 Medium severity rules issued violations in snapshot 'final'
Design Assistant Results: 0 of 10 Low severity rules issued violations in snapshot 'final'
Quartus Prime Timing Analyzer was unsuccessful. 1 error, 1 warning
```

3.5.5.5. Waiving Design Assistant Rules

After running an initial design rule check, you can waive (ignore) design rule violations that you determine are unimportant for one or more iterations of design rule checking. When you create a waiver, Design Assistant does not check for compliance with the rules that match the violation conditions you specify, nor report results for the rule. For teams or individual designers, rule waivers also provide an audit trail that tracks the user, description, and reason for the design rule waiver.

You can create rule waivers to ignore violations for which you already have identified the root cause and correction, for violations that occur in a block that another developed owns, or to waive specific rules that you determine are not an issue for your design.

Initially, run Design Assistant checks without rule waivers to evaluate the complete list of violations. As you begin root cause analysis and violation correction, you can consider creating waivers to eliminate one or more rule violations from obscuring the rule violations that are still relevant.

After creating a design rule waiver, you can modify the rule parameters to fine tune rule checks, or you can delete waivers. For example, if a first pass rule check reports 800 violations with the **Max_Violations** per rule parameter set to default of 500, Design Assistant reports only the first 500 of the 800 total violations. You could then create rule waivers to omit the first 100 rule violations that you correct, thereby reporting rule violations number 501 and higher the next time you run Design Assistant.

When a Design Assistant waiver becomes completely unneeded, you can delete the waiver in the **Design Assistant Manage Waivers** dialog box or directly from the Design Assistant Waivers (`.dawf`) file.

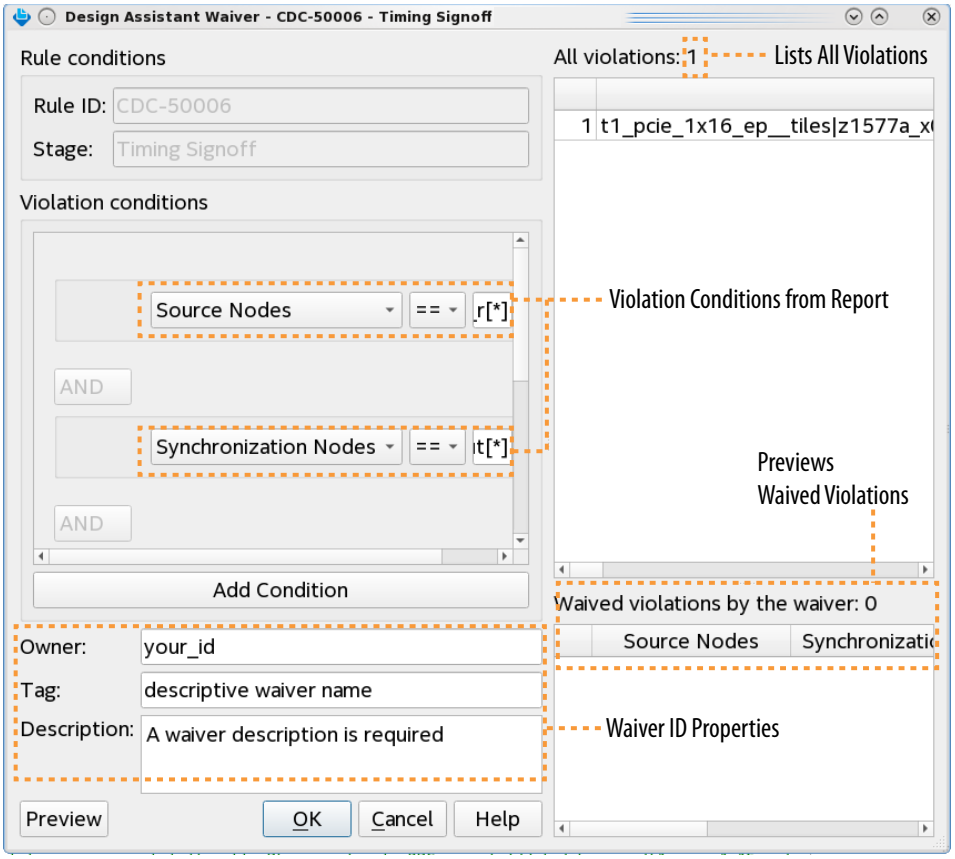
Creating Design Assistant Waivers on page 113
Design Assistant Waiver Dialog Box on page 115
Deleting Design Assistant Waivers on page 116
Design Assistant Waiver Tcl Commands on page 117
drc::add_waiver Command on page 117
drc::get_waivers Command on page 118
drc::report_waivers Command on page 119

3.5.5.5.1. Creating Design Assistant Waivers

To create a design rule waiver, follow these steps:

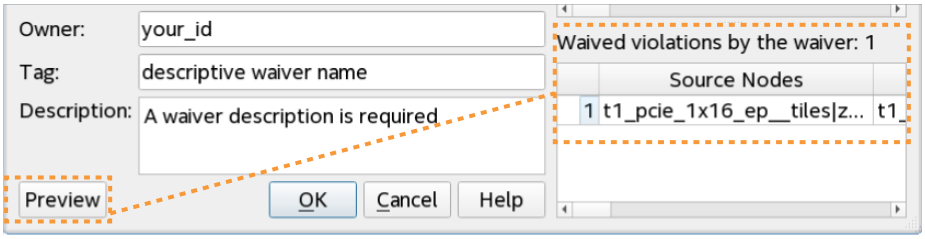
1. Run one or more stages of the Compiler to generate Design Assistant reports for the rules that you enable for your design.
2. In the Design Assistant report, right-click one or more rule violations, or right-click an entire rule category in the rule summary list, and then click **Design Assistant Waiver**. The **Design Assistant Waiver** dialog box opens preset with values from your violation selection.

Figure 42. Right-Click Rule Violation in Report to Create Waiver for Violation



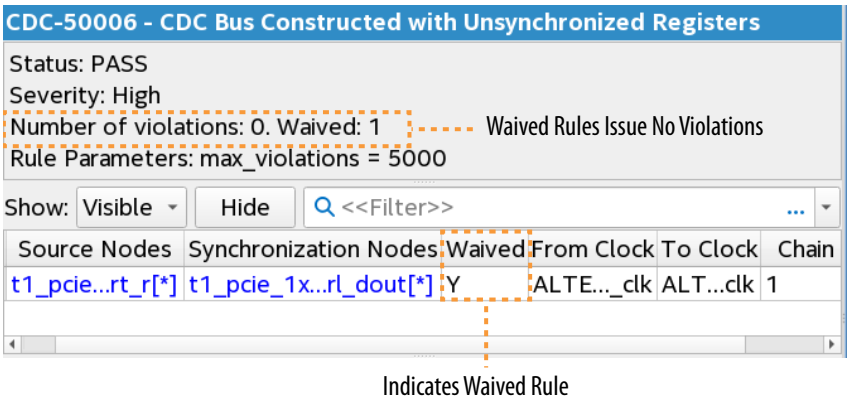
3. Modify any of the default **Violation conditions** that define when the waiver applies. The default settings are the most descriptive, using all applicable fields. The comparison operator is always **==** (equal to) by default for all conditions. Refer to Design Assistant Waiver Dialog Box on page 115 for all available options.
4. Click the **X** button to delete a sub-condition and simplify the query. Click **Add Condition** to add a violation sub-condition.
5. For waiver identification and audit tracking, optionally specify the waiver **Owner** name, a descriptive **Tag**, and a text **Description**.

Figure 43. Design Assistant Waiver Dialog Box



6. To preview the waived violations, click the **Preview** button. The **Waived violations by the waiver** list shows the waived rule violations during the next Design Assistant run. When you create a waiver after running Design Assistant, the newly added waiver specifies **To be Waived** in the **Waived** column. For any waivers that you delete, the **Waiver** column specifies **Y + To be unwaived**.
7. When waiver definition is complete, click **OK** to apply the waiver the next time you run Design Assistant. Design Assistant does not check for compliance with the rules that match waiver conditions, nor report results for the rules you waive. The Design Assistant reports indicate waived violations following compilation.

Figure 44. Applied Waiver Reported in Compilation Report



The report's **Waived** column specifies **Y** (for yes) for waived violations.

Design Assistant saves the rule waiver to a `da_drc.dawf` file in the project directory.

3.5.5.5.2. Design Assistant Waiver Dialog Box

You can define and apply waivers to Design Assistant rule violations that are not of concern in the **Design Assistant Waiver** dialog box.

The following table shows the **Design Assistant Waiver** dialog box options:

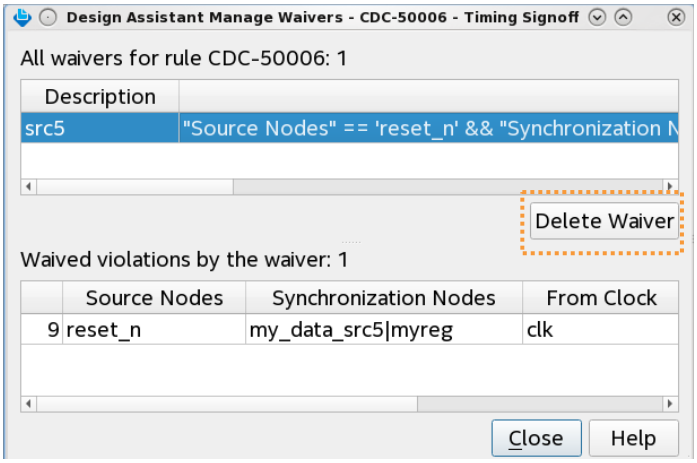
Table 6. Design Assistant Waiver Dialog Box Options

Setting	Description
Rule conditions	Automatically specifies the alphanumeric **Rule ID** and Compiler **Stage** of the rule violation.
Violation conditions	Specifies the conditions that define a rule violation waiver. The default **Violation conditions** automatically reflect the currently selected rule violation from the report. The available condition attributes are context-sensitive: • **==**is equal to • **!=** is not equal to • **<** is less than • **>** is greater than • **<=** is less than or equal to • **>=** is greater than or equal to • **!** is a negative unary operator for boolean expressions • **=~**—string contains • **!~**—string does not contain The join operators between conditions are always **AND**.
Add Condition	Click the **Add Condition** button to add more conditions to the rule waiver definition. Clicking the button adds a condition of default type.
Owner	Specifies the waiver owner's ID.
Tag	Specifies an identifying tag for the rule waiver.
Description	Required value that specifies a text description of the waiver for tracking and identifying.
Preview button	Previews the rule violations waived during the next Design Assistant run in the **Waived by the waiver** field.
Waived violations by the waiver	Upon clicking the **Preview** button, lists the rule violations that are waived during the next Design Assistant run.
All violations	Lists all violations of the currently selected Design Assistant rule.

3.5.5.5.3. Deleting Design Assistant Waivers

To fully remove (delete) any Design Assistant waivers that are no longer useful, delete the waiver in the **Design Assistant Manage Waivers** dialog box.

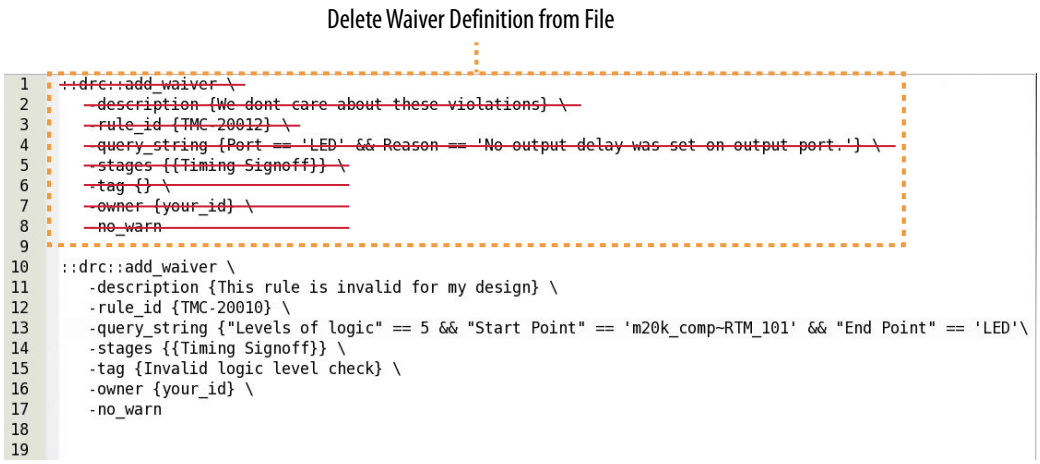
Figure 45. Design Assistant Manage Waivers Dialog Box



1. Right-click a rule violation with a waiver, and click **Waivers ➤ Manage Waivers**.
2. In the **Design Assistant Manager Waivers** dialog box, select the waiver you want to delete under **All waivers for rule**.
3. Click the **Delete Waiver** button to delete the waiver.

As an alternative to the GUI, you can also create and modify Design Assistant waivers by editing the Design Assistant Waivers File (`da_drc.dawf`), or by using Tcl commands within an interactive Tcl shell. The `da_drc.dawf` stores the waiver definitions you specify in the GUI. To avoid unintended results, do not edit the `da_drc.dawf` file while Compiler processes are running.

Figure 46. Deleting Waiver from Design Waivers File



3.5.5.5.4. Design Assistant Waiver Tcl Commands

As an alternative to the Design Assistant GUI, you can create, query, and report Design Assistant waivers by specifying the following Tcl commands within an interactive Tcl shell:

3.5.5.5.5. drc::add_waiver Command

Description

Creates a new Design Assistant waiver for the rule, Compiler stages, and query string that you specify. Optionally maintain audit tracking with the `timestamp` and `owner` arguments. Specify wildcard characters with the `stages` argument to create generic waivers across multiple rules. Specify any subset of violation arguments, comparison operators, and join operators to build any generic query string.

Syntax

```
drc::add_waiver
-description <description text>
-owner <userid>
-rule <rule_id>
query_string
-stages <compiler stages>
[-tag <string>]
```

Arguments

description Text description explaining the reason for the waiver.

owner User ID of waiver creator for audit trail.

rule Alpha-numeric rule ID.

query_string Query string specifying violation column arguments to define the violation patterns the waiver ignores. You can specify all or a subset of all violation column arguments, depending on the scope of the waiver.

stages Subset of Compiler stage(s) to which the rule waiver applies.

tag Short text description for tracking different types of violations across the entire project.

3.5.5.5.6. drc::get_waivers Command

Description

Returns a Design Assistant waiver for the rule, Compiler stages, owner, and query string that you specify. If you specify no option, `drc::get_waivers` returns all existing waivers.

Syntax

```
drc::get_waivers
-owner <userid>
-rule <rule_id>
query_string
-stages <compiler stages>
[-tag <string>]
```

Arguments

owner User ID of waiver creator for audit trail.

rule Alpha-numeric rule ID.

query_string Query string specifying violation column arguments to define the violation patterns the waiver ignores. The query string must exactly match the `query_string` you specify during waiver creation with the `add_waiver` command.

stages Subset of Compiler stage(s) to which the rule waiver applies.

tag Short text description for tracking different types of violations across the entire project.

3.5.5.5.7. drc::report_waivers Command

Description

Returns a collection of Design Assistant waivers for the rule, Compiler stages, and owner that you specify, or adds those waivers to a waiver file that you specify.

Syntax

```
drc::report_waivers
-file <waiver_file>
-owner <userid>
-rule <rule_id>
-stages <compiler stages>
[-tag <string>]
```

Arguments

file A waiver output file that design assistant creates and appends the returned waivers.

owner User ID of waiver creator for audit trail.

rule Alpha-numeric rule ID.

stages Subset of Compiler stage(s) to which the rule waiver applies.

tag Short text description for tracking different types of violations across the entire project.

3.5.5.6. Design Assistant Tags

Different Design Assistant **Tags** apply to each rule to extend search or filter capabilities based on the following facets of the rule. Refer to the **Design Assistant Rule Settings** to view which tags apply to each rule.

Table 7. Design Assistant Tags

Tag	Description
cdc-bus	Design rule checks related to topologies that use a bus to transfer multiple bits of data between clock domains at once.
clock-skew	Design rule checks related to clock skew.
design-partition	Design rule checks which check design partitions.
dsp	Design rule checks related to DSP blocks inside the FPGA fabric.
false-positive-synchronizer	Design rule checks related to automatically-detected synchronizer chains that may have been over-zealously detected.

continued...

Tag	Description
global-signal	Design rule checks related to global signals.
impossible-requirements	Design rule checks which check the requirements on failing timing paths and flag those which fail by construction.
ip-parameterization	Design rule checks which look for parameterizable IP modules which may need to be adjusted to meet performance specifications.
intrinsic-margin	Design rule checks which use the Intrinsic Margin metric (slack ignoring cell delay, IC delay and clock skew) to diagnose potential timing issues on failing paths.
latch	Design rule checks related to latches.
logic-levels	Design rule checks which flag potentially problematic amounts of logic on a timing path.
minimum-pulse-width	Design rule checks related to minimum pulse width.
nonstandard-timing	Design rule checks related to topologies which have unique timing analysis methodologies and may prove problematic.
partial-reconfiguration	Design rule checks which check Partial Reconfiguration designs.
place	Design rule checks which pertain to the Compiler's Place stage.
project-settings	Design rule checks related to validating the project settings.
ram	Design rule checks related to M20k blocks inside the FPGA fabric.
region-constraints	Design rule checks related to region constraints in the design (both placement and routing).
register-duplication	Design rule checks related to duplication of registers in the design, either manually or automatically.
register-spread	Design rule checks related to measuring the spread of a register's sinks, as found in the "Report Register Spread" command.
reset-usage	Design rule checks related to safe resets or appropriate use of reset modes.
reset-reachability	Design rule checks related to reachability analysis of reset signals, including convergence of multiple reset signals.
resource-usage	Design rule checks related to managing the resource usage of the design.
retime	Design rule checks which pertain to the Compiler's Retime stage.
route	Design rule checks which pertain to the Compiler's Route stage.
sdc	Design rule checks related to SDC validity checking.
synchronizer	Design rule checks related to synchronizer chains.
synthesis	Design rule checks which pertain to the Compiler's Analysis & Synthesis stage.
system	Design rule checks which validate full-system design.

3.5.6. Design Assistant Rule Categories

Each Design Assistant rule has a unique alphanumeric ID that reflects the rule category. You can enable or disable Design Assistant rules for specific stages of compilation. The following lists all categories of Design Assistant rules, and provides a link to rule documentation in Quartus Prime Help. Some categories are device-specific.

Table 8. Design Assistant Rule Categories

Rule Category and Help Link	Acronym	Description
Block-based Design Flow Rules	BBD	Rule checks for block-based design flow issues, such as block preservation and reuse, periphery reuse, and partial reconfiguration issues.
Clock Domain Crossing Rules	CDC	Rule checks for clock domain crossings
Clock Rules	CLK	Rule checks for clocks
Floorplanning Rules	FLP	Rule checks for floorplanning, such as Logic Lock regions and congestion
IP Connectivity Rules	IPC	Rule checks for IP connectivity, such as a system clock frequency mismatch.
Linting Rules	LNT	Rules checks of source code for programmatic and stylistic errors
Project Rules	PRJ	Rule checks related to Quartus Prime projects
Reset Domain Crossing Rules	RDC	Rule checks for reset domain crossings
Reset Rules	RES	Rule checks for resets
Timing Closure Rules	TMC	Rule checks for timing closure

Related Information

Design Assistant Rules Help

3.6. Recommended Design Practices Revision History

The following revision history applies to this chapter:

Document Version	Quartus Prime Version	Changes
2025.04.14	25.1	• Applied Altera rebranding throughout as necessary. • Updated throughout for Agilex 3 support.
2024.12.11	24.3	• Removed broken link to BBD rules.
2024.09.30	24.3	• Updated the recommendation in *Design Assistant Rule Categories* for IPC.
2023.04.14	23.1	• Updated the recommendation in *Optimizing Clocking Schemes*.
2022.06.20	22.2	• Removed obsolete Block Based Design rule category from *Design Assistant Design Rule Checking* topic.
2021.10.04	21.3	• Updated *Design Assistant Design Rule Checking* topic for latest rule categories.
2021.03.29	21.1	• Updated *Design Assistant Design Rule Checking* topic with screenshot, minor wording changes, and link to *AN 919: Improving Quality of Results with Design Assistant*. • Updated *Setting Up Design Assistant* topic for rule tags. • Updated *Running Design Assistant During Compilation* step 4 wording. • Updated *Opening Design Assistant Rule Help* topic to show rule included in this release. • Updated screenshots in *Cross-Probing from Design Assistant to Timing Analyzer* topic.

continued...

Document Version	Quartus Prime Version	Changes
		• Removed *Filtering and Hiding Rule Violations* section as waivers are most effective method in current version. • Updated *Changing the Default Number of Violations per Rule* for minor wording changes. • Created new *Design Assistant Tags* topic to define all tags. • Created new *Design Assistant Waiver Dialog Box* topic to define all settings in this dialog box. including new waiver features. • Updated *Creating Design Assistant Waivers* topic for waiver preview and reporting. • Updated *Deleting Design Assistant Waivers* topic for new GUI method. • Updated *Design Assistant Rule Categories* topic for latest rule categories. • Revised *Design Assistant Rule Severity Levels* descriptions.
2020.09.28	20.3	• Added new "Waiving Design Assistant Rules" section. • Added new "Cross-Probing with Design Assistant" section. • Added description of new Fatal severity level Design Assistant rule to "Modifying Rule Severity Levels" topic. • Updated "Setting Up Design Assistant" for new Fatal severity level and RDC rule category. • Updated "Running Design Assistant in Analysis Mode" with screenshots and detailed steps. • Added RDC to "Design Assistant Rule Categories" topic. • Removed individual Design Assistant rule descriptions from document and linked to updated rule description in Help. • Replaced obsolete rule screenshots throughout.
2020.04.13	20.1	• Added support for Design Assistant during Timing Signoff to "Setting Up Design Assistant" and "Running Design Assistant During Compilation" topics. • Added new Design Assistant rules. • Revised Design Assistant rules HRR-10201, HRR-10201, and RES-30131. • Removed various obsolete Design Assistant rules.
2019.11.01	19.3.0	• Revised "Changing Design Assistant Rule Scope or Number of Violations" topic for clarity. • Created separate "Clock Domain Crossing (CDC) Rules" category and topic. • Added "CLK-30026: Missing Clock Assignment" Design Assistant rule. • Added "CLK-30027: Multiple Clock Assignment" Design Assistant rule. • Added "CLK-30028: Invalid Generated Clock" Design Assistant rule. • Added "CLK-30029: Invalid Clock Assignment" Design Assistant rule. • Added "CLK-30030: PLL Setting Violation" Design Assistant rule. • Added "CLK-30031: Input Delay Assigned to Clock" Design Assistant rule.
2019.09.30	19.3.0	• Added new "Setting Up Design Assistant" topic. • Added new "Managing Design Assistant Rules" topic. • Added new "Enabling Rules for Specific Compiler Stages" topic. • Added new "Specifying Rule Parameters for Specific Compiler Stages" topic. • Added new "Modifying Rule Severity Levels" topic. • Added new "Filtering and Hiding Rule Violations" topic. • Added new "Filter Options Dialog Box" topic. • Added new "Linking to Design Assistant Rule Documentation" topic.

continued...

Document Version	Quartus Prime Version	Changes
		• Updated screenshots for latest GUI elements. • Added the following new Design Assistant rules: ☐ CDC-50001: Missing 1-Bit Synchronizer ☐ CDC-50002: 1-Bit Synchronized Missing Constraint ☐ CDC-50003: CE-Type CDC No Constraints ☐ HRR-10015: High Fan-out Signal ☐ HRR-10201: Registers Cannot Power Up with Don't Care Logic Level ☐ HRR-10204: Reset Release Instance Count Check ☐ RES-30132: Registers May Not Be Properly Reset ☐ TMC-20500: Hierarchical Tree Duplication was Shallower than Possible ☐ TMC-20501: Hierarchical Tree Duplication was Shallower than Requested ☐ TMC-20550: Duplication Candidate Rejected for Placement Constraint ☐ TMC-20551: Automatically-Discovered Duplication Candidate Likely Requires More Duplication ☐ TMC-20552: User-Directed Duplication Candidate was Rejected ☐ TMC-20601: Registers with High Immediate Fan-Out Tension ☐ TMC-20602: Registers with High Timing Path Endpoint Tension ☐ TMC-20603: Registers with High Immediate Fan-Out Span ☐ TMC-20604: Registers with High Timing Path Endpoint Span • Removed obsolete Design Assistant rules: ☐ HRR-10014: High Fan-out Nets Driving Clock-Enable Pins ☐ HRR-10016: Registers Cannot Power-Up With Dont Care Logic Level
2019.04.01	19.1.0	• Described new Design Assistant design rule checking tool. • Added new topics describing each of the Design Assistant rules, under the *Recommended Design Practices > Checking for Design Rule Violations* section.
2018.09.24	18.1.0	• Created subtopics: *Clock Region Assignments in Intel Arria 10 and Older Device Families* and *Clock Region Assignments in Intel Stratix 10 Devices* from content in topic: *Use Clock Region Assignments to Optimize Clock Constraints*.
2017.11.06	17.1.0	• Updated topic: *Optimizing Timing Closure*. • Updated topic *Use Global Clock Network Resources* and added topic *Use Clock Region Assignments to Optimize Clock Constraints* for Intel Stratix 10 support.
2017.05.08	17.0.0	• Removed information about Integrated Synthesis. • Removed information about quartus_drc.
2016.10.31	16.1.0	• Implemented Intel rebranding.
2016.05.03	16.0.0	• Replaced Internally Synchronized Reset code sample with corrected version. • Removed information about deprecated physical synthesis options. • Removed information about unsupported Design Assistant.
2015.11.02	15.1.0	• Changed instances of *Quartus II* to *Quartus Prime*.
2014.12.15	14.1.0	Updated location of Fitter Settings, Analysis & Synthesis Settings, and Physical Optimization Settings to Compiler Settings.
June 2014	14.0.0	Removed references to obsolete MegaWizard Plug-In Manager.
November 2013	13.1.0	Removed HardCopy device information.
May 2013	13.0.0	Removed PrimeTime support.

continued...

Document Version	Quartus Prime Version	Changes
June 2012	12.0.0	Removed survey link.
November 2011	11.0.1	Template update.
May 2011	11.0.0	Added information to Reset Resources .
December 2010	10.1.0	• Title changed from Design Recommendations for Altera Devices and the Quartus II Design Assistant. • Updated to new template. • Added references to Quartus II Help for "Metastability" on page 9–13 and "Incremental Compilation" on page 9–13. • Removed duplicated content and added references to Help for "Custom Rules" on page 9–15.
July 2010	10.0.0	• Removed duplicated content and added references to Quartus II Help for Design Assistant settings, Design Assistant rules, Enabling and Disabling Design Assistant Rules, and Viewing Design Assistant reports. • Removed information from "Combinational Logic Structures" on page 5–4 • Changed heading from "Design Techniques to Save Power" to "Power Optimization" on page 5–12 • Added new "Metastability" section • Added new "Incremental Compilation" section • Added information to "Reset Resources" on page 5–23 • Removed "Referenced Documents" section
November 2009	9.1.0	• Removed documentation of obsolete rules.
March 2009	9.0.0	• No change to content.
November 2008	8.1.0	• Changed to 8-1/2 x 11 page size • Added new section "Custom Rules Coding Examples" on page 5–18 • Added paragraph to "Recommended Clock-Gating Methods" on page 5–11 • Added new section: "Design Techniques to Save Power" on page 5–12
May 2008	8.0.0	• Updated Figure 5–9 on page 5–13; added custom rules file to the flow • Added notes to Figure 5–9 on page 5–13 • Added new section: "Custom Rules Report" on page 5–34 • Added new section: "Custom Rules" on page 5–34 • Added new section: "Targeting Embedded RAM Architectural Features" on page 5–38 • Minor editorial updates throughout the chapter • Added hyperlinks to referenced documents throughout the chapter

4. Managing Metastability with the Quartus Prime Software

You can use the Quartus Prime software to analyze the average mean time between failures (MTBF) due to metastability caused by synchronization of asynchronous signals, and optimize the design to improve the metastability MTBF.

All registers in digital devices, such as FPGAs, have defined signal-timing requirements that allow each register to correctly capture data at its input ports and produce an output signal. To ensure reliable operation, the input to a register must be stable for a minimum amount of time before the clock edge (register setup time or t_{SU}) and a minimum amount of time after the clock edge (register hold time or t_H). The register output is available after a specified clock-to-output delay (t_{CO}).

If the data violates the setup or hold time requirements, the output of the register might go into a metastable state. In a metastable state, the voltage at the register output hovers at a value between the high and low states, which means the output transition to a defined high or low state is delayed beyond the specified t_{CO}. Different destination registers might capture different values for the metastable signal, which can cause the system to fail.

In synchronous systems, the input signals must always meet the register timing requirements, so that metastability does not occur. Metastability problems commonly occur when a signal is transferred between circuitry in unrelated or asynchronous clock domains, because the signal can arrive at any time relative to the destination clock.

The MTBF due to metastability is an estimate of the average time between instances when metastability could cause a design failure. A high MTBF (such as hundreds or thousands of years between metastability failures) indicates a more robust design. You should determine an acceptable target MTBF in the context of your entire system and taking in account that MTBF calculations are statistical estimates.

The metastability MTBF for a specific signal transfer, or all the transfers in a design, can be calculated using information about the design and the device characteristics. Improving the metastability MTBF for your design reduces the chance that signal transfers could cause metastability problems in your device.

The Quartus Prime software provides analysis, optimization, and reporting features to help manage metastability in Altera designs. These metastability features are supported only for designs constrained with the Quartus Prime Timing Analyzer. Both typical and worst-case MBTF values are generated for select device families.

4.1. Metastability Analysis in the Quartus Prime Software

When a signal transfers between circuitry in unrelated or asynchronous clock domains, the first register in the new clock domain acts as a synchronization register.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

To minimize the failures due to metastability in asynchronous signal transfers, circuit designers typically use a sequence of registers (a synchronization register chain or synchronizer) in the destination clock domain to resynchronize the signal to the new clock domain and allow additional time for a potentially metastable signal to resolve to a known value. Designers commonly use two registers to synchronize a new signal, but a standard of three registers provides better metastability protection.

The timing analyzer can analyze and report the MTBF for each identified synchronizer that meets its timing requirements, and can generate an estimate of the overall design MTBF. The software uses this information to optimize the design MTBF, and you can use this information to determine whether your design requires longer synchronizer chains.

Related Information

- Metastability and MTBF Reporting on page 128
- MTBF Optimization on page 131

4.1.1. Data Synchronization Register Chains

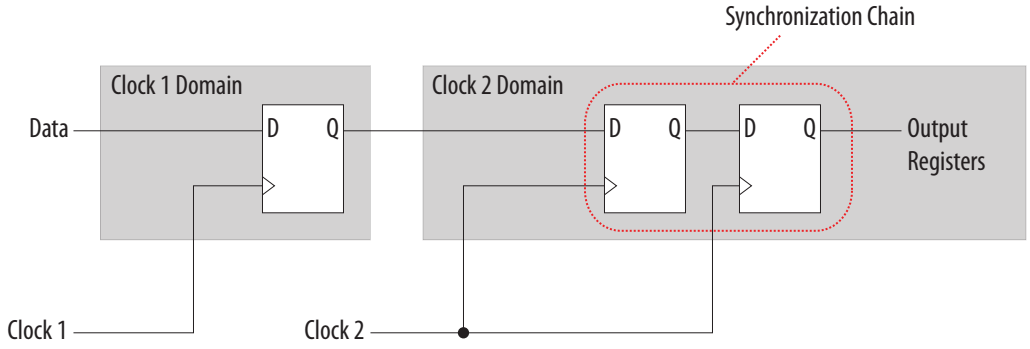
A synchronization register chain, or synchronizer, is defined as a sequence of registers that meets the following requirements:

- The registers in the chain are all clocked by the same clock or phase-related clocks.
- The first register in the chain is driven asynchronously or from an unrelated clock domain.
- Each register fans out to only one register, except the last register in the chain.

For Quartus Prime software to identify a synchronization register chain, the registers in the chain must not include any resets.

The length of the synchronization register chain is the number of registers in the synchronizing clock domain that meet the requirements listed earlier. The following figure shows a sample two-register synchronization chain.

Figure 47. Sample Synchronization Register Chain



The timing slack available in the register-to-register paths of the synchronizer allows a metastable signal to settle, and is referred to as the available settling time. The available settling time in the MTBF calculation for a synchronizer is the sum of the output timing slacks for each register in the chain. Adding available settling time with additional synchronization registers improves the metastability MTBF.

Related Information

- How Timing Constraints Affect Synchronizer Identification and Metastability Analysis on page 127
- Force the Identification of Synchronization Registers on page 133
- Identify Synchronizers for Metastability Analysis on page 127

4.1.2. Identify Synchronizers for Metastability Analysis

The first step in enabling metastability MTBF analysis and optimization in the Quartus Prime software is to identify which registers are part of a synchronization register chain. You can apply synchronizer identification settings globally to automatically list possible synchronizers with the **Synchronizer identification** option on the **Timing Analyzer** page in the **Settings** dialog box.

Synchronization chains are already identified within most Altera FPGA intellectual property (IP) cores.

Related Information

Force the Identification of Synchronization Registers on page 133

4.1.3. How Timing Constraints Affect Synchronizer Identification and Metastability Analysis

The timing analyzer can analyze metastability MTBF only if a synchronization chain meets its timing requirements. The metastability failure rate depends on the timing slack available in the synchronizer's register-to-register connections, because that slack is the available settling time for a potential metastable signal. Therefore, you must ensure that your design is correctly constrained with the real application frequency requirements to get an accurate MTBF report.

In addition, the **Auto** and **Forced If Asynchronous** synchronizer identification options use timing constraints to automatically detect the synchronizer chains in the design. These options check for signal transfers between circuitry in unrelated or asynchronous clock domains, so clock domains must be related correctly with timing constraints.

The timing analyzer views input ports as asynchronous signals unless they are associated correctly with a clock domain. If an input port fans out to registers that are not acting as synchronization registers, apply a `set_input_delay` constraint to the input port; otherwise, the input register might be reported as a synchronization register. Constraining a synchronous input port with a `set_max_delay` constraint for a setup (t_{SU}) requirement does not prevent synchronizer identification because the constraint does not associate the input port with a clock domain.

Instead, use the following command to specify an input setup requirement associated with a clock:

`set_input_delay -max -clock` <*clock name*> <*latch – launch – tsu requirement*> <*input port name*>

Registers that are at the end of false paths are also considered synchronization registers because false paths are not timing-analyzed. Because there are no timing requirements for these paths, the signal may change at any point, which may violate the t_{SU} and t_H of the register. Therefore, these registers are identified as synchronization registers. If these registers are not used for synchronization, you can turn off synchronizer identification and analysis. To do so, set **Synchronizer Identification** to **Off** for the first synchronization register in these register chains.

4.2. Metastability and MTBF Reporting

The Quartus Prime software reports the metastability analysis results in the Compilation Report and Timing Analyzer reports.

The MTBF calculation uses timing and structural information about the design, silicon characteristics, and operating conditions, along with the data toggle rate.

If you change the **Synchronizer Identification** settings, you can generate new metastability reports by rerunning the timing analyzer. However, you should rerun the Fitter first so that the registers identified with the new setting can be optimized for metastability MTBF.

Related Information

- Metastability Reports on page 128
- MTBF Optimization on page 131
- Synchronizer Data Toggle Rate in MTBF Calculation on page 130

4.2.1. Metastability Reports

Metastability reports summarize the results of the metastability analysis. In addition to the MTBF Summary and Synchronizer Summary reports, the Timing Analyzer tool reports additional statistics for each synchronizer chain.

If the design uses only the **Auto Synchronizer Identification** setting, the reports list likely synchronizers but do not report MTBF. To obtain an MTBF for each register chain you must force identification of synchronization registers.

If the synchronizer chain does not meet its timing requirements, the reports list identified synchronizers but do not report MTBF. To obtain MTBF calculations, ensure that the design is constrained correctly, and that the synchronizer meets its timing requirements.

Related Information

- Identify Synchronizers for Metastability Analysis on page 127
- How Timing Constraints Affect Synchronizer Identification and Metastability Analysis on page 127

4.2.1.1. MTBF Summary Report

The MTBF Summary reports an estimate of the overall robustness of cross-clock domain and asynchronous transfers in the design. This estimate uses the MTBF results of all synchronization chains in the design to calculate an MTBF for the entire design.

4.2.1.1.1. Typical and Worst-Case MTBF of Design

The MTBF Summary Report shows the **Typical MTBF of Design** and the **Worst-Case MTBF of Design** for supported fully-characterized devices. The typical MTBF result assumes typical conditions, defined as nominal silicon characteristics for the selected device speed grade, as well as nominal operating conditions. The worst-case MTBF result uses the worst case silicon characteristics for the selected device speed grade.

When you analyze multiple timing corners in the timing analyzer, the MTBF calculation may vary because of changes in the operating conditions, and the timing slack or available metastability settling time. Altera recommends running multi-corner timing analysis to ensure that you analyze the worst MTBF results, because the worst timing corner for MTBF does not necessarily match the worst corner for timing performance.

Related Information

Timing Analyzer
In *Quartus Prime Help*

4.2.1.1.2. Synchronizer Chains

The MTBF Summary report also lists the **Number of Synchronizer Chains Found** and the length of the **Shortest Synchronizer Chain**, which can help you identify whether the report is based on accurate information.

If the number of synchronizer chains found is different from what you expect, or if the length of the shortest synchronizer chain is less than you expect, you might have to add or change **Synchronizer Identification** settings for the design. The report also provides the **Worst Case Available Settling Time**, defined as the available settling time for the synchronizer with the worst MTBF.

You can use the reported **Fraction of Chains for which MTBFs Could Not be Calculated** to determine whether a high proportion of chains are missing in the metastability analysis. A fraction of 1, for example, means that MTBF could not be calculated for any chains in the design. MTBF is not calculated if you have not identified the chain with the appropriate **Synchronizer identification** option, or if paths are not timing-analyzed and therefore have no valid slack for metastability analysis. You might have to correct your timing constraints to enable complete analysis of the applicable register chains.

4.2.1.1.3. Increasing Available Settling Time

The MTBF Summary report specifies how an increase of 100ps in available settling time increases the MTBF values. If your MTBF is not satisfactory, this metric can help you determine how much extra slack would be required in your synchronizer chain to allow you to reach the desired design MTBF.

4.2.1.2. Synchronizer Summary Report

The **Synchronizer Summary** lists the synchronization register chains detected in the design depending on the Synchronizer Identification setting.

The **Source Node** is the register or input port that is the source of the asynchronous transfer. The **Synchronization Node** is the first register of the synchronization chain. The **Source Clock** is the clock domain of the source node, and the **Synchronization Clock** is the clock domain of the synchronizer chain.

This summary reports the calculated **Worst-Case MTBF**, if available, and the **Typical MTBF**, for each appropriately identified synchronization register chain that meets its timing requirement.

Related Information

Synchronizer Chain Statistics Report in the Timing Analyzer on page 130

4.2.1.3. Synchronizer Chain Statistics Report in the Timing Analyzer

The timing analyzer provides an additional report for each synchronizer chain.

The **Chain Summary** tab matches the Synchronizer Summary information described in the Synchronizer Summary Report, while the **Statistics** tab adds more details. These details include whether the **Method of Synchronizer Identification** was **User Specified** (with the **Forced if Asynchronous** or **Forced** settings for the **Synchronizer Identification** setting), or **Automatic** (with the **Auto** setting). The **Number of Synchronization Registers in Chain** report provides information about the parameters that affect the MTBF calculation, including the **Available Settling Time** for the chain and the **Data Toggle Rate Used in MTBF Calculation**.

The following information is also included to help you locate the chain in your design:

- **Source Clock** and **Asynchronous Source** node of the signal.
- **Synchronization Clock** in the destination clock domain.
- Node names of the **Synchronization Registers** in the chain.

Related Information

Synchronizer Data Toggle Rate in MTBF Calculation on page 130

4.2.2. Synchronizer Data Toggle Rate in MTBF Calculation

The MTBF calculations assume the data being synchronized is switching at a toggle rate of 12.5% of the source clock frequency. That is, the arriving data is assumed to switch once every eight source clock cycles.

If multiple clocks apply, the highest frequency is used. If no source clocks can be determined, the data rate is taken as 12.5% of the synchronization clock frequency.

To help ensure accurate MTBF analysis for reset synchronizers, explicitly specify the `toggle_rate` for reset synchronizers. If you know an approximate rate at which the data changes, specify it with the **Synchronizer Toggle Rate** logic option assignment in the Assignment Editor.

Note: Starting in Quartus Prime Pro Edition software version 25.1, the **Synchronizer Toggle Rate** logic option assignment is now double type (rather than integer), allowing input of decimal values.

You can also apply this assignment to an entity or the entire design. Set the data toggle rate, in number of transitions per second, on the first register of a synchronization chain. The timing analyzer takes the specified rate into account when computing the MTBF of that register chain. If a data signal never toggles and does not affect the reliability of the design, you can set the **Synchronizer Toggle Rate** to **0** for the synchronization chain so the MTBF is not reported. To apply the assignment with Tcl, use the following command:

```
set_instance_assignment -name SYNCHRONIZER_TOGGLE_RATE <toggle rate in
transitions/second> -to <register name>
```

In addition to **Synchronizer Toggle Rate,** there are two other assignments associated with toggle rates, which are not used for metastability MTBF calculations. The I/O Maximum Toggle Rate is only used for pins, and specifies the worst-case toggle rates used for signal integrity purposes. The Power Toggle Rate assignment is used to specify the expected time-averaged toggle rate, and is used by the Power Analyzer to estimate time-averaged power consumption.

4.3. MTBF Optimization

In addition to reporting synchronization register chains and MTBF values found in the design, the Quartus Prime software can also protect these registers from optimizations that might negatively impact MTBF and can optimize the register placement and routing if the MTBF is too low.

Synchronization register chains must first be explicitly identified as synchronizers. Altera recommends that you set **Synchronizer Identification** to **Forced If Asynchronous** for all registers that are part of a synchronizer chain.

Optimization algorithms, such as register duplication and logic retiming in physical synthesis, are not performed on identified synchronization registers. The Fitter protects the number of synchronization registers specified by the **Synchronizer Register Chain Length** option.

In addition, the Fitter optimizes identified synchronizers for improved MTBF by placing and routing the registers to increase their output setup slack values. Adding slack in the synchronizer chain increases the available settling time for a potentially metastable signal, which improves the chance that the signal resolves to a known value, and exponentially increases the design MTBF. The Fitter optimizes the number of synchronization registers specified by the **Synchronizer Register Chain Length** option.

Metastability optimization is **on** by default. To view or change the **Optimize Design for Metastability** option, click **Assignments ➤ Settings ➤ Compiler Settings ➤ Advanced Settings (Fitter)**. To turn the optimization on or off with Tcl, use the following command:

```
set_global_assignment -name OPTIMIZE_FOR_METASTABILITY <ON|OFF>
```

Related Information

Identify Synchronizers for Metastability Analysis on page 127

4.3.1. Synchronization Register Chain Length

The **Synchronization Register Chain Length** option specifies how many registers should be protected from optimizations that might reduce MTBF for each register chain, and controls how many registers should be optimized to increase MTBF with the **Optimize Design for Metastability** option.

For example, if the **Synchronization Register Chain Length** option is set to **2**, optimizations such as register duplication or logic retiming are prevented from being performed on the first two registers in all identified synchronization chains. The first two registers are also optimized to improve MTBF when the **Optimize Design for Metastability** option is turned on.

The default setting for the **Synchronization Register Chain Length** option is **3**. The first register of a synchronization chain is always protected from operations that might reduce MTBF, but you should set the protection length to protect more of the synchronizer chain. Altera recommends that you set this option to the maximum length of synchronization chains you have in your design so that all synchronization registers are preserved and optimized for MTBF.

Click **Assignments ➤ Settings ➤ Compiler Settings ➤ Advanced Settings (Synthesis)** to change the global **Synchronization Register Chain Length** option.

You can also set the **Synchronization Register Chain Length** on a node or an entity in the Assignment Editor. You can set this value on the first register in a synchronization chain to specify how many registers to protect and optimize in this chain. This individual setting is useful if you want to protect and optimize extra registers that you have created in a specific synchronization chain that has low MTBF, or optimize less registers for MTBF in a specific chain where the maximum frequency or timing performance is not being met. To make the global setting with Tcl, use the following command:

```
set_global_assignment -name SYNCHRONIZATION_REGISTER_CHAIN_LENGTH <number of registers>
```

To apply the assignment to a design instance or the first register in a specific chain with Tcl, use the following command:

```
set_instance_assignment -name SYNCHRONIZATION_REGISTER_CHAIN_LENGTH <number of registers> -to <register or instance name>
```

4.4. Reducing Metastability Effects

You can check your design's metastability MTBF in the Metastability Summary report, and determine an acceptable target MTBF given the context of your entire system and the fact that MTBF calculations are statistical estimates. A high metastability MTBF (such as hundreds or thousands of years between metastability failures) indicates a more robust design.

This section provides guidelines to ensure complete and accurate metastability analysis, and some suggestions to follow if the Quartus Prime metastability reports calculate an unacceptable MTBF value.

Related Information

Metastability Reports on page 128

4.4.1. Apply Complete System-Centric Timing Constraints for the Timing Analyzer

To enable the Quartus Prime metastability features, make sure that the timing analyzer is used for timing analysis.

Ensure that the design is fully timing constrained and that it meets its timing requirements. If the synchronization chain does not meet its timing requirements, MTBF cannot be calculated. If the clock domain constraints are set up incorrectly, the signal transfers between circuitry in unrelated or asynchronous clock domains might be identified incorrectly.

Use industry-standard system-centric I/O timing constraints instead of using FPGA-centric timing constraints.

You should use `set_input_delay` constraints in place of `set_max_delay` constraints to associate each input port with a clock domain to help eliminate false positives during synchronization register identification.

Related Information

How Timing Constraints Affect Synchronizer Identification and Metastability Analysis on page 127

4.4.2. Force the Identification of Synchronization Registers

Use the guidelines in *Identifying Synchronizers for Metastability Analysis* to ensure the software reports and optimizes the appropriate register chains.

Identify synchronization registers by setting **Synchronizer Identification** to **Forced If Asynchronous** in the Assignment Editor. If there are any registers that the software detects as synchronous, but you want to analyze for metastability, apply the **Forced** setting to the first synchronizing register. Set **Synchronizer Identification** to **Off** for registers that are not synchronizers for asynchronous signals or unrelated clock domains.

To help you find the synchronizers in your design, you can set the global **Synchronizer Identification** setting on the **Timing Analyzer** page of the **Settings** dialog box to **Auto** to generate a list of all the possible synchronization chains in your design.

Related Information

Identify Synchronizers for Metastability Analysis on page 127

4.4.3. Set the Synchronizer Data Toggle Rate

The MTBF calculations assume the data being synchronized is switching at a toggle rate of 12.5% of the source clock frequency.

To help ensure accurate MTBF analysis for reset synchronizers, specify the `toggle_rate` for all reset synchronizers in your design. If you know an approximate rate at which the data changes, you can specify this with the **Synchronizer Toggle Rate** logic option assignment in the Assignment Editor.

Note: Starting in Quartus Prime Pro Edition software version 25.1, the **Synchronizer Toggle Rate** logic option assignment is now double type (rather than integer), allowing input of decimal values.

Related Information

Synchronizer Data Toggle Rate in MTBF Calculation on page 130

4.4.4. Optimize Metastability During Fitting

Ensure that the **Optimize Design for Metastability** setting is turned on.

Related Information

MTBF Optimization on page 131

4.4.5. Increase the Length of Synchronizers to Protect and Optimize

Increase the Synchronizer Chain Length parameter to the maximum length of synchronization chains in your design. If you have synchronization chains longer than 2 identified in your design, you can protect the entire synchronization chain from operations that might reduce MTBF and allow metastability optimizations to improve the MTBF.

Related Information

Synchronization Register Chain Length on page 132

4.4.6. Increase the Number of Stages Used in Synchronizers

Designers commonly use two registers in a synchronization chain to minimize the occurrence of metastable events, and a standard of three registers provides better metastability protection. However, synchronization chains with two or even three registers may not be enough to produce a high enough MTBF when the design runs at high clock and data frequencies.

If a synchronization chain is reported to have a low MTBF, consider adding an additional register stage to your synchronization chain. This additional stage increases the settling time of the synchronization chain, allowing more opportunity for the signal to resolve to a known state during a metastable event. Additional settling time increases the MTBF of the chain and improves the robustness of your design. However, adding a synchronization stage introduces an additional stage of latency on the signal.

If you use the Altera IP core with separate read and write clocks to cross clock domains, increase the metastability protection (and latency) for better MTBF. In the DCFIFO parameter editor, choose the **Best metastability protection, best fmax, unsynchronized clocks** option to add three or more synchronization stages. You can increase the number of stages to more than three using the **How many sync stages?** setting.

4.4.7. Select a Faster Speed Grade Device

The design MTBF depends on process parameters of the device used. Faster devices are less susceptible to metastability issues. If the design MTBF falls significantly below the target MTBF, switching to a faster speed grade can improve the MTBF substantially.

4.5. Scripting Support

You can run procedures and make settings described in this chapter in a Tcl script. You can also run procedures at a command prompt.

For detailed information about scripting command options, refer to the Quartus Prime Command-Line and Tcl API Help browser. To run the Help browser, type the following command at the command prompt and then press Enter:

```
quartus_sh --qhelp
```

Related Information

- Quartus Prime Pro Edition Settings File Reference Manual
- Quartus Prime Pro Edition User Guide: Scripting

4.5.1. Identifying Synchronizers for Metastability Analysis

To apply the global Synchronizer Identification assignment, use the following command:

```
set_global_assignment -name SYNCHRONIZER_IDENTIFICATION <OFF|AUTO|"FORCED IF
ASYNCHRONOUS">
```

To apply the **Synchronizer Identification** assignment to a specific register or instance, use the following command:

```
set_instance_assignment -name SYNCHRONIZER_IDENTIFICATION <AUTO|"FORCED IF
ASYNCHRONOUS"|FORCED|OFF> -to <register or instance name>
```

4.5.2. Synchronizer Data Toggle Rate in MTBF Calculation

To help ensure accurate MTBF analysis for reset synchronizers, explicitly specify the `toggle_rate` for all reset synchronizers in your design. If you know an approximate rate at which the data changes, you can specify this with the **Synchronizer Toggle Rate** logic option assignment in the Assignment Editor.

Note: Starting in Quartus Prime Pro Edition software version 25.1, the **Synchronizer Toggle Rate** logic option assignment is now double type (rather than integer), allowing input of decimal values.

To specify a toggle rate for MTBF calculations as described on page "R**Synchronizer Data Toggle Rate in MTBF Calculation", use the following command:

```
set_instance_assignment -name SYNCHRONIZER_TOGGLE_RATE <toggle rate in
transitions/second> -to <register name>
```

Related Information

Synchronizer Data Toggle Rate in MTBF Calculation on page 130

4.5.3. report_metastability and Tcl Command

If you use a command-line or scripting flow, you can generate the metastability analysis reports described in "C**Metastability Reports" outside of the Quartus Prime and user interfaces.

The table describes the options for the `report_metastability` and Tcl command.

Table 9. report_metastabilty Command Options

Option	Description
`-append`	If output is sent to a file, this option appends the result to that file. Otherwise, the file is overwritten.
`-file` <name>	Sends the results to an ASCII or HTML file. The extension specified in the file name determines the file type — either ***.txt** or ***.html**.
`-panel_name` <name>	Sends the results to the panel and specifies the name of the new panel.
`-stdout`	Indicates the report be sent to the standard output, via messages. This option is required only if you have selected another output format, such as a file, and would also like to receive messages.

Related Information

Metastability Reports on page 128

4.5.4. MTBF Optimization

To ensure that metastability optimization described on page "C**MTBF Optimization" is turned on (or to turn it off), use the following command:

```
set_global_assignment -name OPTIMIZE_FOR_METASTABILITY <ON|OFF>
```

Related Information

MTBF Optimization on page 131

4.5.5. Synchronization Register Chain Length

To globally set the number of registers in a synchronization chain to be protected and optimized as described on page "C**Synchronization Register Chain Length", use the following command:

```
set_global_assignment -name SYNCHRONIZATION_REGISTER_CHAIN_LENGTH <number of registers>
```

To apply the assignment to a design instance or the first register in a specific chain, use the following command:

```
set_instance_assignment -name SYNCHRONIZATION_REGISTER_CHAIN_LENGTH <number of registers> -to <register or instance name>
```

Related Information

Synchronization Register Chain Length on page 132

4.6. Managing Metastability

Altera's Quartus Prime software provides industry-leading analysis and optimization features to help you manage metastability in your FPGA designs. Set up your Quartus Prime project with the appropriate constraints and settings to enable the software to

analyze, report, and optimize the design MTBF. Take advantage of these features in the Quartus Prime software to make your design more robust with respect to metastability.

4.7. Managing Metastability with the Quartus Prime Software Revision History

The following revisions history applies to this chapter:

Document Version	Quartus Prime Version	Changes
2025.04.14	25.1	• Applied Altera rebranding throughout as necessary. • Updated throughout for Agilex 3 support. • Updated *Synchronizer Data Toggle Rate in MTBF Calculation* topic for Synchronizer Toggle Rate logic option change. • Updated *Set the Synchronizer Data Toggle Rate* topic for Synchronizer Toggle Rate logic option change.
2022.03.28	22.1	• Renamed "Synchronization Register Chains" to "Data Synchronization Register Chains" and revised the topic. • Removed references to obsolete Advisors.
2018.05.07	18.0.0	• First release as part of the stand-alone *Design Recommendations User Guide*
2017.11.06	17.1.0	• Corrected broken links to other documents.
2016.10.31	16.1.0	• Implemented Intel rebranding.
2015.11.02	15.1.0	• Changed instances of *Quartus II* to *Quartus Prime*.
2014.12.15	14.1.0	Updated location of Fitter Settings, Analysis & Synthesis Settings, and Physical Optimization Settings to Compiler Settings.
June 2014	14.0.0	Updated formatting.
June 2012	12.0.0	Removed survey link.
November 2011	10.0.2	Template update.
December 2010	10.0.1	Changed to new document template.
July 2010	10.0.0	Technical edit.
November 2009	9.1.0	Clarified description of synchronizer identification settings. Minor changes to text and figures throughout document.
March 2009	9.0.0	Initial release.

UG-20131 | 2025.04.14

Send Feedback

5. Quartus Prime Pro Edition User Guide: Design Recommendations Archive

For the latest and previous versions of this user guide, refer to *Quartus Prime Pro Edition User Guide: Design Recommendations*. If a software version is not listed, the user guide for the previous software version applies.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

A. Quartus Prime Pro Edition User Guides

Refer to the following user guides for comprehensive information on all phases of the Quartus Prime Pro Edition FPGA design flow.

Related Information

- Quartus Prime Pro Edition User Guide: Getting Started
  Introduces the basic features, files, and design flow of the Quartus Prime Pro Edition software, including managing Quartus Prime Pro Edition projects and IP, initial design planning considerations, and project migration from previous software versions.
- Quartus Prime Pro Edition User Guide: Platform Designer
  Describes creating and optimizing systems using Platform Designer, a system integration tool that simplifies integrating customized IP cores in your project. Platform Designer automatically generates interconnect logic to connect intellectual property (IP) functions and subsystems.
- Quartus Prime Pro Edition User Guide: Design Recommendations
  Describes best design practices for designing FPGAs with the Quartus Prime Pro Edition software. HDL coding styles and synchronous design practices can significantly impact design performance. Following recommended HDL coding styles ensures that Quartus Prime Pro Edition synthesis optimally implements your design in hardware.
- Quartus Prime Pro Edition User Guide: Design Compilation
  Describes set up, running, and optimization for all stages of the Quartus Prime Pro Edition Compiler. The Compiler synthesizes, places, and routes your design before generating a device programming file.
- Quartus Prime Pro Edition User Guide: Design Optimization
  Describes Quartus Prime Pro Edition settings, tools, and techniques that you can use to achieve the highest design performance in Altera FPGAs. Techniques include optimizing the design netlist, addressing critical chains that limit retiming and timing closure, optimizing device resource usage, device floorplanning, and implementing engineering change orders (ECOs).
- Quartus Prime Pro Edition User Guide: Programmer
  Describes operation of the Quartus Prime Pro Edition Programmer, which allows you to configure Altera FPGA devices, and program CPLD and configuration devices, via connection with an Altera FPGA download cable.
- Quartus Prime Pro Edition User Guide: Block-Based Design
  Describes block-based design flows, also known as modular or hierarchical design flows. These advanced flows enable preservation of design blocks (or logic that comprises a hierarchical design instance) within a project, and reuse of design blocks in other projects.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

- Quartus Prime Pro Edition User Guide: Partial Reconfiguration
  Describes Partial Reconfiguration, an advanced design flow that allows you to reconfigure a portion of the FPGA dynamically, while the remaining FPGA design continues to function. Define multiple personas for a particular design region, without impacting operation in other areas.
- Quartus Prime Pro Edition User Guide: Third-party Simulation
  Describes RTL- and gate-level design simulation support for third-party simulation tools by Aldec*, Cadence*, Siemens EDA, and Synopsys* that allow you to verify design behavior before device programming. Includes simulator support, simulation flows, and simulating Altera IP.
- Quartus Prime Pro Edition User Guide: Third-party Synthesis
  Describes support for optional synthesis of your design in third-party synthesis tools by Siemens EDA, and Synopsys*. Includes design flow steps, generated file descriptions, and synthesis guidelines.
- Quartus Prime Pro Edition User Guide: Third-party Logic Equivalence Checking Tools
  Describes support for optional logic equivalence checking (LEC) of your design in third-party LEC tools by OneSpin*.
- Quartus Prime Pro Edition User Guide: Debug Tools
  Describes a portfolio of Quartus Prime Pro Edition in-system design debugging tools for real-time verification of your design. These tools provide visibility by routing (or "tapping") signals in your design to debugging logic. These tools include System Console, Signal Tap logic analyzer, system debugging toolkits, In-System Memory Content Editor, and In-System Sources and Probes Editor.
- Quartus Prime Pro Edition User Guide: Timing Analyzer
  Explains basic static timing analysis principals and use of the Quartus Prime Pro Edition Timing Analyzer, a powerful ASIC-style timing analysis tool that validates the timing performance of all logic in your design using an industry-standard constraint, analysis, and reporting methodology.
- Quartus Prime Pro Edition User Guide: Power Analysis and Optimization
  Describes the Quartus Prime Pro Edition Power Analysis tools that allow accurate estimation of device power consumption. Estimate the power consumption of a device to develop power budgets and design power supplies, voltage regulators, heat sink, and cooling systems.
- Quartus Prime Pro Edition User Guide: Design Constraints
  Describes timing and logic constraints that influence how the Compiler implements your design, such as pin assignments, device options, logic options, and timing constraints. Use the Interface Planner to prototype interface implementations, plan clocks, and quickly define a legal device floorplan. Use the Pin Planner to visualize, modify, and validate all I/O assignments in a graphical representation of the target device.
- Quartus Prime Pro Edition User Guide: PCB Design Tools
  Describes support for optional third-party PCB design tools by Siemens EDA and Cadence*. Also includes information about signal integrity analysis and simulations with HSPICE and IBIS Models.
- Quartus Prime Pro Edition User Guide: Scripting
  Describes use of Tcl and command line scripts to control the Quartus Prime Pro Edition software and to perform a wide range of functions, such as managing projects, specifying constraints, running compilation or timing analysis, or generating reports.